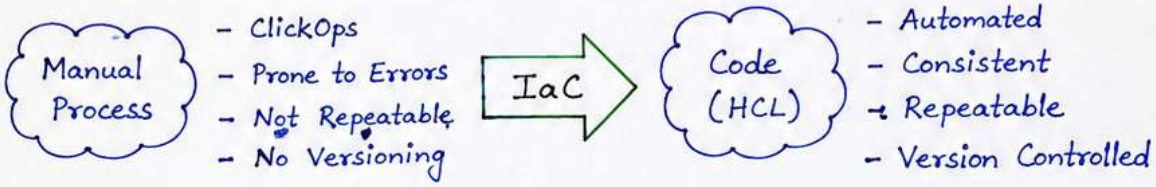
* 1. What is Infrastructure as Code (IaC) ?

- Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure through code instead of manual processes.
- We define, deploy, update and destroy infrastructure using code (usually in HCL, YAML, JSON).
- IaC helps us to build repeatable, Consistent and version controlled environments.



* 2. What is Terraform ?

- Terraform is an open-source Infrastructure as Code (IaC) tool created by HashiCorp.
- It allows us to define and provision infrastructure across multiple cloud providers using a declarative configuration Language called HCL (HashiCorp Configuration Language).
- Terraform maintains the state of our infrastructure and makes changes in a safe and predictable way.

Popular Providers:

AWS, Azure, GCP, Kubernetes, VMware, Docker, GitHub, etc.

* 3. Why Terraform ?

- ☑ Cloud Agnostic
- ☑ Simple & Declarative (HCL)
- ☑ Reusable (Modules)
- ☑ State Management
- ☑ Execution Plan (Plan before apply)
- ☑ Drift Detection
- ☑ Strong Community Support
- ☑ Integration with CI/CD
- ☑ Cost Effective
- ☑ Multi-Environment Support
- ☑ Version Controlled
- ☑ Safe Changes (Rollback Possible)
- ☑ Supports 1000+ Providers

Real-time Example:

In our project, we use Terraform to provision AWS VPC, EKS, RDS, IAM roles, S3 buckets, Security Groups etc. All infra is managed through Git + Terraform.

Note: IaC is not just about provisioning, it's about managing the entire infrastructure lifecycle in a repeatable and reliable way.

Engineer_cloudops

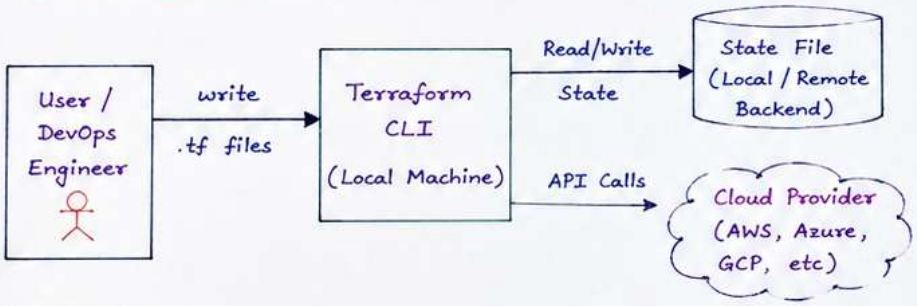
4. Terraform vs CloudFormation

Feature	Terraform	AWS CloudFormation
Provider	Multi-Cloud (AWS, Azure, GCP, etc)	AWS Only
Language	HCL (HashiCorp Configuration Language)	JSON / YAML
State Management	Maintains State file	Stack based, state managed by AWS
Infrastructure Changes	Detects drift using "plan"	Drift detection available but manual
Modularity	Modules (Reusable)	Nested Stacks
Resource Coverage	1000+ Providers	AWS Native Resources
Learning Curve	Easy to learn	Slightly Complex (JSON)
Version Control	Easy (code + state)	Template based
Best For	Multi-Cloud, DevOps teams, Automation	AWS Native, Simple AWS infra

Notes:

- Terraform is cloud agnostic and flexible.
- CloudFormation is tightly coupled with AWS.
- Terraform provides better automation and DRY approach with modules.
- In real-time we use Terraform for multi-cloud or hybrid environments.

5. Terraform Architecture



Components:

1. Configuration Files (.tf)
2. Terraform CLI
3. State File (Local/Remote)
4. Providers (AWS, Azure...)
5. Resources (Real Infra)

Key Points:

- Terraform CLI is the core engine.
- It reads .tf files, maintains state and talks to cloud APIs.
- State file keeps track of real infrastructure.

6. Terraform Workflow



Workflow Details:

1. terraform init → Downloads providers, initializes backend.
2. terraform plan → Shows what will be created/changed/destroyed.
3. terraform apply → Applies the changes (asks for confirmation).
4. terraform destroy → Destroys all managed infrastructure.

> Always run "plan" before "apply" in Production.
> Use Remote State + Locking in team environments.

Engineer_cloudops

7. Terraform Installation

A. On macOS (using Homebrew)

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

B. On Ubuntu / Debian

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y gnupg software-properties-common
wget -O- https://apt.releases.hashicorp.com/gpg | sudo apt-key add -
sudo apt-add-repository "deb [arch=amd64] https://apt.releases.hashicorp.com \
                         $(lsb_release -cs) main"
sudo apt update
sudo apt install terraform
```

C. Verify Installation

```
terraform -v
```

Notes:

- Terraform is a single binary written in Go.
- Works on Windows, macOS, Linux.
- No extra dependencies.

Tip:

Always use latest stable version in local & keep the version same in team (use .terraform-version file with tfenv).

8. Terraform CLI Commands

Command	Purpose	Example
terraform init	Initializes working directory, downloads providers & prepares backend.	terraform init
terraform validate	Validates the configuration files.	terraform validate
terraform fmt	Formats the code to standard style.	terraform fmt -recursive
terraform plan	Creates an execution plan, shows what will be created/changed.	terraform plan -out=plan.tfplan
terraform apply	Applies the changes required to reach desired state.	terraform apply plan.tfplan or terraform apply
terraform destroy	Destroys all managed infrastructure.	terraform destroy
terraform show	Shows the current state or plan details.	terraform show
terraform output	Displays output values from the root module.	terraform output
terraform state list	Lists all resources in the state.	terraform state list
terraform state show <res>	Shows details of a specific resource from state.	terraform state show aws_instance.web
terraform import	Imports existing infra into state.	terraform import aws_s3_bucket.mybucket my-bucket
terraform taint	Marks a resource for recreation.	terraform taint aws_instance.web
terraform untaint	Removes taint mark.	terraform untaint aws_instance.web
terraform refresh	Updates the state file with real infra.	terraform refresh
terraform workspace	Manages workspaces.	terraform workspace list

Common Options:

- -auto-approve Apply without confirmation.
- -input=false Non-interactive mode.
- -var-file= Load variables from a file.
- -out= Save plan to a file.

9. Terraform Project Structure (Best Practice)

```
my-terraform-project/
├── envs/
│   ├── dev/
│   │   ├── main.tf
│   │   └── terraform.tfvars
│   ├── stage/
│   │   ├── main.tf
│   │   └── terraform.tfvars
│   └── prod/
│       ├── main.tf
│       └── terraform.tfvars
├── modules/
│   ├── vpc/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── ec2/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
├── providers.tf
├── backend.tf
├── variables.tf
├── outputs.tf
└── README.md
```

Structure Explained:

- envs/ : Environment specific configurations (dev, stage, prod).
- modules/ : Reusable modules (VPC, EC2, RDS, etc).
- providers.tf : Define providers & versions.
- backend.tf : Remote state configuration.
- variables.tf : Declare input variables.
- outputs.tf : Define output values.
- README.md : Project documentation.

Best Practices:

- Use remote backend (S3 + DynamoDB).
- Keep modules reusable and small.
- Use variables and outputs properly.
- Follow consistent naming & formatting.
- Always run terraform fmt and terraform validate before committing code.

Engineer_cloudops

Module 2 : Terraform Language Basics

1. Providers

- Providers are plugins that allow Terraform to communicate with cloud platforms or other APIs.
- They define the API endpoints and authentication details.
- We can configure multiple providers in a single configuration.

Example : AWS Provider

```
provider "aws" {
  region  = "ap-south-1"
  profile = "my-profile"
  # shared_credentials_file = "~/.aws/credentials"
}
```

Common Providers

- aws - Amazon Web Services
- azurerm - Microsoft Azure
- google - Google Cloud Platform
- kubernetes - Kubernetes Cluster
- random, local, time, null - Utility Providers

Notes :

- Every terraform configuration must have at least one provider.
- Providers can be aliased when we work with multi regions or accounts.

2. Resources

- Resources are the most important building blocks in Terraform.
- They represent infrastructure objects like EC2, VPC, S3, RDS, etc.
- Each resource has a type and a name.
- Terraform creates, updates, and deletes real infrastructure based on resource blocks.

Example : Create an AWS EC2 Instance

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  key_name      = "my-key"

  tags = {
    Name        = "WebServer"
    Environment = "Dev"
  }
}
```

Key Points :

- Format :
  ```
  resource "TYPE" "NAME" {
    arguments...
  }
  ```
- NAME is local to the configuration.
- Changes in resource are tracked in state file.

Common Resource Syntax :

```
resource "provider_resource_type" "resource_name" { ... }
```

3. Data Sources

- Data sources allow us to fetch information about existing resources.
- They do not create or manage infrastructure.
- Useful to reference existing resources (like VPC, AMI, S3 bucket).

Example : Get Existing VPC

```
data "aws_vpc" "default" {
  default = true
}
```

Example : Get Latest Amazon Linux AMI

```
data "aws_ami" "amazon_linux" {
  owners      = ["amazon"]
  most_recent = true
  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}
```

Key Points :

- Use 'data' block with resource type.
- Data is read-only.
- Useful for lookups and dependencies.

4. Variables

- Variables make our configuration dynamic and reusable.
- They accept input values at runtime.
- Improves reusability and maintainability.

Example :

```
variable "region" {
  description = "AWS Region"
  type        = string
  default     = "ap-south-1"
}
```

```
# Using variable
provider "aws" {
  region = var.region
}
```

Ways to set variable values :

1. default value (in code)
2. terraform.tfvars file
3. -var="region=us-east-1"
4. environment variables
5. *.auto.tfvars file

Best Practices :

- Always use meaningful variable names.
- Provide description for every variable.
- Avoid hardcoding values.
- Use sensitive = true for secrets.

Engineer_cloudops

5. Variable Types

- Terraform variables can hold different types of values.
- Defining type helps in catching errors early.
- If no type is specified, it defaults to string.

Type	Example
string	"us-east-1"
number	30
bool	true / false
list(string)	["a", "b", "c"]
list(number)	[80, 443, 8080]
set(string)	toset(["a", "b", "c"])
map(string)	{ env = "dev", team = "ops" }
object	{ name = "web", port = 80 }
any	Any type (use carefully)

Example :

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}
```

Best Practices :

- Always define type for every variable.
- Use specific types instead of 'any'.
- Helpful in large scale projects.

6. Variable Validation

- Validation allows us to define rules for variable values.
- Helps to avoid incorrect or unexpected inputs.
- Validation is defined using validation block.

Example : Validate AWS Region

```
variable "region" {
  description = "AWS Region"
  type        = string
  validation {
    condition     = contains(["us-east-1","us-west-2","eu-west-1"],var.region)
    error_message = "Invalid region. Allowed values are
                     us-east-1 , us-west-2 , eu-west-1."
  }
}
```

Key Points :

- Validation runs during plan/apply.
- 'condition' must return true/false.
- Provide clear error_message.

7. Local Values

- Local values help to assign a name to an expression.
- Used to avoid repeating complex expressions.
- Improves readability and maintainability.

Example : Using Local Values

```
locals {
  common_tags = {
    Project     = "Demo"
    Environment = var.environment
    ManagedBy   = "Terraform"
  }
  name_prefix   = "${var.environment}-web"
}
# Using locals
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type
  tags          = merge(local.common_tags, {
                  Name = "${local.name_prefix}-server"
                })
}
```

Key Points :

- Defined in 'locals' block.
- Refer using local.<name>
- Cannot be overridden from outside.

Best Practice :

Use locals for tags, naming conventions, and repeated expressions.

8. Outputs

- Outputs are used to return values from Terraform after apply.
- Useful to display important information.
- Can be used in other modules or via CLI.

Example :

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type
  tags = {
    Name        = "Demo-Web"
  }
}
```

```
output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.web.id
}
```

```
output "public_ip" {
  description = "Public IP of the EC2 instance"
  value       = aws_instance.web.public_ip
}
```

Key Points :

- Reference resources using resource_name.attribute
- Use 'output' block.
- Sensitive data can be hidden using sensitive = true

CLI Usage :

```
terraform output
terraform output instance_id
terraform output -json
```

Engineer_cloudops

9. Expressions

- Expressions let us combine values, variables, functions, and operators to compute results.
- Terraform has many types of expressions.

Type of Expression	Example	Result
Arithmetic	`1 + 2 * 3`	7
String	`"Hello" + " World"`	Hello World
Boolean	`true && false`	false
Comparison	`5 > 3`	true
Conditional (Ternary)	`var.env == "prod" ? "yes" : "no"`	yes or no
Index	`["a", "b", "c"][1]`	b
Attribute	`aws_instance.web.id`	i-0abcd1234
Splat	`aws_instance.web[*].id`	list of ids
For Expression	`[for x in var.names : upper(x)]`	List of upper case

Example :

```
variable "size" {
  type    = number
  default = 3
}
locals {
  area    = var.size * var.size
  message = "Area is ${local.area}"
}
output "area_output" {
  value   = local.message
}
```

→ Computes square of size using expression.

→ Uses interpolation to include value in string.

Tips :

- Use expressions to avoid hardcoding values.
- Makes configuration dynamic and reusable.

10. Functions

- Terraform has 100+ built-in functions.
- Functions help to manipulate strings, numbers, dates, files, collections, etc.
- Improves readability and reduces complex logic.

Category	Example Functions	Example Usage	Result
String	`lower()`, `upper()`, `trim()`, `replace()`	`upper("hello")`	HELLO
Numeric	`min()`, `max()`, `abs()`, `ceil()`, `floor()`	`max(10, 20)`	20
Collection	`length()`, `merge()`, `keys()`, `values()`	`length(["a", "b"])`	2
Logical	`can()`, `try()`, `coalesce()`	`coalesce("", "default")`	default
Date & Time	`formatdate()`, `timestamp()`	`formatdate("YYYY-MM-DD", timestamp())`	2025-05-18
File	`file()`, `fileexists()`, `templatefile()`	`fileexists("test.txt")`	true/false

Example :

```
locals {
  names       = ["alice", "bob", "charlie"]
  upper_names = [for n in local.names : upper(n)]
  first_name  = local.names[0]
  name_count  = length(local.names)
}
output "names" {
  value   = local.upper_names
}
```

→ Converts all names to upper case using for expression + upper() function.
→ Access first element from list.
→ Count of names using length() function.

Commonly Used Functions :

- lower(), upper()
- length()
- join(), split()
- contains()
- lookup()
- merge()
- fileexists()
- templatefile()
- coalesce()

11. Comments

- Comments are used to explain the code.
- They are ignored by Terraform during execution.
- Improve readability and help in team collaboration.

Single Line Comment :

```
# This is a single line comment
resource "aws_s3_bucket" "example" {
  bucket = "my-bucket" # Inline comment
}
```

Multi Line Comment :

```
/*
This is a multi-line comment.
Use this to explain complex logic
or important information.
*/
resource "aws_instance" "web" {
  ami           = "ami-123456"
  instance_type = "t2.micro"
}
```

Best Practices :

- Always add comments for complex logic or non-obvious values.
- Keep comments updated with code changes.
- Don't overuse comments, keep it clean and minimal.

Engineer_cloudops

Module 3 : Terraform State

1. What is State File?

- Terraform state is a file that keeps track of the real infrastructure that Terraform creates.
- It maps our configuration (*.tf files) to actual resources in the cloud.
- Stored in JSON format.
- By default, stored locally in terraform.tfstate
- In team, we store remotely (S3 + DynamoDB).

Example (State File - JSON)

```
{
  "version": 4,
  "terraform_version": "1.6.6",
  "resources": [
    {
      "mode": "managed",
      "type": "aws_instance",
      "name": "web",
      "provider": "provider[\"registry.
                   terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "attributes": {
            "id": "i-0abcd1234efgh5678",
            "ami": "ami-123456",
            "instance_type": "t2.micro",
            "tags": {
              "Name": "WebServer"
            }
          }
        }
      ]
    }
  ]
}
```

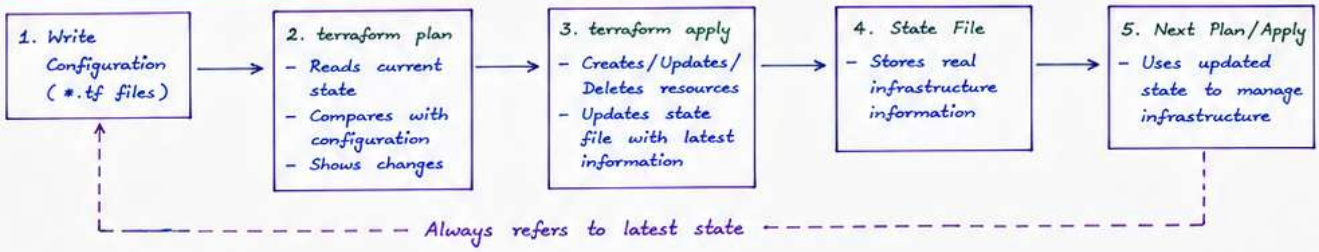
Why State is Important?

- Terraform knows what it created.
- Helps in planning changes.
- Prevents accidental deletion of resources.
- Stores metadata of resources (id, name, type, etc).
- Required for update and destroy operations.

Best Practices

- Never edit state file manually.
- Store state remotely.
- Enable state locking.
- Backup state file regularly.

2. Terraform State Workflow



3. State Commands

Command	Purpose	Example	Notes
terraform state list	Lists all resources in state.	terraform state list	Quick way to see what Terraform is tracking.
terraform state show <resource>	Shows details of a specific resource in state.	terraform state show aws_instance.web	Useful for debugging.
terraform state pull	Downloads the current state file.	terraform state pull > backup.tfstate	Good for backup.
terraform state push	Uploads a state file.	terraform state push backup.tfstate	Use carefully.
terraform state rm <resource>	Removes a resource from state (does not delete from cloud).	terraform state rm aws_instance.old	Use when resource is deleted manually.
terraform import <resource> <id>	Imports existing resource into state.	terraform import aws_s3_bucket.mybucket my-bucket	Brings existing infra under Terraform management.
terraform state mv <source> <destination>	Moves/renames a resource address in state.	terraform state mv aws_instance.old aws_instance.new	Useful when refactoring modules or names.

Tips:

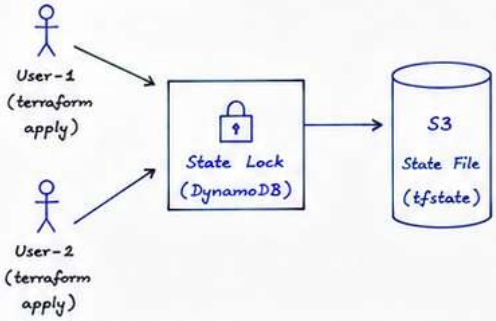
- State file is critical. Handle with care.
- Enable versioning for S3 state bucket.
- Use DynamoDB for state locking.
- Always run terraform plan before apply.

Engineer_cloudops

4. State Locking

- State locking prevents multiple users or processes from modifying the state file simultaneously.
- Without locking, state file can get corrupted.
- Terraform uses locking mechanism provided by the backend (like S3 + DynamoDB).

How it works?



Example: DynamoDB Lock Table

Table Name : terraform-locks
Partition Key : LockID (String)
This table stores lock info when a user runs terraform plan/apply.

Example: backend block

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state"
    key            = "dev/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

Key Points:

- Lock acquired during plan/apply.
- Lock released after operation completes.
- If operation fails, lock is released automatically.
- Stale lock may require manual unlock.

5. State Backup

- State file is critical. Losing it means losing track of infrastructure.
- Always take regular backups of state file.
- With remote backend (S3), versioning provides automatic backup.
- Manual backup also recommended before major changes.

Backup Strategies

Strategy	How it Works	Example / Notes
1. S3 Versioning	Enable versioning on S3 bucket. Each change creates a new version.	Enabled versioning allows you to restore previous versions anytime.
2. Cross-Region Replication	Replicate state file to another region for disaster recovery.	Protects from regional failures or accidental deletion.
3. Manual Backup	Manually copy tfstate file and store securely.	Use before risky operations like terraform import or state mv.
4. Automated Backup	Use scripts / CI jobs to backup state file at regular intervals.	Store backup in another bucket / folder.

Example: Enable S3 Versioning

```
resource "aws_s3_bucket_versioning" "state_versioning" {
  bucket = "my-terraform-state"
  versioning_configuration {
    status = "Enabled"
  }
}
```

Tips:

- Always test restore process.
- Keep multiple backups.
- Protect state file like production data.

6. State Drift

- State drift occurs when real infrastructure changes outside of Terraform.
- State file and actual infra become out of sync.
- Example: Someone manually deletes an EC2 instance.
- This can cause unexpected results in future plan/apply.

How Drift Happens?

- Manual changes in AWS Console / CLI
- Other automation tools (Ansible, CloudFormation)
- Deleted or modified resources
- Tag or attribute changes done outside Terraform

How to Detect Drift?

1. terraform plan
   - Compares state with real infrastructure.
   - Shows drift in the plan.
2. terraform refresh (deprecated in v0.15+)
   - Old command to update state from real infra.

Example Scenario

1. You create an EC2 instance using Terraform.
2. Someone stops the instance from AWS Console.
3. terraform plan will show:
   ```
   ~ resource "aws_instance" "web" {
     ~ instance_state = "running" -> "stopped"
   }
   ```
4. Apply will bring it back to desired state (running).

Best Practices

- Always run terraform plan before apply.
- Enable CloudTrail / Config for tracking changes.
- Use IAM policies to restrict manual changes.
- Monitor drift regularly in production.

Engineer_cloudops

1. Import Existing Resources

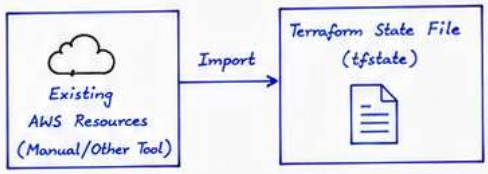
- Import brings existing infrastructure under Terraform management.
- Terraform will not create the resource, it just tracks it in state.
- Useful when infra was created manually or by another tool.
- After import, run plan to see no changes.

Example : Import an existing EC2

```
# Resource block in configuration
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  tags = {
    Name = "WebServer"
  }
}

# Import existing instance
terraform import aws_instance.web i-0abcd1234ef567890
```

How it works?



Key Points :

- Terraform won't modify resources during import.
- Only starts managing them.
- Ensure resource block matches actual resource.
- Import one resource at a time.

2. Refresh State

- Refresh updates the state file with real infrastructure.
- It reads the actual resources and syncs the state.
- Helps to fix drift between state and real infra.
- Automatically run during plan and apply.

Example : Refresh State

```
# Manually refresh state
terraform refresh

# (OR) Normal plan also does refresh
terraform plan
```

When to use?

- After manual changes in AWS Console.
- When resources were modified outside Terraform.
- Before creating a plan.

Note :
terraform refresh is deprecated in v0.15+ but terraform plan/apply automatically refreshes state.

3. Remove Resource from State

- Sometimes we don't want Terraform to manage a resource anymore, but we want the resource to remain.
- terraform state rm removes the resource from state file only.
- It will not delete the real infrastructure.
- Useful before destroying or migrating.

Example : Remove EC2 from State

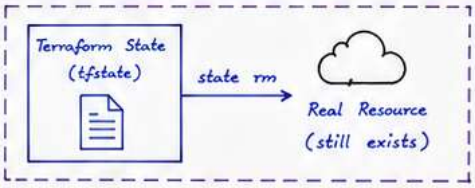
```
# Check resource is in state
terraform state list

# Remove from state
terraform state rm aws_instance.web

# Now Terraform will stop tracking
# but EC2 instance will remain.
```

Use Cases :

- Stop managing a resource.
- Before using another IaC tool.
- When resource is managed by other team.



4. Move State

- Used to move/rename resources within the state.
- Helpful when you change resource name, module structure or refactoring.
- terraform state mv updates the state without recreating resources.

Example : Move / Rename Resource

```
# Old resource name
resource "aws_instance" "web" { ... }

# New resource name in configuration
resource "aws_instance" "web_server" { ... }

# Move state from old to new
terraform state mv \
  aws_instance.web aws_instance.web_server
```

Use Cases :

- Renaming resources.
- Moving resources into modules.
- Refactoring without recreation.

Important :
Always run terraform plan after mv command to ensure Terraform understands the changes.

Best Practices

- Always backup state before making changes.
- Use remote backend (S3 + DynamoDB) for team work.
- Never edit state file manually.
- Lock state to avoid concurrent changes.
- Regularly refresh and check for drift.
- Test in lower environments before making critical changes.
- Document any manual import, rm or mv operations.

Engineer_cloudops

Module 4 : Backend

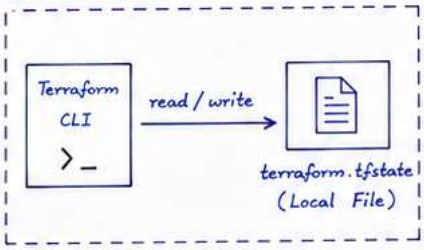
Engineer_cloudops

- Backend defines where Terraform stores the state file.
- Good backend = collaboration, locking, security, reliability.

1. Local Backend (Default)

- By default, Terraform uses local backend.
- State file is stored on local machine in the working directory.
- Good for learning and POC.
- Not suitable for team environments.

How it works ?



Example (No backend block)

```
# No backend block means
# local backend
# State file -> terraform.tfstate
# stored in current directory
```

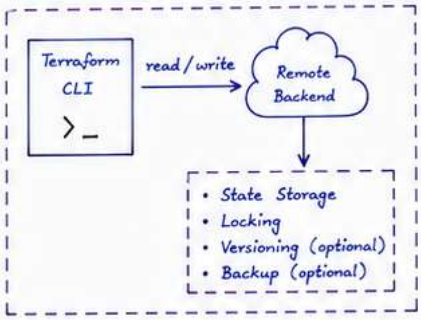
Key Points :

- No additional configuration.
- No locking.
- Risk of state file corruption.
- Avoid committing state file to Git.

2. Remote Backend

- Remote backend stores the state file in a remote system.
- Enables team collaboration.
- Supports state locking.
- Prevents state file corruption.
- Examples : S3, Terraform Cloud, Azure Blob, GCS.

How it works ?



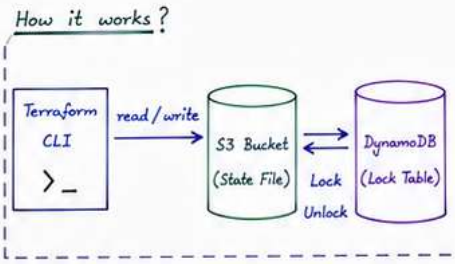
Key Benefits :

- Centralized state.
- State locking (avoids concurrent writes).
- Access control via IAM.
- Versioning & recovery.
- Works with CI/CD pipelines.

3. S3 Backend (Most Used)

- Stores state file in an S3 bucket.
- Use DynamoDB for state locking.
- Highly available, durable, scalable.
- Industry standard for AWS.

How it works ?



Example : backend block for S3

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state"
    key            = "envs/dev/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

Quick Comparison

Feature	Local Backend	Remote Backend	S3 Backend (with DynamoDB)
State Location	Local machine (terraform.tfstate)	Remote system	S3 Bucket
State Locking	No	Depends on backend	Yes (with DynamoDB)
Team Collaboration	No	Yes	Yes (Best for team)
Security (IAM)	No	Depends on backend	Yes (IAM policies)
Versioning / Backup	No	Depends on backend	Yes (S3 Versioning optional)
Availability	Low	Medium to High	High (99.99%)
Best For	Learning / POC	General use	Production (AWS)

Best Practices :

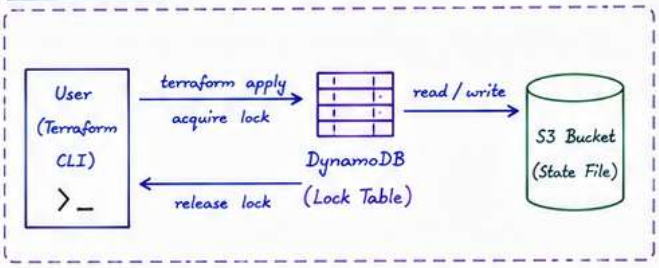
- Always use remote backend for team.
- Use S3 + DynamoDB for AWS.
- Enable S3 versioning.
- Restrict access using IAM.
- Never store state file in Git.
- Regularly backup state (versioning helps).

Engineer_cloudops

1. DynamoDB State Locking

- When using S3 backend, enable state locking with DynamoDB.
- Prevents multiple users/processes from modifying the state file at the same time.
- Terraform creates a lock item in the DynamoDB table before running any operation.
- Lock is released automatically when operation completes.

How it works?



Example: DynamoDB Table (terraform-locks)

Attribute Name	Type	Key Type
LockID	String	Partition Key
Info	String	-
Who	String	-
Created	String	-
Version	String	-

Backend configuration (S3 + DynamoDB lock)

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state"
    key            = "envs/dev/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-locks"  # lock table
    encrypt        = true
  }
}
```

Key Points:

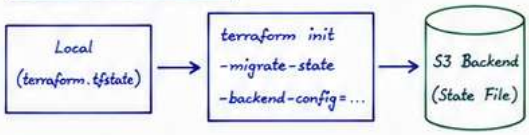
- DynamoDB table must have Partition Key: LockID (String).
- No read/write capacity is required (on-demand works).
- Ensures safe concurrent access.
- If lock is stale (crashed process), we can remove it manually.

Tip:
Always enable locking in team environments.

2. Backend Migration

- Moving state from one backend to another.
- Example: Local → S3
- Terraform provides "-migrate-state" flag during init.
- Make sure backup state before migration.

Workflow: Local → S3



Steps:

1. Backup current state file.
2. Update backend block to new backend (S3).
3. Run: terraform init -migrate-state
4. Terraform copies state to new backend.
5. Verify in S3 bucket.

Example init command:

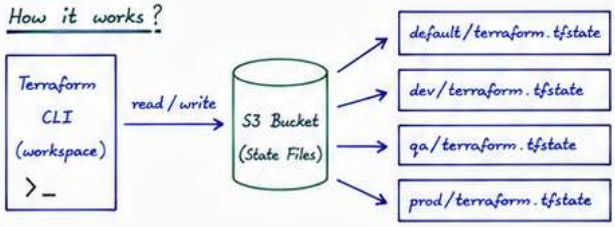
```
terraform init -migrate-state \
  -backend-config="bucket=my-terraform-state" \
  -backend-config="key=envs/dev/terraform.tfstate" \
  -backend-config="region=ap-south-1" \
  -backend-config="dynamodb_table=terraform-locks"
```

Note:
- Migration happens only during init.
- After migration, local state is not used.
- Always commit backend block to Git.

3. Workspace with Remote Backend

- Workspaces help manage multiple environments using same config.
- With remote backend (S3), each workspace has its own state file.
- Example: default, dev, qa, prod
- Avoids maintaining multiple directories.

How it works?



Common Commands:

- terraform workspace list
- terraform workspace new dev
- terraform workspace select dev
- terraform workspace show
- terraform workspace delete qa

Example Usage:

1. Create workspace:
   terraform workspace new dev
2. Select workspace:
   terraform workspace select dev
3. Apply for that workspace:
   terraform apply

Backend config (same for all workspaces):

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state"
    key            = "envs/${terraform.workspace}/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

Key Points:

- Each workspace has isolated state file.
- Use workspaces for dev/test environments.
- For strict isolation, use separate AWS accounts.

Module 5 : Terraform Lifecycle

Engineer_cloudops

Terraform follows a simple workflow to manage infrastructure as Code. Below are the core lifecycle commands I use daily.

Command	Purpose	Example	Notes / Real World Usage
1. terraform init (Initialize Working Directory)	Initializes a Terraform working directory. Downloads providers, modules and sets up the backend.	`$ terraform init`	- Run this first in any new Terraform project. - Re-run if backend or providers are changed.
2. terraform validate (Validate Configuration)	Validates the Terraform configuration files for syntax and internal consistency.	`$ terraform validate`	- Catches basic syntax errors early. - Use in CI/CD before plan.
3. terraform fmt (Format Code)	Rewrites Terraform configuration files to a canonical format and style.	`$ terraform fmt` `$ terraform fmt -recursive`	- Keeps code clean and consistent. - Run before committing to Git.
4. terraform plan (Preview Changes)	Creates an execution plan. Shows what will be created, changed or destroyed.	`$ terraform plan` `$ terraform plan -out=plan.tfplan`	- Always run before apply. - Use -out to save the plan for later apply. - Helps in code review process.
5. terraform apply (Apply Changes)	Applies the changes required to reach the desired state of the configuration.	`$ terraform apply` `$ terraform apply plan.tfplan`	- Without file: shows plan and waits for approval. - With file: applies the saved plan. - Use in automation with -auto-approve (use carefully).
6. terraform destroy (Destroy Infrastructure)	Destroys all resources managed by this configuration.	`$ terraform destroy`	- Destroys everything in state. - Use -auto-approve in automation. - Be very careful in production!
7. terraform refresh (Refresh State)	Updates the state file to match the real infrastructure (without making changes).	`$ terraform refresh`	- Useful if something changed outside Terraform (manual changes). - Now part of "terraform plan/apply" implicitly in latest versions.

8. terraform output (Show Output Values)	Diisplays output values defined in the configuration.	`$ terraform output`
		`$ terraform output <name>`
		`$ terraform output -json`
			- Get information like IPs, URLs, ARNs after apply.
			- Use in scripts and automation.

9. terraform show (Show State/Plan)	Shows human-readable output of the current state or a plan file.	`$ terraform show` `$ terraform show plan.tfplan`	- Without args: shows current state. - With plan file: shows saved plan in readable format.

Typical Workflow (Day to Day)

1. terraform init
2. terraform fmt
3. terraform validate
4. terraform plan -out=plan.tfplan
5. terraform apply plan.tfplan
6. terraform output (to get values)

Quick Tips from Experience

- Always run plan before apply (even in automation).
- Use remote backend + state locking for team environments.
- Keep your state file safe. It is the source of truth.
- Use workspaces for dev/test/prod isolation.
- Review the plan carefully, especially before destroy or large changes.

Remember:

Plan shows you the future, Apply makes it real! ☺

Happy Automating !! Keep Learning, Keep Building

Module 6 : Meta Arguments

Engineer_cloudops

Meta-arguments help us to control how resources are created, updated or behave in Terraform.

1. count

- Creates multiple instances of a resource based on a number.
- Index starts from 0.
- Useful for identical resources.

Example :

```
resource "aws_instance" "web" {
  count         = 3
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  tags = {
    Name = "Web-${count.index}"
  }
}
```

Notes / Real World Usage :

- Good when you know the number of resources.
- Example : Create 3 web servers.
- Access attributes using aws_instance.web[count.index]

Tip : If the count value changes, resources might be destroyed and recreated.

2. for_each

- Creates resources based on a map or set of strings.
- Each resource has a unique key.
- Best for non-identical resources.

Example :

```
variable "servers" {
  type = map(string)
  default = {
    web = "t2.micro"
    app = "t2.small"
    db  = "t2.medium"
  }
}

resource "aws_instance" "server" {
  for_each      = var.servers
  ami           = "ami-12345678"
  instance_type = each.value
  tags = {
    Name = each.key
  }
}
```

Notes / Real World Usage :

- Keys must be unique and stable.
- Changing keys will destroy and recreate resources.
- Example : Different instance types for web, app, db.

Tip : for_each is preferred over count when working with maps/sets.

3. depends_on

- Explicitly defines dependency between resources.
- By default Terraform understands dependencies, but use this when it can't detect.

Example :

```
resource "aws_instance" "app" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  depends_on    = [aws_iam_role_policy.attach]
}
```

Notes / Real World Usage :

- Use when order of creation matters.
- Example : Wait for IAM policy attachment before creating instance.
- Avoid overusing. Use only when required.

4. lifecycle

- Controls how Terraform handles changes to a resource.
- Prevents unnecessary destroy/create.

Example :

```
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  lifecycle {
    create_before_destroy = true
    ignore_changes        = [tags["Environment"]]
    prevent_destroy       = false
  }
}
```

Notes / Real World Usage :

- create_before_destroy : Create new resource first, then destroy old one.
- ignore_changes : Ignore changes to specific attributes.
- prevent_destroy : Protect resource from being destroyed accidentally.

Warning : prevent_destroy = true can block destroy even when required.

5. provider

- Specifics which provider configuration a resource should use.
- Useful when working with multiple providers or regions.

Example :

```
provider "aws" {
  alias  = "us_east"
  region = "us-east-1"
}

provider "aws" {
  alias  = "us_west"
  region = "us-west-2"
}
```

```
resource "aws_s3_bucket" "east" {
  provider = aws.us_east
  bucket   = "my-bucket-east"
}

resource "aws_s3_bucket" "west" {
  provider = aws.us_west
  bucket   = "my-bucket-west"
}
```

Notes / Real World Usage :

- Manage resources in different regions/accounts.
- Example : S3 bucket in us-east-1 and another in us-west-2.
- Requires provider alias.

6. dynamic blocks

- Generates nested blocks dynamically using for_each.
- Useful for complex nested structures.

Example :

```
resource "aws_security_group" "sg" {
  name = "dynamic-sg"

  dynamic "ingress" {
    for_each = var.ports
    content {
      from_port   = ingress.value.from
      to_port     = ingress.value.to
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

```
variable "ports" {
  type = list(object({
    from = number
    to   = number
  }))
  default = [
    { from = 80,  to = 80 },
    { from = 443, to = 443 }
  ]
}
```

Notes / Real World Usage :

- Avoids repetition of similar nested blocks.
- Example : Multiple ingress rules in Security Group.
- Makes code clean and scalable.

Quick Summary

- ☑ count → for same resources in numbers.
- ☑ for_each → for map/set based unique resources.
- ☑ depends_on → for explicit dependency.
- lifecycle → control creation/updates/destroy.
- provider → choose required provider configuration.
- dynamic blocks → generate nested blocks dynamically.

Practics these with real examples. That's the best way to learn Terraform! ☺

Engineer_cloudops

Module 7 : Lifecycle Rules

Engineer_cloudops

Lifecycle rules in Terraform allow you to control how Terraform manages the lifecycle of resources during create, update and destroy.

1. create_before_destroy

- Creates the new resource before destroying the old one.
- Ensures zero downtime for critical resources.
- Useful for production environments where availability is important.

Example :

```
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  lifecycle {
    create_before_destroy = true
  }
}
```

Notes :

- Terraform will create a new instance first.
- Once the new instance is healthy, Terraform destroys the old one.
- Helps in maintaining availability.

2. prevent_destroy

- Prevents a resource from being deleted by Terraform.
- Useful for critical resources like production databases, VPCs, etc.
- Terraform will throw an error if destroy is attempted.

Example :

```
resource "aws_db_instance" "db" {
  identifier     = "prod-db"
  instance_class = "db.t3.medium"
  lifecycle {
    prevent_destroy = true
  }
}
```

Notes :

- Resource cannot be destroyed (destroy plan will fail).
- Must be removed manually or lifecycle rule updated.
- Protects critical infrastructure.

3. ignore_changes

- Ignores changes to specified attributes during updates.
- Terraform will not try to modify those fields even if they change in configuration or in the real world.
- Useful when the value is managed outside Terraform.

Example :

```
resource "aws_instance" "app" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  tags = {
    Name = "app-server"
  }
  lifecycle {
    ignore_changes = [tags["Name"]]
  }
}
```

Notes :

- Terraform will ignore changes to tags["Name"].
- Keeps the value as managed by something else (e.g., scripts or external systems).
- Avoids unnecessary updates.

4. replace_triggered_by

- Replaces a resource when a specific resource or value changes.
- Useful for dependent resources that need recreation when another resource changes.

Example :

```
resource "null_resource" "reload_app" {
  triggers = {
    config_hash = filesha256("app.conf")
  }
  lifecycle {
    replace_triggered_by = [
      aws_s3_bucket.app_config
    ]
  }
}
```

Notes :

- Terraform will recreate this resource if aws_s3_bucket.app_config changes.
- Helps in forcing recreation when external dependencies change.
- Useful for config-based triggers.

Quick Summary :

Lifecycle Rule	Purpose	When to Use
create_before_destroy	Create new resource before destroying old one	Zero downtime, high availability
prevent_destroy	Prevent resource deletion	Protect critical resources
ignore_changes	Ignore specific attribute changes	When value is managed outside Terraform
replace_triggered_by	Recreate resource when dependent resource/value changes	For dependent resources, config-based triggers

Module 8 : Expressions & Functions

Engineer_cloudops

Expressions help us build dynamic and reusable configurations.
Terraform provides many built-in functions to work with data.

1. Conditional Expressions (Ternary Operator)

- Returns one value if condition is true and another if false.
- Syntax : condition ? true_val : false_val

Example :

```
variable "env" {
  type = string
}

locals {
  instance_type = var.env == "prod" ? "t3.large" : "t3.micro"
}

# If env is prod → t3.large else t3.micro
```

Real World Usage :

- Use different instance types per environment.
- Enable/disable features based on conditions.
- Set defaults when values are not provided.

Tip :
Useful for environment based configurations.

2. For Expressions

- Transforms a list or map into another list or map.
- Syntax (list) : [for item in list : expression]
- Syntax (map) : { for k, v in map : k => expression }

Example (List) :

```
variable "azs" {
  type = list(string)
  default = ["us-east-1a", "us-east-1b", "us-east-1c"]
}

locals {
  az_names = [for az in var.azs : upper(az)]
}

# Result : ["US-EAST-1A", "US-EAST-1B", "US-EAST-1C"]
```

Example (Map) :

```
variable "tags" {
  type = map(string)
  default = {
    env = "dev"
    owner = "cloudops"
  }
}

locals {
  tag_list = { for k, v in var.tags : k => upper(v) }
}

# Result : { env = "DEV", owner = "CLOUDOPS" }
```

Real World Usage :

- Create multiple resources from a list/map.
- Transform data for modules or outputs.
- Build dynamic tags, names, or blocks.

3. Splat Expressions

- Access attributes from a list of resources.
- Returns a list of that attribute.
- Syntax : resource.type.name[*].attr or resource.type.name[*]

Example :

```
resource "aws_instance" "web" {
  count         = 3
  ami           = "ami-12345678"
  instance_type = "t2.micro"
}

output "public_ips" {
  value = aws_instance.web[*].public_ip
}

# Output will be list of all public IPs.
```

Real World Usage :

- Get list of IDs, ARNs, IPs from multiple resources.
- Pass list of values to modules or outputs.

Tip :
Modern Terraform prefers full splat [*] over legacy splat (.*).

4. Built-in Functions

Terraform has 100+ built-in functions.

Type Conversion
- tostring()
- tonumber()
- tobool()
- tolist()
- tomap()

Collection
- length()
- contains()
- keys()
- values()
- merge()
- distinct()

Numeric
- abs()
- ceil()
- floor()
- max()
- min()
- sum()

Date & Time
- timestamp()
- formatdate()
- timeadd()
- timecmp()

Encoding
- base64encode()
- base64decode()
- jsonencode()
- jsondecode()

Real World Usage :

- Manipulate and transform values.
- Work with dates, numbers, strings.
- Encode/decode data for APIs.

5. String Functions (Most Used)

- upper(str) → Convert to UPPERCASE
- lower(str) → Convert to lowercase
- title(str) → Convert to Title Case
- trim(str) → Remove leading/trailing spaces
- substr(str, start, len) → Substring
- replace(str, old, new) → Replace text
- split(str, sep) → Split string into list
- join(sep, list) → Join list into string
- format("Hi %s", var.name) → String formatting

Example :

```
locals {
  name        = "  engineer_cloudops  "
  clean_name  = trim(lower(local.name))
  short_name  = substr(local.clean_name, 0, 8)
  greeting    = format("Hello %s", title(local.clean_name))
  parts       = split("_", local.clean_name)
}

# clean_name  = "engineer_cloudops"
# short_name  = "enginer"
# greeting    = "Hello Engineer_cloudops"
# parts       = ["engineer", "cloudops"]
```

Real World Usage :

- Create consistent resource names.
- Clean and validate user input.
- Build dynamic strings for tags, ARNs, or names.

Engineer_cloudops

Master Expressions today, Automate everything tomorrow!

Module 8 : More Functions in Terraform

Engineer_cloudops

Terraform provides a rich set of built-in functions to work with numbers, collections, files and dates. These help us write flexible, clean and dynamic configurations.

1. Numeric Functions

- Used for mathematical operations on numbers.
- Helpful in calculations, randomization, and value conversions.
- Common functions : abs, ceil, floor, max, min, sum, tonumber, round.

Example :

```
# Find max of two numbers
locals {
  max_count   = max(10, 25)

  # Round a number
  rounded     = round(3.14159, 2)

  # Convert string to number
  port        = tonumber("8080")
}
```

Real World Usage :

- Calculate instance count based on input.
- Round budget values for cost estimation.
- Convert string inputs (from tfvars) to numbers.

2. Collection Functions

- Used to work with lists, maps, and sets.
- Helpful for filtering, merging, and transforming data.
- Common functions : length, keys, values, merge, concat, distinct, flatten, contains, slice.

Example :

```
locals {
  envs      = ["dev", "staging", "prod", "prod"]
  unique    = distinct(local.envs)

  tags1     = { team = "devops" }
  tags2     = { env  = "prod" }
  all_tags  = merge(local.tags1, local.tags2)

  first_two = slice(local.envs, 0, 2)
}
```

Real World Usage :

- Merge common and environment specific tags.
- Get unique values from a list (e.g., AZs, regions).
- Use length() to set counts dynamically.

3. File Functions

- Used to read and work with files in Terraform.
- Helpful for loading scripts, templates and configuration files.
- Common functions : file, fileexists, templatefile, fileset, dirname, basename.

Example :

```
locals {
  user_data   = file("./scripts/userdata.sh")
  config_json = file("./config/app.json")
  config_tmpl = templatefile("./templates/app.tmpl",{
                  env = "prod"
                })
  exists      = fileexists("./config/app.json")
}
```

Real World Usage :

- Load user data script for EC2.
- Use templatefile() to render config files with variables.
- Check if a file exists before using it (useful in modules).

4. Date Functions

- Used to work with date and time values.
- Helpful for generating timestamps, formatting dates and setting time based values.
- Common functions : timestamp, formatdate, timeadd.

Example :

```
locals {
  now       = timestamp()
  formatted = formatdate("YYYY-MM-DD hh:mm", local.now)
  next_week = timeadd(local.now, "168h")  # 7 days
}
```

Real World Usage :

- Add timestamp to resource names (for uniqueness).
- Set expiry time for temporary resources.
- Use formatted date in tags or logs.

Quick Tips :

- ☑ Use tonumber() / tobool() / tostring() for type conversion.
- ☑ Use merge() and concat() for flexible data handling.
- ☑ Use templatefile() for dynamic config generation.
- ☑ Always validate file paths with fileexists().
- ☑ Use formatdate() to keep date format consistent.

Key Takeaway :

→ Functions make Terraform code more dynamic and reusable.
→ They help us handle real world scenarios like data transformation, file loading and time based logic.
→ Use the right function for the right data type to keep your code clean and maintainable.

Learn ✦ Automate ✦ Build Better !! 🚀

Engineer_cloudops

Module 9 : Modules

Engineer_cloudops

Modules in Terraform are reusable packages of infrastructure code. They help us write DRY (Don't Repeat Yourself) code, improve consistency and make code easy to manage and scale.

1. What are Modules ?

- A module is a collection of Terraform configurations in a directory.
- It can contain resources, variables, outputs and other files.
- Modules help us reuse the same infrastructure code in multiple places.
- Terraform Registry has many public modules built by community.

Example - Structure of a Module

```
my-module/
├── main.tf
├── variables.tf
├── outputs.tf
├── providers.tf (optional)
└── README.md
```

Inside main.tf

```
resource "aws_s3_bucket" "this" {
  bucket = var.bucket_name
  acl    = "private"
  tags   = var.tags
}
```

Inside variables.tf

```
variable "bucket_name" {
  type = string
}
variable "tags" {
  type = map(string)
  default = {}
}
```

Inside outputs.tf

```
output "bucket_name" {
  value = aws_s3_bucket.this.bucket
}
output "bucket_arn" {
  value = aws_s3_bucket.this.arn
}
```

2. Local Modules

- Local modules are stored in our local filesystem.
- Best for private modules or organization specific standards.
- We just reference the local path in the module block.

Tip :
Use local modules for reusable components like VPC, Security Groups, IAM roles, etc.

Example - Project Structure

```
project-root/
├── main.tf
├── variables.tf
└── modules/
    └── s3_bucket/
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

In main.tf (calling the module)

```
module "s3" {
  source      = "../modules/s3_bucket"
  bucket_name = "my-local-bucket"
  tags = {
    Environment = "dev"
    Owner       = "cloudops"
  }
}
```

How it works ?

1. Terraform reads module block.
2. It goes to the given local path.
3. Loads the module files and provisions resources.

Real World Usage :

- Build standard modules for VPC, EC2, RDS.
- Reuse across multiple environments (dev, test, prod).
- Keep modules in a shared Git repo inside your org.

3. Remote Modules

- Remote modules are stored outside our local system.
- Common sources :
  - Terraform Registry
  - Git Repositories (GitHub, GitLab, Bitbucket)
  - HTTP URLs
  - S3 Bucket
- Useful for community modules or sharing across teams.

Tip :
Use remote modules from Terraform Registry to save time and follow best practices.

Example - Using Terraform Registry

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.1.2"

  name            = "my-vpc"
  cidr            = "10.0.0.0/16"
  azs             = ["ap-south-1a", "ap-south-1b"]
  public_subnets  = ["10.0.1.0/24", "10.0.2.0/24"]
}
```

Example - Using GitHub Module

```
module "iam_role" {
  source = "github.com/cloudposse/terraform-
            aws-iam-role?ref=main"
  role_name = "example-role"
  tags = {
    Team = "DevOps"
  }
}
```

How it works ?

1. Terraform downloads the module from the source.
2. Stores it in .terraform/modules/ directory.
3. Uses it like a local module.

Real World Usage :

- Use community modules to save time.
- Centralize modules in a Git repo and reuse.
- Pin module versions for stability.

Modules = Reusability + Consistency + Maintainability

Engineer_cloudops

Module 9 : Modules (Deep Dive)

Engineer_cloudops

Modules help us write reusable, maintainable and scalable Terraform code. Let's understand the core building blocks of any module.

1. Module Structure

- A module is just a directory with .tf files.
- It can contain resources, variables, outputs, data sources and even child modules.
- Typical structure :

```
my-module/
├── main.tf          # resources
├── variables.tf     # inputs
├── outputs.tf       # outputs
├── providers.tf     # (optional)
└── README.md
```

Files Purpose :

main.tf	→ Defines resources
variables.tf	→ Declares input variables
outputs.tf	→ Declares output values
providers.tf	→ Provider configuration (optional)
README.md	→ Documentation

2. Module Inputs

- Inputs are defined using variable blocks.
- They make modules dynamic and reusable.
- Default values are optional but helpful.

Inside variables.tf

```
variable "instance_type" {
  description = "Type of EC2 instance"
  type        = string
  default     = "t3.micro"
}

variable "tags" {
  description = "Common tags"
  type        = map(string)
  default     = {}
}
```

Calling the module (in root module)

```
module "ec2" {
  source        = "../modules/ec2"
  instance_type = "t3.small"
  tags = {
    Environment = "dev"
    Owner       = "cloudops"
  }
}
```

Real World Usage :

- Pass environment, instance types, CIDR blocks, names etc.
- Easily change values from one place.

Tip : Always validate inputs using type and description.

3. Module Outputs

- Outputs expose important values from the module.
- Other modules or root module can use these values.
- Helps in chaining modules.

Inside outputs.tf

```
output "instance_id" {
  description = "ID of EC2 instance"
  value       = aws_instance.main.id
}

output "instance_public_ip" {
  description = "Public IP of EC2"
  value       = aws_instance.main.public_ip
}
```

Accessing outputs in root module

```
module "ec2" {
  source        = "../modules/ec2"
  instance_type = "t3.micro"
}

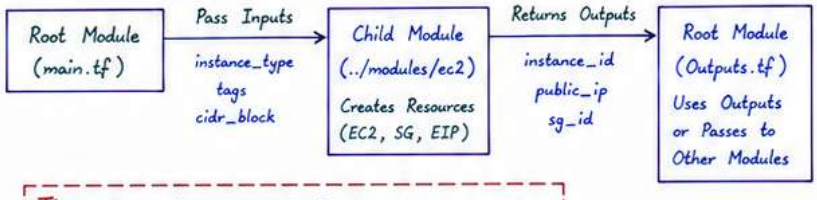
output "ec2_id" {
  value = module.ec2.instance_id
}
output "ec2_ip" {
  value = module.ec2.instance_public_ip
}
```

Real World Usage :

- Get VPC ID, Subnet IDs, Security Group IDs, etc.
- Pass outputs to another module as inputs.

Tip : Keep outputs minimal and meaningful. Don't expose sensitive data.

Flow Example (Root → Module → Output)



Tip : Think of modules like functions in programming.
Inputs → Process → Outputs

Best Practices

- ☑ Use clear names for inputs and outputs.
- ☑ Always add descriptions.
- ☑ Keep modules small and focused.
- ☑ Use default values where possible.
- ☑ Version your modules (especially remote modules).
- ☑ Document your module (README).

Write once. Use anywhere. That's the power of Terraform Modules! 💪

Engineer_cloudops

Module 9 : Modules (Deep Dive)

Engineer_cloudops

> ⭐ Modules make Terraform code Reusable, Maintainable and Scalable.
> Let's understand Versioning, Reusability and Module Hierarchy. ✓

1 Module Versioning

- ✓ Versioning helps us control changes and keep stability.
- ✓ Mostly used with Remote Modules (Registry, Git, S3).
- ✓ We define version in module block.

Example – Using Version

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.7.0"   # specific version
  # OR use version range
  # version = ">= 5.0.0, < 6.0.0"
  name    = "my-vpc"
  cidr    = "10.0.0.0/16"
  azs     = ["ap-south-1a", "ap-south-1b"]
}
```

Best Practices

- ☑ Always pin the module version.
- ☑ Avoid "latest" (unstable).
- ☑ Use semantic versions (MAJOR.MINOR.PATCH)
- ☑ Test before upgrading.

> **Tip :** Lock the version in production for stability.

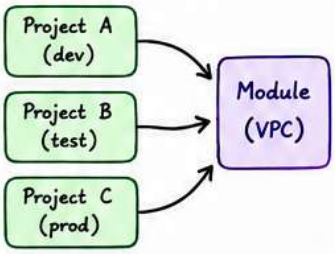
2 Reusable Modules

- → Write once, use anywhere.
- → Share across projects/teams.
- → Keep modules small and focused.
- → Use variables for flexibility.

Example – Reusable Module (VPC)

```
modules/
└── vpc/
    ├── main.tf
    ├── variables.tf
    ├── outputs.tf
    └── README.md
```

How it is Reused ?



Real World Use Cases

- ☑ VPC, Subnet, SG
- ☑ IAM Roles
- ☑ RDS, S3, Lambda
- ☑ EKS Cluster
- ☑ Networking

... and many more !

3 Child vs Root Module

📌 Terraform has two main types of modules :

Root Module

- ☑ The entry point of Terraform.
- ☑ The module you run (terraform apply) from.
- ☑ Often contains environment specific configs.
- ☑ Can call child modules.

Example Structure :

```
environments/
└── dev/  (Root Module)
    ├── main.tf
    ├── variables.tf
    └── terraform.tfvars
```

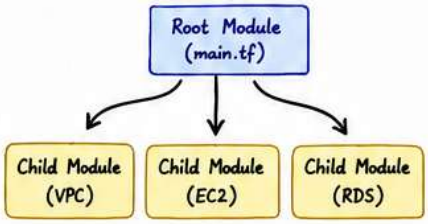
Child Module

- ☑ A reusable module called by root or another module.
- ☑ Contains actual resource definitions.
- ☑ Does NOT need backend.
- ☑ Can be nested (module inside module).

Example Structure :

```
modules/
└── ec2/  (Child Module)
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```

Visual Representation



Quick Comparison

Aspect	Root Module	Child Module
Purpose	Entry point, orchestrates	Reusable component
Called By	You (terraform apply)	Called by Root/Other modules
Backend	Yes (usually in Root)	No
Contains	Module blocks, variables, outputs	Resources, variables, outputs

Example – Root calling Child Module

```
# In Root Module (main.tf)
module "ec2" {
  source        = ".../modules/ec2"
  instance_type = "t3.micro"
  ami_id        = "ami-0abcd1234"
  tags = {
    Environment = "dev"
    Owner       = "cloudops"
  }
}
```

```
# Inside Child Module (modules/ec2/main.tf)
resource "aws_instance" "this" {
  ami           = var.ami_id
  instance_type = var.instance_type
  tags          = var.tags
}
```

🎯 Key Takeaways

- ☑ Use modules to avoid code duplication.
- ☑ Version your remote modules for stability.
- ☑ Keep modules focused and reusable.
- ☑ Understand the difference between Root and Child modules.

💡 Pro Tips from Real World

- → Keep modules in a separate Git repo.
- → Publish internal modules in private registry (GitHub/Artifactory).
- → Document inputs, outputs and usage in README.md.
- → Test modules before using in production.

⭐ Remember

Good modules = Clean Code + Consistency + Reusability ✓

Build once. Use everywhere. That's the power of Terraform Modules! 🚀

Engineer_cloudops

Module 10 : Provisioners

Engineer_cloudops

Provisioners allow you to execute scripts or commands on local machine or remote resources during resource creation or destruction. Use with care!

1. local-exec

- Runs a command on the machine where Terraform is running.
- Useful for local script execution, formatting, notifications, etc.

Example :

```
resource "null_resource" "example" {
  provisioner "local-exec" {
    command = "echo 'Resource created!' > result.txt"
  }
}
```

Use Cases :

- Run shell scripts
- Send notification
- Generate files
- Local cleanup tasks

2. remote-exec

- Executes commands on a remote resource after it is created.
- Requires connection block to connect via SSH or WinRM.

Example :

```
resource "aws_instance" "web" {
  ami           = "ami-0abcd1234"
  instance_type = "t2.micro"

  provisioner "remote-exec" {
    inline = [
      "sudo apt update -y",
      "sudo apt install -y nginx"
    ]
  }

  connection {
    type        = "ssh"
    user        = "ubuntu"
    private_key = file("~/.ssh/id_rsa")
    host        = self.public_ip
  }
}
```

Use Cases :

- Install software
- Configure server
- Run setup/bootstrapping tasks

Tip :
Use security groups and IAM roles to allow SSH access securely.

3. file Provisioner

- Copies files or directories from the local machine to a remote resource.
- Works with remote-exec using the same connection block.

Example :

```
resource "aws_instance" "web" {
  ami           = "ami-0abcd1234"
  instance_type = "t2.micro"

  provisioner "file" {
    source      = "./app/"
    destination = "/home/ubuntu/app/"
  }

  connection {
    type        = "ssh"
    user        = "ubuntu"
    private_key = file("~/.ssh/id_rsa")
    host        = self.public_ip
  }
}
```

Use Cases :

- Upload application code
- Copy configuration files
- Copy scripts or templates

Tip :
Use relative paths for portability and keep files lightweight.

4. Null Resource

- Special resource used mainly with provisioners.
- Does not manage real infrastructure.
- Triggers provisioners based on dependencies or changes.
- Commonly used with local-exec.

Example :

```
resource "null_resource" "trigger_example" {
  triggers = {
    always_run = timestamp() # change to run again
  }

  provisioner "local-exec" {
    command = "echo 'Triggered at ${timestamp()}'"
  }
}
```

Use Cases :

- Run tasks on every apply
- Trigger scripts when files change
- Glue between resources and actions

Tip :
Use triggers wisely to avoid unnecessary executions.

5. Why Provisioners are discouraged ?

Challenges :

- ✗ Not idempotent - scripts may fail or produce different results.
- ✗ Hard to debug and track.
- ✗ Not aware of infrastructure state.
- ✗ Can break Terraform workflow.
- ✗ Hard to maintain across teams and environments.

Better Alternatives :

- ✓ Use Cloud-init, User Data for bootstrapping.
- ✓ Use Configuration Management tools (Ansible, Chef, Puppet).
- ✓ Use Packer for building golden images.
- ✓ Use CI/CD pipelines for post-deploy tasks.

Golden Rule :
Use provisioners only as a last resort. Terraform should manage infrastructure, not configuration. ☆

Provisioners are powerful but risky. Keep your Terraform code declarative and predictable!

Engineer_cloudops

Module 11 : Workspaces

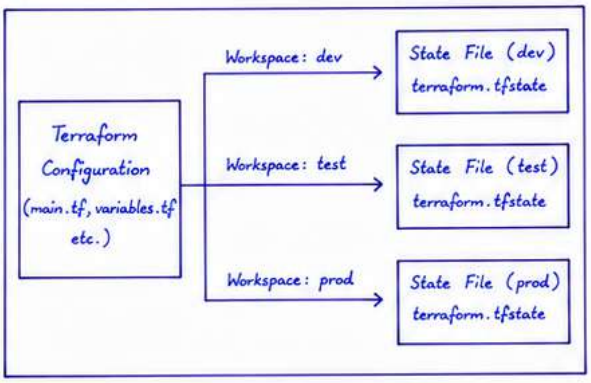
Engineer_cloudops

Workspaces help you manage multiple environments (Dev, Test, Prod, etc.) using the same Terraform configuration.
Each workspace has its own state file.

1. Workspaces

- A workspace is an isolated environment within the same Terraform configuration.
- Each workspace has its own state file.
- Default workspace is "default".
- Useful for managing multiple environments.

How it works ?



Benefits :

- ✓ Manage multiple environments with same code.
- ☑ Isolated state for each environment.
- ☑ Easy switching between environments.
- ☑ Reduces code duplication.

Tip :
Workspaces are great for small to medium teams. For large setups, use separate backends (S3 buckets).

2. Dev / Test / Prod Environment

- You can use workspaces to create Dev, Test and Prod environments.
- Same code, different variables (optional) and state.
- Example structure :

```
project-root/
├── main.tf
├── variables.tf
├── outputs.tf
└── terraform.tfvars (optional)
```

Workspace	Purpose	Example Usage
dev	For development and initial testing	Develop new features, unit testing
test	For QA and UAT testing	Integration testing, bug fixing
prod	For production live environment	Live traffic, real users

Best Practice :
Use workspaces + tfvars files to manage environment specific variables.
Example: dev.tfvars, test.tfvars, prod.tfvars

3. Workspace Commands

Command	Description	Example	Notes
terraform workspace list	List all workspaces.	terraform workspace list	★ shows current workspace
terraform workspace new <name>	Create a new workspace and switch to it.	terraform workspace new dev	Creates and switches to "dev" workspace
terraform workspace select <name>	Switch to an existing workspace.	terraform workspace select test	Switches to "test" workspace
terraform workspace show	Show the current workspace.	terraform workspace show	Displays current workspace name
terraform workspace delete <name>	Delete a workspace.	terraform workspace delete dev	Cannot delete "default" workspace
terraform workspace select default	Switch back to default workspace.	terraform workspace select default	Useful to go back to default

Example Workflow :

1. terraform workspace new dev
2. terraform apply -var-file=dev.tfvars
3. terraform workspace new test
4. terraform apply -var-file=test.tfvars
5. terraform workspace new prod
6. terraform apply -var-file=prod.tfvars

Important Notes :

- ★ Workspaces only separate state files.
- ★ Resources are still defined in the same configuration.
- ★ For complete isolation, use separate backends.
- ★ Don't store sensitive data in tfvars files.

Workspaces make your Terraform code more flexible and environment ready.
Use them wisely and keep your states safe ! ☺

Engineer_cloudops

Module 12 : Advanced Terraform (Deep Dive)

Engineer_cloudops

These advanced features will help you write flexible, reusable and secure Terraform code like a PRO! ☆

1. Dynamic Blocks

- Used to generate nested blocks dynamically.
- Helpful when the number of blocks is not fixed.
- Works with for_each.

Example - AWS Security Group with Dynamic Ingress Rules

```
variable "ingress_rules" {
  type    = list(object({
    from_port    = number
    to_port      = number
    protocol     = string
    cidr_blocks  = list(string)
  }))
}
resource "aws_security_group" "example" {
  name          = "example-sg"
  description   = "SG with dynamic ingress"

  dynamic "ingress" {
    for_each    = var.ingress_rules
    content {
      from_port    = ingress.value.from_port
      to_port      = ingress.value.to_port
      protocol     = ingress.value.protocol
      cidr_blocks  = ingress.value.cidr_blocks
    }
  }
}
```

Use Case :

- When you don't know how many ingress / egress rules are needed.
- Avoids code repetition.

Tip :
Dynamic blocks are evaluated at plan time. for_each must be known.

2. Templatefile

- Renders an external template file with given variables.
- Better than heredoc for large configs (user-data, policies, etc).

Example - Using templatefile() for user-data

user-data.sh.tpl

```
#!/bin/bash
echo "App Name: ${app_name}"
echo "Env     : ${env}"
echo "Owner   : ${owner}"
```

main.tf

```
locals {
  userdata = templatefile("user-data.sh.tpl", {
    app_name  = var.app_name
    env       = var.env
    owner     = var.owner
  })
}
```

Use Case :

- EC2 user data scripts
- IAM policies
- Kubernetes manifests
- Any large config files

Tip :
Keep templates in .tpl files for clean and reusable code.

3. Sensitive Variables

- Mark variables as sensitive to prevent showing in CLI output and state.
- Still stored in state file, so secure your backend!

Example

```
variable "db_password" {
  type      = string
  sensitive = true
}

resource "aws_db_instance" "db" {
  identifier  = "mydb"
  password    = var.db_password
}
```

Use Case :

- Passwords
- API keys
- Tokens
- Private keys

Tip :
Use sensitive = true and restrict access to your state files.

4. Random Provider

- Generates random values during resource creation.
- Useful for names, passwords, strings, etc.
- Provided by hashicorp / random provider.

Example - Random Password

```
resource "random_password" "db_pass" {
  length  = 16
  special = true
}

resource "aws_db_instance" "db" {
  identifier  = "mydb"
  password    = random_password.db_pass.result
}
```

Other Random Resources :

- random_id
- random_string
- random_integer
- random_pet

Tip :
Random values will change if resource is tainted or recreated. Use lifecycle rules if you want to keep them stable.

Key Takeaways :

- ✓ Use dynamic blocks for flexible nested structures.
- ✓ Use templatefile() for external templates.
- ✓ Mark sensitive values to protect secrets.
- ✓ Use random provider to generate unique values.

Engineer_cloudops

Module 12 : Advanced Terraform (Deep Dive - Part 2)

Engineer_cloudops

5. Time Provider

- time provider is used to work with date and time in Terraform.
- Helpful for triggers, naming, or expiring resources.
- Commonly used with null_resource.

Example - Create a timestamp

```
terraform {
  required_providers {
    time = {
      source  = "hashicorp/time"
      version = ">= 0.9.0"
    }
  }
}
resource "time_timestamp" "now" {}

output "current_time" {
  value = time_timestamp.now.rfc3339
}
```

Use Case :

- Unique names
- Cache busting
- Triggering resources on time change

> **Tip :** Use time_sleep for delays in automation (rarely needed).

6. External Data Source

- The external data source allows Terraform to use data from external programs.
- The program must return JSON to stdout.
- Useful when data is not available from providers.

Example - Get data from external script

```
data "external" "get_ami" {
  program = ["bash", "${path.module}/get_ami.sh"]
}

output "ami_id" {
  value = data.external.get_ami.result.ami_id
}
```

get_ami.sh (example)

```
#!/bin/bash
# return the latest Amazon Linux 2 AMI
AMI_ID=$(aws ec2 describe-images \
    --owners amazon \
    --filters "Name=name,Values=amzn2-ami-hvm-*-x86_64-gp2" \
    --query 'Images|sort_by(@,&CreationDate)|[-1].ImageId' \
    --output text)

echo "{\"ami_id\": \"$AMI_ID\"}"
```

Use Case :

- Get dynamic values
- Integrate with 3rd party tools
- Use any CLI / script output

> **Tip :** Make sure the script is executable and returns valid JSON.

7. terraform console

- Interactive REPL to test expressions.
- Great for debugging and learning functions.
- Does not require apply.

Example - Using terraform console

```
$ terraform console
> 1 + 2
3
> var.environment
"dev"
> upper("hello")
"HELLO"
> jsonencode({a=1,b=2})
"{\"a\":1,\"b\":2}"
> [for i in [1,2,3] : i * 2]
 [2, 4, 6]
```

Use Case :

- Test expressions
- Validate complex logic
- Explore variables and locals

> **Tip :** Use console before writing complicated expressions in code.

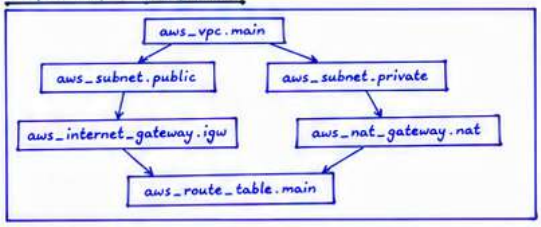
8. terraform graph

- Generates a visual graph of resources and dependencies.
- Helps to understand resource relationships.
- Useful for troubleshooting and documentation.

Example - Generate graph

```
$ terraform graph -type=plan | dot -Tpng > graph.png
```

Sample Output (Simplified)



Use Case :

- Visualize dependencies
- Find implicit dependencies
- Debug creation / destroy order

> **Tip :** Install Graphviz (dot) to render the graph image.

Master these advanced features to write smart, dynamic and production-ready Terraform code! Keep practicing and keep automating! ☺

Engineer_cloudops

Module 13 : AWS Resources EC2

Engineer_cloudops

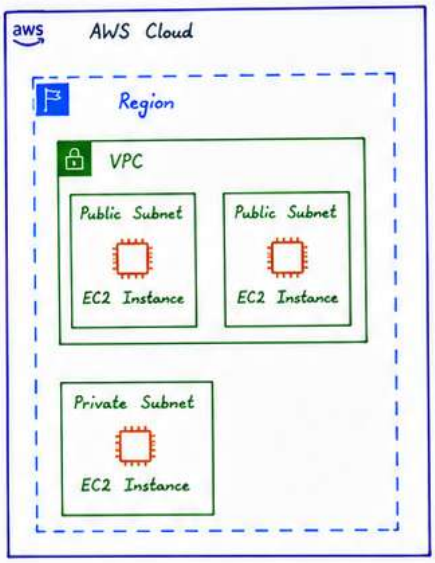
EC2 (Elastic Compute Cloud) provides resizable compute capacity in the cloud. It is the most used service in AWS.

1. What is EC2 ?

- EC2 allows us to launch virtual machines (instances) in AWS.
- Pay for what you use.
- Fully scalable and configurable.
- We can control OS, storage, networking, security etc.

2. EC2 Core Components

- AMI (Amazon Machine Image)
- Instance Type
- Key Pair
- Security Group
- Subnet
- Elastic IP
- IAM Role (for instance profile)
- User Data (for bootstrapping)
- EBS (Elastic Block Store)
- Tags



3. Basic EC2 Resource in Terraform

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0" # Amazon Linux 2
  instance_type = "t2.micro"
  key_name      = "my-key"
  subnet_id     = "subnet-0abc1234def567890"
  vpc_security_group_ids = ["sg-0a1b2c3d4e5f6g7h8"]

  tags = {
    Name        = "Terraform-EC2"
    Environment = "Dev"
  }
}
```

4. Important Notes

- Always use latest AMI IDs (use data source).
- Use appropriate instance type as per workload.
- Attach Security Group (never open all ports).
- Use IAM Role instead of access keys.
- Enable termination protection for critical servers.
- Use User Data to automate setup.
- Don't forget to tag your resources.

5. Data Source for Latest AMI

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"
  ...
}
```

6. Useful Arguments

Argument	Description
ami	AMI ID to launch the instance
instance_type	Type of instance (t2.micro, t3.medium, etc.)
key_name	Key pair name to SSH into the instance
subnet_id	Subnet where instance will launch
vpc_security_group_ids	List of Security Groups
user_data	Script to run at first boot (multi-line supported)
iam_instance_profile	IAM role for EC2
tags	Key value pairs for identification

7. User Data Example (bash script)

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "Hello from Terraform EC2" > /var/www/html/index.html
```

Tip :
User data helps to automate software installation and configuration.

EC2 is the backbone of AWS infrastructure.
Build it right, secure it tight, and automate everything! 💪

Engineer_cloudops

Module 13 : AWS Resources - VPC

Engineer_cloudops

Amazon VPC (Virtual Private Cloud) lets you launch AWS resources in a logically isolated network that you define.

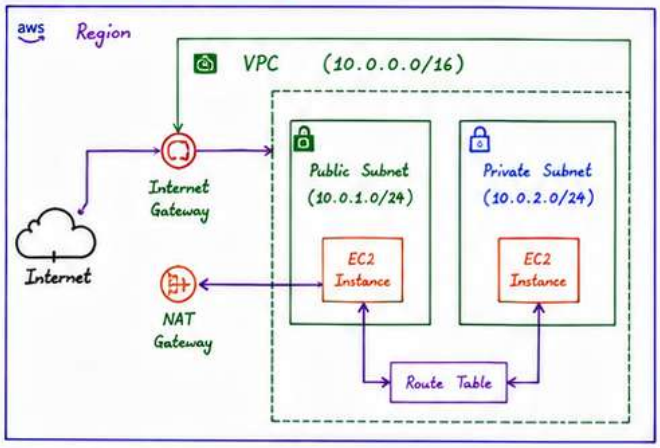
1. What is VPC ?

- Your own isolated network in AWS.
- Full control over IP address range, routing, subnets, gateways, etc.
- You can define multiple subnets (public/private).
- Secure, scalable and highly available.

2. VPC Components

- VPC
- Subnet (Public / Private)
- Internet Gateway (IGW)
- NAT Gateway
- Route Table
- Security Group
- Network ACL
- Elastic IP
- VPC Peering / Endpoints
- DHCP Options Set

3. VPC Design (Example)



4. CIDR Planning (Example)

Network	CIDR	Purpose
VPC	10.0.0.0/16	Overall VPC range
Public Subnet 1	10.0.1.0/24	Public resources
Private Subnet 1	10.0.2.0/24	Private resources
Public Subnet 2	10.0.3.0/24	For HA (AZ-2)
Private Subnet 2	10.0.4.0/24	For HA (AZ-2)

5. VPC in Terraform (Basic)

```
resource "aws_vpc" "main" {
  cidr_block       = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = {
      Name = "Terraform-VPC"
  }
}

resource "aws_subnet" "public" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
  map_public_ip_on_launch = true
  availability_zone = "ap-south-1a"
  tags = { Name = "public-subnet" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags = { Name = "igw" }
}

resource "aws_route_table" "public-rt" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "public-rt" }
}

resource "aws_route_table_association" "public-asso"{
  subnet_id = aws_subnet.public.id
  route_table_id = aws_route_table.public-rt.id
}
```

Tip :
Always enable DNS support & hostnames in VPC.

Use Cases :

- Web applications
- Backend systems
- Databases
- Microservices
- Hybrid connectivity
- Secure environments

6. Important Notes

- Public Subnet needs route to Internet Gateway.
- Private Subnet uses NAT Gateway for internet access.
- Use separate route tables for public & private subnets.
- Security Groups are stateful, NACLs are stateless.
- Plan CIDR blocks carefully. Avoid overlap.

Tip :
Keep VPC simple but scalable.

7. Common Commands (AWS CLI)

```
aws ec2 describe-vpcs                    # List all VPCs
aws ec2 describe-subnets                 # List subnets
aws ec2 describe-route-tables            # List route tables
aws ec2 describe-security-groups         # List security groups
aws ec2 describe-internet-gateways       # List IGWs
aws ec2 describe-nat-gateways            # List NAT Gateways
aws ec2 describe-network-acls            # List NACLs
```

VPC is the foundation of your AWS infrastructure.
Design it right, secure it right, and scale it right ! ☑

Engineer_cloudops

Module 13 : AWS Resources - Subnets

Engineer_cloudops

Subnets are the building blocks inside a VPC.
They divide your VPC CIDR block into smaller networks across Availability Zones.

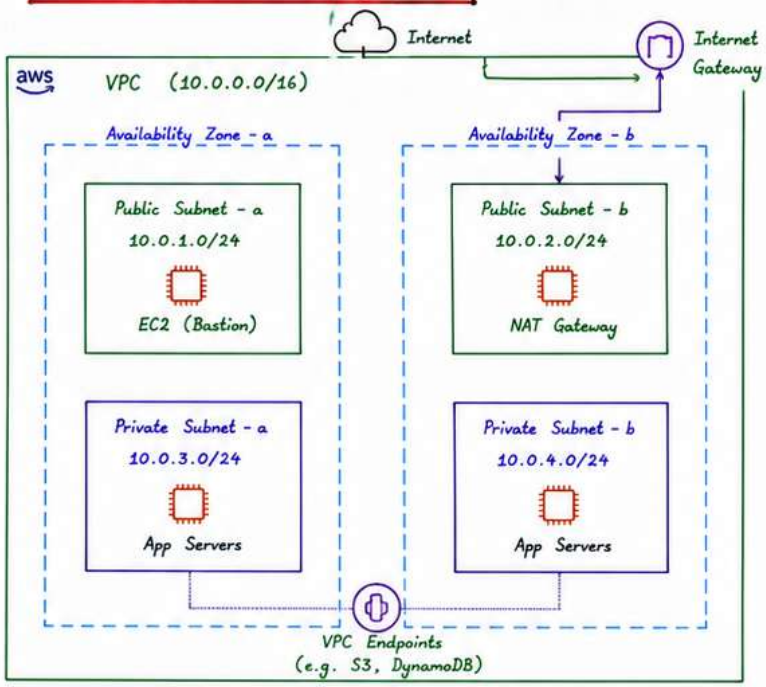
1. What is a Subnet ?

- A subnet is a range of IP addresses in your VPC.
- Resources (EC2, RDS, etc) must be launched inside a subnet.
- Each subnet exists in one Availability Zone (AZ).
- Subnets can be Public or Private.

2. Types of Subnets

Public Subnet	- Has route to Internet Gateway via Route Table. - Auto-assign Public IP = Yes - Used for resources that need internet access (Bastion, ALB, NAT GW).
Private Subnet	- No direct route to Internet. - Auto-assign Public IP = No - Used for backend systems (EC2, RDS, Apps, etc).

3. Subnet Example Architecture (2 AZs)



4. Create Subnet in Terraform (Example)

```
resource "aws_subnet" "public_subnet_a" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "ap-south-1a"
  map_public_ip_on_launch = true
  tags = {
    Name = "public-subnet-a"
    Type = "public"
  }
}
resource "aws_subnet" "private_subnet_a" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.3.0/24"
  availability_zone       = "ap-south-1a"
  map_public_ip_on_launch = false
  tags = {
    Name = "private-subnet-a"
    Type = "private"
  }
}
```

Tip :
Use different AZs for high availability. Never put all resources in one subnet !

5. Subnet Important Arguments

Argument	Description
vpc_id	ID of the VPC where subnet will be created.
cidr_block	CIDR range for the subnet (must be within VPC CIDR).
availability_zone	AZ where subnet will be created.
map_public_ip_on_launch	true = Auto-assign public IP to instances launched in this subnet.
tags	Key-value pairs for identification.

6. Route Table Association

- Each subnet must be associated with a Route Table.
- Public Subnet → Route Table with route to Internet Gateway.
- Private Subnet → Route Table with route to NAT Gateway (or VPC Endpoints).

Tip :
Always check subnet route table to understand if it's public or private.

7. Useful Commands (AWS CLI)

```
aws ec2 describe-subnets                                          # List all subnets
aws ec2 describe-subnets --filters "Name=vpc-id,Values=vpc-123"   # List subnets in a VPC
aws ec2 describe-route-tables                                     # List route tables
aws ec2 describe-route-tables --filters "Name=association.subnet-id,Values=subnet-123"
                                                                  # Route table of a subnet
```

8. Best Practices

- ✓ Use separate public and private subnets.
- ✓ Distribute subnets across multiple AZs.
- ✓ Keep CIDR blocks non-overlapping.
- ✓ Use appropriate route tables.
- ✓ Enable DNS support & hostnames in VPC.

Well designed subnets = Secure, Scalable and Highly Available AWS architecture ! ☑

Engineer_cloudops

Module 13 : AWS Resources - Internet Gateway

Engineer_cloudops

An Internet Gateway (IGW) is a horizontally scaled, redundant, and highly available VPC component that allows communication between instances in your VPC and the internet.

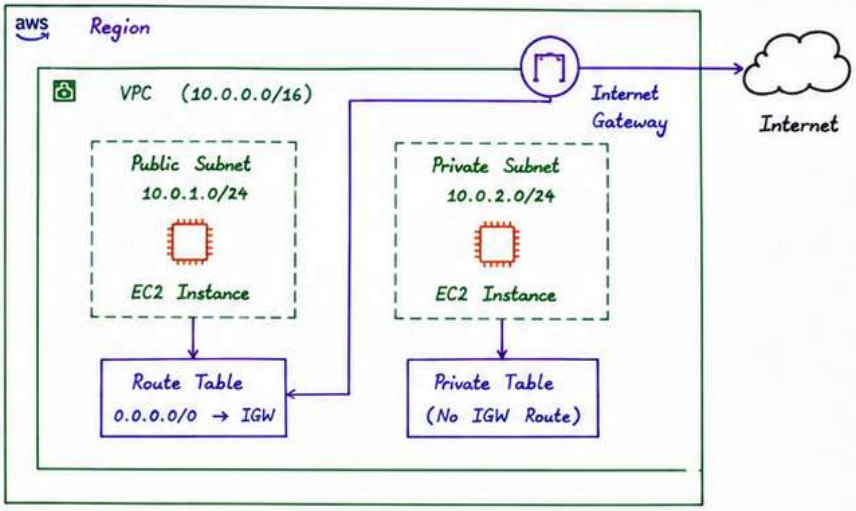
1. What is Internet Gateway ?

- It is a VPC component.
- It provides a target for internet-routable traffic.
- Required for resources in Public Subnet to access the internet.
- Attached to a VPC (one IGW per VPC).

2. Key Points

- Must be attached to a VPC.
- Used in conjunction with a Route Table.
- Add a route 0.0.0.0/0 → IGW in the public route table.
- Does not connect to on-prem directly (use VPN/Direct Connect for that).

3. Architecture Overview



Tip : Only resources in Public Subnet with route to IGW can access the Internet.

4. Create Internet Gateway in Terraform

```
# Internet Gateway
resource "aws_internet_gateway" "igw" {
  vpc_id =  aws_vpc.main.id
  tags = {
    Name    =  "terraform-igw"
  }
}
```

Tip : IGW is created inside a VPC using vpc_id.

5. Attach IGW to VPC (Same resource above)

- aws_internet_gateway resource automatically attaches the IGW to the VPC when vpc_id is provided.
- No separate attachment resource required.

6. Add Route in Public Route Table

```
resource "aws_route_table" "public_rt" {
  vpc_id =  aws_vpc.main.id

  route {
    cidr_block    =  "0.0.0.0/0"
    gateway_id    =  aws_internet_gateway.igw.id
  }

  tags = {
    Name     =  "public-route-table"
  }
}
```

Tip : Route 0.0.0.0/0 tells AWS to send all internet traffic to the IGW.

7. Associate Public Subnets to Route Table

```
resource "aws_route_table_association" "public_a" {
  subnet_id       =  aws_subnet.public_a.id
  route_table_id  =  aws_route_table.public_rt.id
}

resource "aws_route_table_association" "public_b" {
  subnet_id       =  aws_subnet.public_b.id
  route_table_id  =  aws_route_table.public_rt.id
}
```

Tip : Associate public subnets with the route table that has IGW route.

8. Verify (AWS CLI)

```
# List Internet Gateways
aws ec2 describe-internet-gateways
# Describe Route Tables
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=<vpc-id>"
# Check IGW attached to VPC
aws ec2 describe-internet-gateways --internet-gateway-ids <igw-id>
```

9. Best Practices

- ✓ Create one IGW per VPC.
- ✓ Use IGW only for resources that need internet access.
- ✓ Keep private subnets without IGW route.
- ✓ Use NAT Gateway for private subnet internet access.
- ✓ Tag your IGW for easy identification.

IGW + Public Route Table + Public Subnet = Internet Access ☑

Engineer_cloudops

Module 13 : AWS Resources - NAT Gateway

Engineer_cloudops

NAT Gateway (Network Address Translation Gateway) allows instances in a private subnet to access the internet or AWS services, but prevents the internet from initiating a connection with those instances.

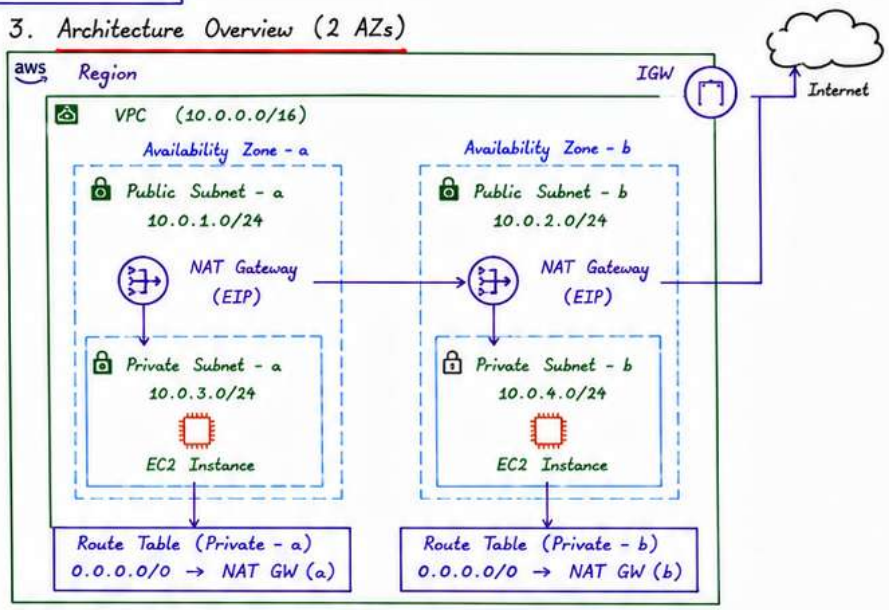
1. What is NAT Gateway ?

- Managed AWS service.
- Placed in a Public Subnet.
- Allows outbound internet access for instances in Private Subnet.
- Uses Elastic IP.
- Highly available within an AZ.
- Automatically scales bandwidth.

2. Key Points

- Created in a Public Subnet.
- Needs route in Private Route Table.
- Uses Elastic IP (EIP).
- One NAT Gateway per AZ for HA.
- Only allows outbound connections.
- Charged per hour + data processed.

3. Architecture Overview (2 AZs)



Private instances can access internet via NAT Gateway.
Internet cannot initiate connections to private instances.

4. Create NAT Gateway in Terraform

```
resource "aws_eip" "nat_eip" {
  domain = "vpc"
  tags   = { Name = "nat-eip" }
}

resource "aws_nat_gateway" "nat_gw" {
  allocation_id = aws_eip.nat_eip.id
  subnet_id     = aws_subnet.public_subnet_a.id
  tags = { Name = "nat-gateway-a" }
}
```

Tip : NAT Gateway must be in Public Subnet with route to IGW.

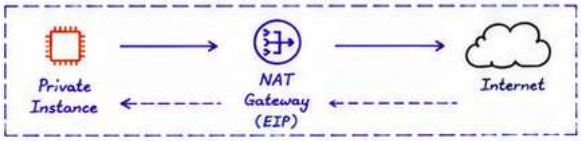
5. Add Route in Private Route Table

```
resource "aws_route" "private_route" {
  route_table_id         = aws_route_table.private_rt_a.id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id         = aws_nat_gateway.nat_gw.id
}
```

Tip : Set destination 0.0.0.0/0 to NAT Gateway, not IGW.

6. How it Works ?

1. Instance in Private Subnet wants to access internet.
2. Traffic goes to Route Table -> matched 0.0.0.0/0.
3. Route points to NAT Gateway.
4. NAT Gateway uses EIP to access internet via IGW.
5. Response comes back to NAT Gateway and forwarded to the instance.



7. Terraform - Full Example (simplified)

```
# EIP for NAT
resource "aws_eip" "nat_eip" {
  domain = "vpc"
}

# NAT Gateway
resource "aws_nat_gateway" "nat_gateway" {
  allocation_id = aws_eip.nat_eip.id
  subnet_id     = aws_subnet.public_subnet.id
  depends_on    = [aws_internet_gateway.igw]
}

# Route in Private Route Table
resource "aws_route" "private_nat_route" {
  route_table_id         = aws_route_table.private_rt.id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id         = aws_nat_gateway.nat_gateway.id
}
```

Tip : Use depends_on or create IGW before NAT GW.

8. Verify (AWS CLI)

```
# List NAT Gateways
aws ec2 describe-nat-gateways
# List Route Tables
aws ec2 describe-route-tables --filters "Name=association.main,Values=false"
# Check NAT Status
aws ec2 describe-nat-gateways --nat-gateway-ids <nat-gw-id>
```

9. Best Practices

- ✓ Deploy one NAT Gateway per AZ.
- ✓ Use Private Route Table for private subnets.
- ✓ Place NAT Gateway in Public Subnet.
- ✓ Monitor with CloudWatch metrics.
- ✓ Use for outbound traffic only.

NAT Gateway = Secure Outbound Internet Access for Private Resources ! ☑

Engineer_cloudops

Module 13 : AWS Resources - Route Tables

Engineer_cloudops

A Route Table contains a set of rules (routes) that are used to determine where network traffic is directed. Every subnet must be associated with a Route Table.

1. What is Route Table ?

- A route table is a set of rules.
- It tells where the network traffic should go.
- Each VPC has a Main Route Table (created by default).
- You can create custom route tables.
- Subnets are associated with a route table.

2. Key Points

- Routes are evaluated in order.
- Longest prefix match is used.
- If no match, the traffic is dropped.
- You can add routes to IGW, NAT GW, TGW, Peering, VPC Endpoint, or other resources.
- Each subnet can be associated with only one route table at a time.

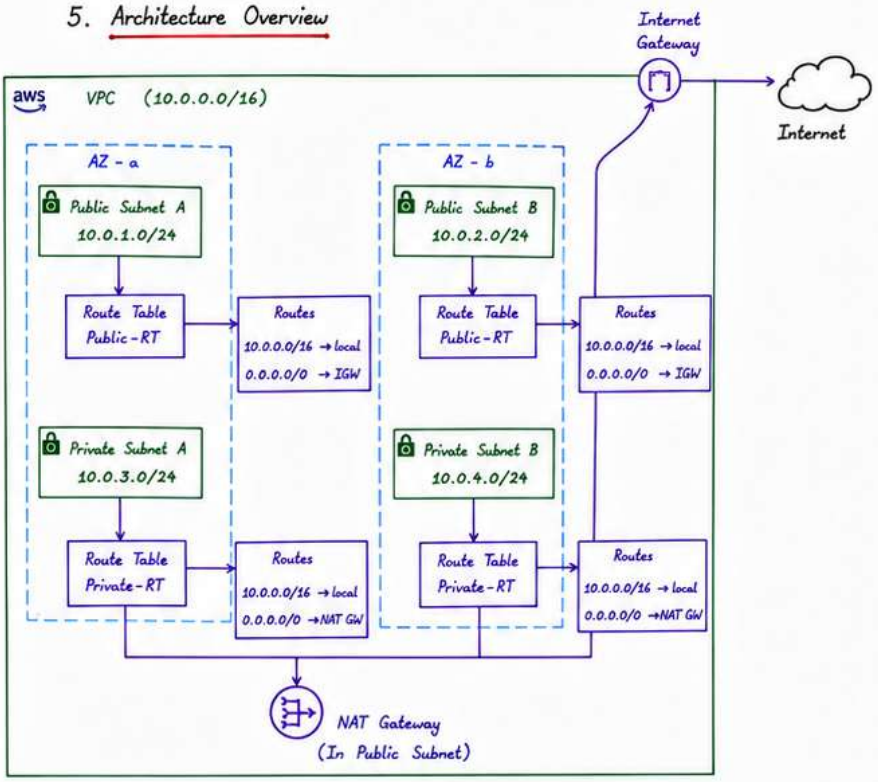
3. Default Route Tables in a VPC

Route Table	Description
Main Route Table	Automatically created. Associated with new subnets by default.
Custom Route Table	Created by you. Associate with specific subnets.

4. Route Table Example

Destination	Target	Purpose
10.0.0.0/16	local	Traffic within the VPC
0.0.0.0/0	igw-12345678	Internet access (Public Subnet)
0.0.0.0/0	nat-0abc12345	Internet access for Private Subnet
10.1.0.0/16	pcx-0aaabbbb	Traffic to Peered VPC
pl-xxxxxxxx	vpce-0abcdefff	PrivateLink / VPC Endpoint

5. Architecture Overview



6. Route Table Association

- Associate a route table with a subnet.
- When an instance sends traffic, AWS checks the association and uses that route table.
- You can change the association anytime.
- Main route table association cannot be removed.

Tip : New subnet is associated with Main Route Table by default unless you specify a custom one.

7. Create Route Table in Terraform

```
resource "aws_route_table" "public_rt" {
  vpc_id       = aws_vpc.main.id

  route {
    cidr_block     = "0.0.0.0/0"
    gateway_id     = aws_internet_gateway.igw.id
  }

  tags = { Name = "public-rt" }
}
```

Tip : Add required routes based on subnet type.

8. Associate Route Table with Subnet (Terraform)

```
resource "aws_route_table_association" "public_a_assoc" {
  subnet_id      = aws_subnet.public_a.id
  route_table_id = aws_route_table.public_rt.id
}
```

Tip : One subnet → One route table association.

9. Useful AWS CLI Commands

```
aws ec2 describe-route-tables                              # List route tables
aws ec2 create-route-table --vpc-id vpc-123                # Create route table
aws ec2 create-route --route-table-id rtb-123 --destination-cidr-block 0.0.0.0/0 \
                     --gateway-id igw-123                  # Add route to IGW
aws ec2 associate-route-table --subnet-id subnet-123 --route-table-id rtb-123 \
                                                           # Associate subnet
aws ec2 disassociate-route-table --association-id rtbassoc-123 # Disassociate
aws ec2 delete-route --route-table-id rtb-123 --destination-cidr-block ©0. # Delete route
```

10. Best Practices

- ✓ Use separate route tables for Public and Private subnets.
- ✓ Public subnet route table → 0.0.0.0/0 to IGW.
- ✓ Private subnet route table → 0.0.0.0/0 to NAT Gateway.
- ✓ Keep routes simple and minimal.
- ✓ Review associations regularly.

Route Tables are the GPS of your VPC - Send traffic in the right direction ! ☑

Engineer_cloudops

Module 13 : AWS Resources - Security Groups

Security Group (SG) is a virtual firewall for your EC2 instances. It controls inbound and outbound traffic based on rules.

1. What is Security Group ?

- → Acts as a stateful firewall
- → Controls inbound (incoming) and outbound (outgoing) traffic.
- → Works at instance level (not subnet level).
- → Allows or denies traffic based on rules.
- → By default, all inbound traffic is blocked and all outbound traffic is allowed.

2. Key Characteristics

- → Stateful (If you allow request, response is automatically allowed)
- → Supports allow rules only (no explicit deny).
- → Can reference IP, CIDR or another Security Group.
- → Can be attached to multiple instances.

3. Inbound vs Outbound Rules

Type	Direction	Purpose	Example
Inbound	Incoming traffic	Allow traffic to your instance	Allow SSH from your IP (22/tcp)
Outbound	Outgoing traffic	Allow instance to access other resources	Allow all outbound (0.0.0.0/0)

4. SG Rule Example

Type	Protocol	Port Range	Source / Destination	Description
Inbound	TCP	22	MyIP/32	SSH access
Inbound	TCP	80	0.0.0.0/0	HTTP access
Inbound	TCP	443	0.0.0.0/0	HTTPS access
Outbound	All	All	0.0.0.0/0	All outbound

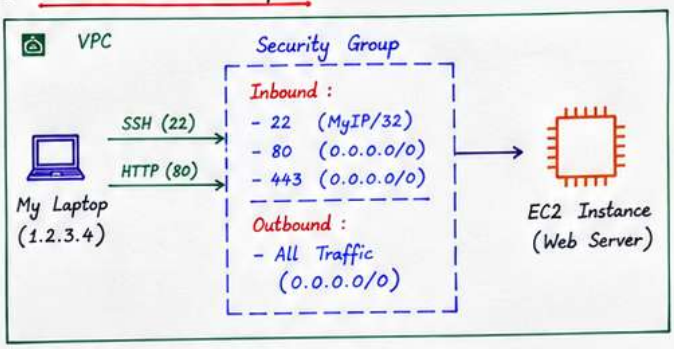
5. Create Security Group (Terraform)

```
resource "aws_security_group" "web_sg" {
  name        = "web-sg"
  description = "Allow HTTP and SSH"
  vpc_id      = aws_vpc.main.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["1.2.3.4/32"]   # your IP
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Tip :
Use small, specific CIDR instead of 0.0.0.0/0 for better security.

6. Architecture Example



7. Useful AWS CLI Commands

```
# List security groups
aws ec2 describe-security-groups

# Create security group
aws ec2 create-security-group --group-name web-sg \
  --description "Web SG" --vpc-id vpc-12345678

# Add inbound rule (SSH)
aws ec2 authorize-security-group-ingress --group-id sg-abcde \
  --protocol tcp --port 22 --cidr 1.2.3.4/32

# Add inbound rule (HTTP)
aws ec2 authorize-security-group-ingress --group-id sg-abcde \
  --protocol tcp --port 80 --cidr 0.0.0.0/0

# Delete rule
aws ec2 revoke-security-group-ingress --group-id sg-abcde \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
```

8. Best Practices

- ✓ Allow only required ports.
- ✓ Use specific IP / CIDR instead of 0.0.0.0/0.
- ✓ Use separate SGs for differenct tiers (web, app, db).
- ✓ Use SG referencing instead of IP where possible.
- ✓ Regularly review and remove unused rules.
- ✓ Follow least privilege principle.

9. Common Use Cases

- → Allow SSH access to bastion host.
- → Allow HTTP/HTTPS access to web servers.
- → Allow app server to communicate with DB using SG reference.
- → Restrict access to internal resources.

Remember :
Security Groups are your 1st line of defense - keep them tight 🔒

Engineer_cloudops

Module 13 : AWS Resources – Network ACL (NACL)

Engineer_cloudops

Network ACL (NACL) is a stateless firewall that controls traffic at the subnet level. It acts as a security layer before traffic reaches any instance in the subnet.

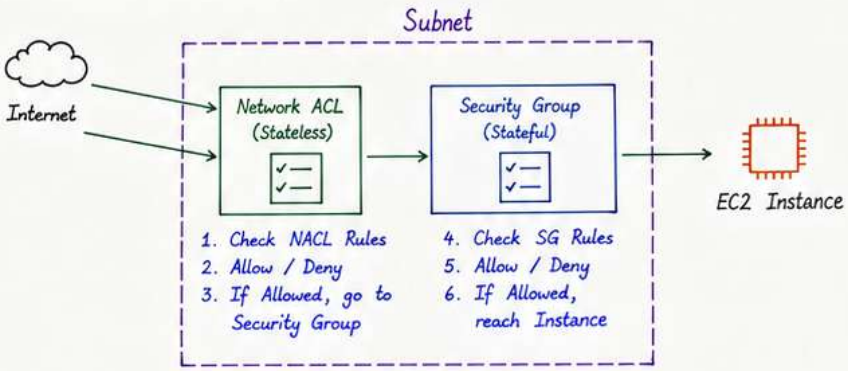
1. What is Network ACL ?

- Operates at the subnet level.
- Stateless (does not remember previous traffic).
- Evaluated before Security Group.
- Has numbered rules, evaluated in order.
- Supports Allow and Deny rules.
- Default NACL is created for every VPC.

2. Key Points

- One NACL can be associated with multiple subnets.
- By default, Allow all inbound and outbound.
- Evaluated in ascending order (by rule number).
- If no rule matches, the default rule (*) is applied.
- Lower number = higher priority.

3. How NACL Works ?



4. Default NACL Rules

Every VPC has a default NACL with these rules :

Rule No.	Type	Traffic	Protocol	Port Range	Source / Destination	Action
100	Inbound	All Traffic	All	All	0.0.0.0/0	ALLOW
*	Inbound	All Traffic	All	All	0.0.0.0/0	ALLOW
100	Outbound	All Traffic	All	All	0.0.0.0/0	ALLOW
*	Outbound	All Traffic	All	All	0.0.0.0/0	ALLOW

(*) = If no rule matches, this rule is applied.

5. NACL Rule Evaluation (Stateless)

- Rules are evaluated in order (low number to high).
- First match is applied.
- NACL is stateless → return traffic must be explicitly allowed.
- Both Inbound and Outbound rules are evaluated.

Example :
If you allow inbound HTTP (80), you must also allow outbound HTTP (80) for response to come back.

6. Example NACL Rules

Allow SSH from specific IP and HTTP from anywhere.

Inbound Rules

Rule No.	Type	Protocol	Port	Source	Action
100	SSH	TCP	22	1.2.3.4/32	ALLOW
110	HTTP	TCP	80	0.0.0.0/0	ALLOW
*	All	ALL	ALL	0.0.0.0/0	DENY

Outbound Rules

Rule No.	Type	Protocol	Port	Destination	Action
100	All	ALL	ALL	0.0.0.0/0	ALLOW
*	All	ALL	ALL	0.0.0.0/0	DENY

Tip :
- Always add an explicit DENY (*) rule at the end.
- Be very careful, one wrong rule can block traffic.

7. NACL vs Security Group

Feature	Network ACL (NACL)	Security Group (SG)
Level	Subnet Level	Instance Level
Stateful	No (Stateless)	Yes (Stateful)
Rule Type	Allow / Deny	Allow Only
Rule Order	Numbered (in order)	No order (all rules evaluated)
Default	Allow all	Deny all
Return Traffic	Must be explicitly allowed	Automatically allowed
Associated With	Subnet (1 NACL → Many Subnets)	ENI / Instance (Many SGs → Many ENIs)

8. Useful AWS CLI Commands

```
# List NACLs
aws ec2 describe-network-acls

# Create NACL
aws ec2 create-network-acl --vpc-id vpc-12345678

# Create Inbound Rule
aws ec2 create-network-acl-entry --network-acl-id acl-12345678 \
    --rule-number 100 --protocol 6 --port-range From=22,To=22 \
    --rule-action allow --cidr-block 1.2.3.4/32 --ingress

# Associate NACL with Subnet
aws ec2 associate-network-acl --network-acl-id acl-12345678 \
    --subnet-id subnet-12345678
```

Note :
NACLs are evaluated before Security Groups. Use both for better security.

9. Best Practices

- ✓ Use NACLs for broad access control at subnet level.
- ✓ Use Security Groups for fine-grained access at instance level.
- ✓ Always keep a DENY (*) rule at the end.
- ✓ Test rules carefully, one wrong rule can lock you out.
- ✓ Document your rules.

NACL + SG = Defense in Depth (Strong Security) ☑

Engineer_cloudops

Module 13: AWS Resources – IAM (Identity and Access Management)

Engineer_cloudops

IAM helps you securely control access to AWS services and resources. You manage "who" can access and "what" they can do.

1. What is IAM ?

- Global AWS service.
- Manage users, groups, roles and policies.
- Implements least privilege access.
- No additional charge.

2. IAM Components

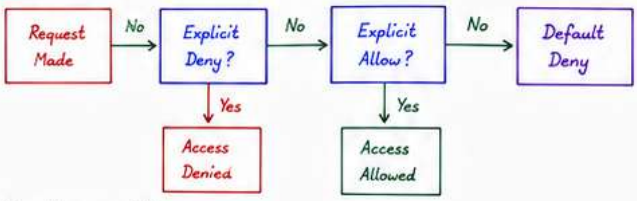
- Users → Human users
- Groups → Collection of users
- Roles → For AWS services or cross-account access
- Policies → JSON documents that define permissions
- Password Policies → Strength & rotation rules
- MFA → Extra layer of security
- Access Keys → For programmatic access

3. IAM Policy Structure (JSON)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject"],
      "Resource": ["arn:aws:s3:::my-bucket",
                   "arn:aws:s3:::my-bucket/*"]
    }
  ]
}
```

→ Policy Version
→ Allow or Deny
→ API actions
→ Resources (ARN)

4. IAM Policy Evaluation Logic



5. User vs Role

Feature	User	Role
Purpose	Human access	AWS service or cross-account access
Authentication	Password / MFA	Assumed by service or user
Access Keys	Can have	Temporary (via STS)
Best For	People	Applications, Services, Automation

6. Best Practices

- ✓ Apply least privilege access.
- ✓ Use Groups to manage permissions.
- ✓ Enable MFA for all users.
- ✓ Rotate Access Keys regularly.
- ✓ Use Roles for EC2, Lambda and Cross-Account access.
- ✓ Audit permissions using IAM Access Analyzer.
- ✓ Never use Root user for daily tasks.

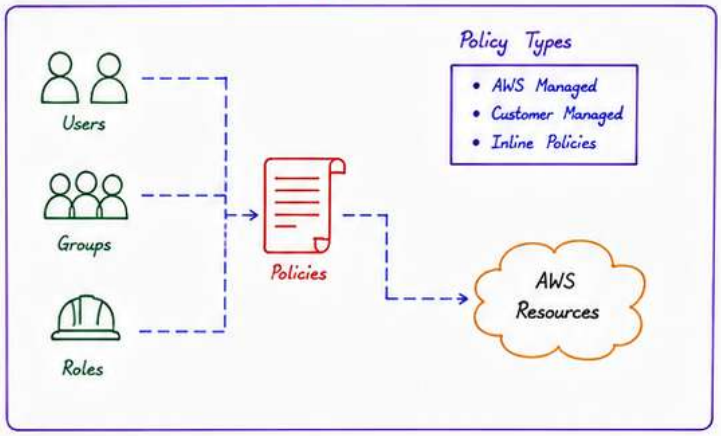
7. IAM Access Key Types

Type	Use Case
Access Key (Long-term)	For CLI, SDK, Tools (store securely)
Access Key (Short-term via STS)	For temporary access (recommended)

8. Common IAM Policies (Examples)

- → ReadOnlyAccess – Read access to all services
- → AdministratorAccess – Full access (use carefully !)
- → AmazonS3FullAccess – Full access to S3
- → PowerUserAccess – Manage most resources but not IAM
- → Customized Policy – Create your own

9. IAM Architecture Overview



10. Example: EC2 Role (Use Case)

- Create a role for EC2.
- Attach policy AmazonS3ReadOnlyAccess.
- Attach the role to EC2 instance.
- EC2 can now read data from S3 without Access Keys.

Why ?
Better security → No need to store Access Keys on the instance.

11. Useful AWS CLI Commands

```
# List users
aws iam list-users

# Create user
aws iam create-user --user-name devuser

# Create group
aws iam create-group --group-name developers

# Add user to group
aws iam add-user-to-group --user-name devuser --group-name developers

# Attach policy
aws iam attach-group-policy --group-name developers --policy-arn
    arn:aws:iam::aws:policy/ReadOnlyAccess

# List attached policies for user
aws iam list-attached-user-policies --user-name devuser
```

Tip:
Use IAM Access Analyzer to find unused permissions.

Least Privilege + Strong Policies = Secure AWS Environment ☑

Engineer_cloudops

Module 13 : AWS Resources – S3 (Simple Storage Service)

Engineer_cloudops

> Amazon S3 is an object storage service that offers industry-leading scalability, data availability, security, and performance.

1. What is S3 ?

- Object storage service.
- Stores data as objects inside buckets.
- Each object = data + metadata + unique key.
- 99.9999999999% (11 9's) durability.
- Use cases: backups, static websites, data lakes, logs, media hosting, etc.

2. S3 Key Concepts

- Bucket : Container for objects (globally unique name).
- Object : File + metadata (key, value).
- Key : Unique name of the object in bucket.
- Region : Where your bucket lives.
- Storage Class : Defines cost & access pattern.
- ACL / Bucket Policy : Control access.

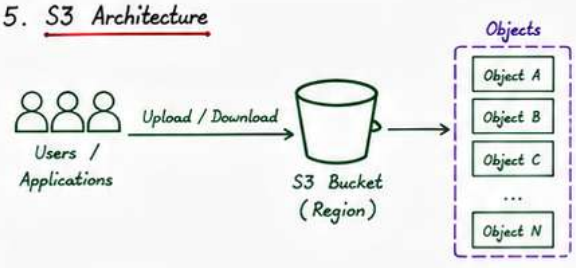
3. S3 Storage Classes

Storage Class	Use Case	Durability	Availability	Min Storage Duration	Cost
Standard	Frequent access	11 9's	99.99%	-	$$
Intelligent-Tiering	Unknown access	11 9's	99.9%-99.99%	-	$$
Standard-IA	Infrequent access	11 9's	99.9%	30 days	$
One Zone-IA	Recreatable data	11 9's	99.5%	30 days	$
Glacier Instant	Archive, rare access	11 9's	99.9%	90 days	$
Glacier Flexible	Long term archive	11 9's	99.9%	90 days	$
Glacier Deep Archive	Very rarely access	11 9's	99.9%	180 days	$

4. S3 Features

- Unlimited storage.
- Versioning (keep multiple versions of object).
- Lifecycle rules (move to IA/Glacier, expire).
- Replication (CRR - cross region, SRR - same region).
- Encryption (SSE-S3, SSE-KMS, SSE-C).
- Static website hosting.
- Object tagging.
- Event notifications (SNS, SQS, Lambda).

5. S3 Architecture



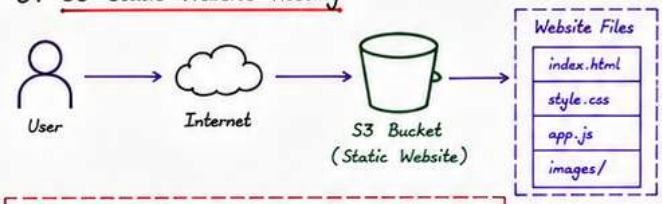
6. S3 Security

- IAM policies (user/role level).
- Bucket Policies (resource level).
- Block Public Access.
- Encryption at rest (SSE-S3 / SSE-KMS / SSE-C).
- Encryption in transit (HTTPS).
- Access logging.

7. S3 Use Cases

- Store application backups.
- Host static websites.
- Big data & analytics (data lake).
- Store logs (ALB logs, CloudTrail, VPC Flow Logs).
- Media files (images, videos).
- Software / package hosting.

8. S3 Static Website Hosting



> **Note :** Enable Static Website Hosting in bucket properties and make files publicly readable (via policy or ACL).

9. S3 Lifecycle Example

Day	Action
0 - 30	Objects in Standard
31 - 90	Move to Standard-IA
91 - 365	Move to Glacier Instant
> 365	Move to Glacier Deep Archive
> 1095	Expire/Delete

10. Useful AWS CLI Commands

```
# List buckets          aws s3 ls
# Create bucket         aws s3 mb s3://my-bucket-name --region ap-south-1
# Upload file           aws s3 cp file.txt s3://my-bucket-name/
# Download file         aws s3 cp s3://my-bucket-name/file.txt .
# Sync folder           aws s3 sync /local/path s3://my-bucket-name/
# Delete object         aws s3 rm s3://my-bucket-name/file.txt
# Delete bucket (empty) aws s3 rb s3://my-bucket-name
```

> **Tip :** Use meaningful bucket names and enable versioning for important data.

11. Best Practices

- ✓ Use versioning for important buckets.
- ✓ Enable MFA delete (extra protection).
- ✓ Use lifecycle rules to reduce cost.
- ✓ Block public access unless required.
- ✓ Encrypt data at rest and in transit.
- ✓ Monitor with S3 Storage Lens and CloudWatch.

> **Remember :** S3 is highly durable but not a backup. Always have a backup strategy.

12. S3 Naming Rules

- 3 to 63 characters.
- Lowercase letters, numbers, dots, hyphens.
- Start and end with letter or number.
- No spaces or underscores.
- Globally unique.

S3 = Simple, Scalable, Secure Storage for Every Need ! ☑

Engineer_cloudops

Module 13 : AWS Resources – EBS (Elastic Block Store)

Engineer_cloudops

Amazon EBS provides persistent block storage volumes for EC2 instances. It's like a virtual hard disk which you can attach to instances.

1. What is EBS ?

- Persistent block storage for EC2.
- Automatically replicated within an Availability Zone.
- Data is safe even if instance stops.
- Used for OS, databases, applications, logs, etc.
- Supports snapshots and encryption.

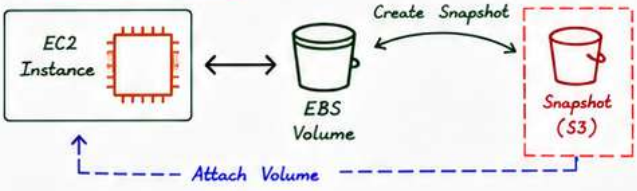
2. EBS Key Concepts

- Volume : Actual storage device (like a disk).
- Snapshot : Backup of a volume stored in S3.
- AMI : Can be created from EBS snapshot.
- Volume Type : SSD/HDD based performance.
- Availability Zone : EBS volume is AZ specific.

3. EBS Volume Types

Type	Best For	Performance	Use Case
gp3	General Purpose (SSD)	Baseline + Burst (Up to 16k IOPS)	Most workloads, Web, App, Dev
gp2	General Purpose (SSD)	3 IOPS/GB (Burst up to 16k)	Small to medium workloads
io1/io2	Provisioned IOPS (SSD)	Up to 64k IOPS (io2 up to 256k)	High performance DB, Critical apps
st1	Throughput Optimized HDD	High throughput up to 500 MB/s	Big data, Data warehousing
sc1	Cold HDD	Low cost, infrequent access	Logs, backups, archival data

4. EBS Architecture



5. EBS Lifecycle

State	Description
In-use	Attached to an EC2 instance.
Available	Not attached, can be attached to another instance.
Creating	New volume is being created.
Deleting	Volume is being deleted.
Error	Something went wrong.

6. EBS Features

- Persistent data.
- Automatic replication within AZ.
- Snapshots for backup.
- Encryption at rest (KMS).
- Fast snapshot restore.
- Multi-Attach (io1/io2/io2 Block Express).
- Resizable (modify volume size).
- Works with AMI.

7. EBS Snapshots

- Point-in-time backup stored in S3.
- Incremental (only changed blocks).
- Can copy snapshots across regions.
- Low cost compared to EBS volume.

Tip : Always enable EBS encryption and regular snapshots for production volumes.

8. Common Use Cases

- OS disk for EC2.
- Database storage (RDS on EC2, MySQL, PostgreSQL).
- Application data storage.
- Logs and monitoring data.
- Temporary scratch disks.

9. Useful AWS CLI Commands

```
# Create EBS volume
aws ec2 create-volume --availability-zone ap-south-1a \
    --size 20 --volume-type gp3

# List volumes
aws ec2 describe-volumes

# Attach volume to instance
aws ec2 attach-volume --volume-id vol-1234567890abcdef0 \
    --instance-id i-0abc123def456 --device /dev/xvdf

# Create snapshot
aws ec2 create-snapshot --volume-id vol-1234567890abcdef0 \
    --description "Daily backup"

# List snapshots
aws ec2 describe-snapshots --owner-ids self
```

10. EBS Best Practices

- ✓ Use gp3 for most workloads (cost effective).
- ✓ Keep regular snapshots (automate with AWS Backup).
- ✓ Encrypt EBS volumes.
- ✓ Delete unused volumes and snapshots.
- ✓ Use io2/io2 Block Express for critical databases.
- ✓ Monitor with CloudWatch (VolumeReadOps, VolumeWriteOps, DiskQueueDepth).

11. EBS vs Instance Store

Feature	EBS	Instance Store
Persistence	Yes (Persistent)	No (Ephemeral)
AZ	Replicated within AZ	Local to hardware
Cost	Paid	Included in instance cost
Use Case	OS, DB, Important data	Cache, Temp, Buffer
Snapshot	Yes	No

EBS = Persistent, Reliable, Scalable Block Storage for EC2 ☑

Engineer_cloudops

Module 13 : AWS Resources – EIP (Elastic IP)

Engineer_cloudops

Elastic IP (EIP) is a static, public IPv4 address from AWS that you can allocate to your AWS account and associate with EC2 instances, NAT gateways, or Network Interfaces.

1. What is EIP ?

- A static public IPv4 address provided by AWS.
- Remains the same until you release it.
- Used for instances that need consistent IP (e.g., SSH, Web servers, Whitelisting).
- You pay for EIP when it is not associated with a running resource.

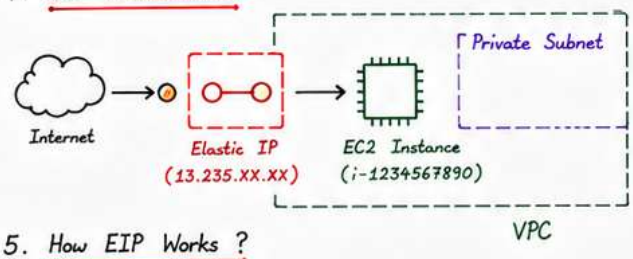
2. Key Concepts

- Public IPv4 address.
- Allocated in a specific Region.
- Can be associated with:
  - EC2 Instance
  - Network Interface (ENI)
  - NAT Gateway
- Can be remapped (disassociated and associated with another resource).

3. EIP Use Cases

✓ Remote access to EC2 (SSH/RDP) with stable IP.
✓ Hosting applications with fixed IP requirements.
✓ NAT Gateway for outbound internet access from private subnets.
✓ Whitelisting IP in firewall / security systems.

4. EIP Architecture



5. How EIP Works ?

1. You allocate an Elastic IP from the AWS pool.
2. Associate it with a resource (EC2 / ENI / NAT GW).
3. Traffic from internet reaches the EIP.
4. EIP forwards traffic to the associated resource.
5. If needed, you can disassociate and associate it to another resource.

Tip :
If an instance is stopped and has an EIP associated, the EIP remains with that instance when it is started again. If you disassociate, you can associate it with another instance.

6. EIP Lifecycle

State	Description
Allocated	EIP is allocated to your AWS account (not associated to any resource).
Associated	EIP is associated with an instance, ENI or NAT Gateway.
Disassociated	EIP is not attached to any resource, but still allocated.
Released	EIP is released back to AWS and can't be used.

7. EIP Types and Limits

- EIP is IPv4 only.
- By default, you can allocate 5 EIPs per Region (can be increased by quota request).
- Each EIP is tied to a Region.

8. EIP Pricing (As of now)

Scenario	Cost
EIP associated with a resource	Free
EIP not associated	$0.005 per hour
Data transfer	Standard data transfer charges apply

9. Useful AWS CLI Commands

```
# Allocate EIP
aws ec2 allocate-address --domain vpc
# Describe EIPs
aws ec2 describe-addresses
# Associate EIP with Instance
aws ec2 associate-address --instance-id i-1234567890abcdef0 \
  --allocation-id eipalloc-0123456789abcdef0
# Disassociate EIP
aws ec2 disassociate-address --association-id eipassoc-0123456789abcdef0
# Release EIP
aws ec2 release-address --allocation-id eipalloc-0123456789abcdef0
```

10. EIP vs Public IP

Feature	EIP (Elastic IP)	Public IP
Type	Static	Dynamic
Changes on Stop/Start	No	Yes
Charge	Free (when in use) $0.005/hr (not in use)	Always Free
Use Case	Stable Applications	Temporary Use
Can Remap	Yes	No

EIP = Static, Reliable & Remappable Public IP for Your AWS Resources ☑

Engineer_cloudops

Module 13: AWS Resources – ALB (Application Lodd Balancer)

Engineer_cloudops

Application Load Balancer (ALB) operates at Layer 7 (Application layer) and is used to route HTTP/HTTPS traffic to targets such as EC2 instances, containers, IP addresses, and Lambda.

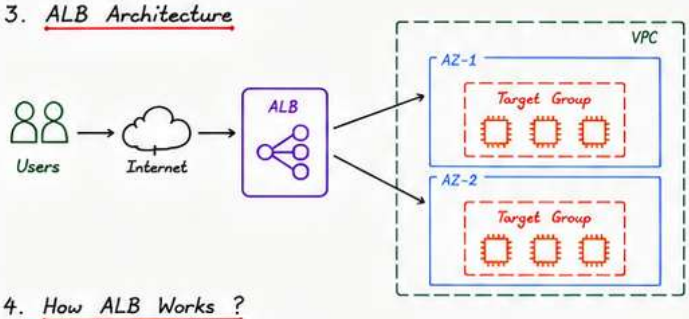
1. What is ALB ?

- Layer 7 Load Balancer.
- Routes traffic based on content (Host, Path, Headers, Query).
- Supports HTTP and HTTPS.
- Ideal for microservices, web apps, containers.
- Provides advanced routing, security and monitoring.

2. ALB Key Features

- Content-based routing (Host / Path / Header).
- Integrated with AWS WAF.
- Sticky sessions (via Cookies).
- SSL/TLS termination.
- Health checks for targets.
- Access logs to S3.
- High availability across AZs.

3. ALB Architecture



4. How ALB Works ?

1. Client sends request to ALB DNS name.
2. ALB listener (HTTP/HTTPS) receives the request.
3. Listener rules evaluate (based on Host/Path/Headers).
4. Request is forwarded to the target group.
5. ALB performs health check and sends traffic to healthy targets.
6. Response is returned back to the client.

5. ALB Components

Component	Description
Load Balancer	The ALB resource with DNS name.
Listener	Checks for connection requests using port & protocol.
Listener Rule	Defines how traffic is routed (conditions + actions).
Target Group	Collection of targets (EC2/IP/Lambda).
Health Check	Monitors target health and removes unhealthy ones.
Security Group	Controls inbound/outbound traffic to ALB.

9. Useful AWS CLI Commands

```
# Create ALB
aws elbv2 create-load-balancer --name app-alb --subnets subnet-aaa,subnet-bbb \
  --security-groups sg-12345678 --type application --scheme internet-facing

# Create Target Group
aws elbv2 create-target-group --name app-tg --protocol HTTP --port 80 \
   --vpc-id vpc-12345678 --health-check-path /health

# Register target
aws elbv2 register-targets --target-group-arn <tg-arn> --targets Id=i-0abcd1234efgh5678,Port=80

# Describe ALB
aws elbv2 describe-load-balancers --names app-alb
```

10. Best Practices

- ✓ Use ALB across multiple AZs for high availability.
- ✓ Enable access logs and store in S3.
- ✓ Use HTTPS with SSL certificate from ACM.
- ✓ Use target group health checks.
- ✓ Use WAF for additional security.
- ✓ Keep listeners and rules simple and organized.

6. Listener Rule Example

Priority	Condition	Action
1	Host: app.example.com	Forward to TG-App
2	Path: /api/*	Forward to TG-API
Default	-	Forward to TG-Default

7. Common Use Cases

- ✓ Host multiple applications on same ALB (multi-domain).
- ✓ Microservices with path-based routing.
- ✓ Secure SSL termination and offloading.
- ✓ Blue/Green or Canary deployments.
- ✓ Web applications with high availability.

8. ALB with Terraform (Example)

```
provider "aws" {
   region = "ap-south-1"
}

# Security Group for ALB
resource "aws_security_group" "alb_sg" {
  name        = "alb-sg"
  description = "Allow HTTP/HTTPS to ALB"
  vpc_id      = "vpc-12345678"

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# Application Load Balancer
resource "aws_lb" "app_alb" {
 name               = "app-alb"
 internal           = false
 load_balancer_type = "application"
 security_groups    = [aws_security_group.alb_sg.id]
 subnets            = ["subnet-aaa111","subnet-bbb222"]

 enable_deletion_protection = false
}

# Target Group
resource "aws_lb_target_group" "app_tg" {
  name     = "app-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = "vpc-12345678"

  health_check {
    path                = "/health"
    matcher             = "200"
    interval            = 30
    timeout             = 5
    healthy_threshold   = 2
    unhealtty_threshold = 2
  }
}
# Listener
resource "aws_lb_listener" "http_listener" {
 load_balancer_arn = aws_lb.app_alb.arn
 port              = "80"
 protocol          = "HTTP"
 default_action {
   type             = "forward"
   target_group_arn = aws_lb_target_group.app_tg.arn
 }
}
# Attach EC2 instance to Target Group
resource "aws_lb_target_group_attachment" "app_tg_att" {
 target_group_arn = aws_lb_target_group.app_tg.arn
 target_id        = "i-0abcd1234efgh5678"  # EC2 Instance ID
 port             = 80
}
```

Note:
Replace VPC, Subnet and other IDs with your actual values.

ALB = Smart Routing + High Availability for Modern Applications ☑

Engineer_cloudops

Module 13 : AWS Resources – Target Groups

Engineer_cloudops

> Target Group (TG) is a logical group of targets (EC2 instances, IP addresses, or Lambda) that receive traffic from a Load Balancer.

1. What is Target Group ?

- Receives traffic from ALB, NLB, or GWLB.
- Routes to registered targets.
- Performs health checks to ensure only healthy targets receive traffic.
- Can be instance, IP, or Lambda targets.

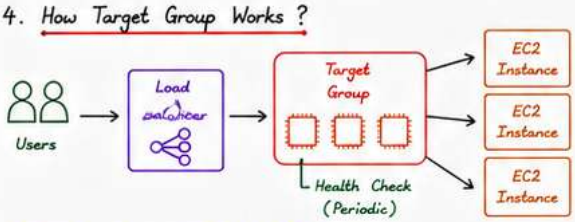
2. Key Concepts

Target : The destination (EC2, IP, or Lambda).
Health Check : Checks if target is healthy.
Port : Port on which traffic is sent to targets.
Protocol : HTTP, HTTPS, TCP, or TLS.
Target Type : instance | ip | lambda (ALB supports all, NLB supports instance & ip, GWLB supports ip)

3. Target Types

Type	Supported By	Description	Use Case
Instance	ALB, NLB	Targets are EC2 instances	Most common for EC2 apps
ip	ALB, NLB, GWLB	Targets are IP addresses	Containers, on-prem, appliances
lambda	ALB	Targets are Lambda functions	Serverless applications

4. How Target Group Works ?



1. Client sends request to Load Balancer.
2. LB forwards to Target Group.
3. TG routes traffic to healthy registered targets.
4. Health checks run periodically.
5. Unhealthy targets are removed from routing.

5. Health Check Settings (Example)

Setting	Description
Protocol	HTTP, HTTPS, TCP
Port	Traffic port or override port
Path	URL path for HTTP/HTTPS (e.g. /health)
Interval	Time between health checks (sec)
Timeout	Health check timeout (sec)
Healthy Threshold	Consecutive successes to mark healthy
Unhealthy Threshold	Consecutive failures to mark unhealthy

6. Target Group Attributes (Common)

- deregistration_delay : Time in seconds to wait before removing target (default 300s)
- slow_start : Gradually increases traffic to new targets
- stickiness : Enable session stickiness at TG level (ALB)
- cross_zone_load_balancing : Enable/disable (NLB)

7. Example Use Cases

✓ ALB + TG (instance) for web apps on EC2.
✓ NLB + TG (ip) for services on on-prem.
✓ ALB + TG (lambda) for serverless apps.
✓ Multi-target groups for different paths (e.g. /api, /static).

8. Terraform Example – Target Group (ALB)

```hcl
# Target Group
resource "aws_lb_target_group" "app_tg" {
  name        = "app-tg"
  port        = 80
  protocol    = "HTTP"
  vpc_id      = "vpc-12345678"
  target_type = "instance"

  health_check {
    protocol            = "HTTP"
    path                = "/health"
    port                = "traffic-port"
    interval            = 30
    timeout             = 5
    healthy_threshold   = 3
    unhealthy_threshold = 2
    matcher             = "200-399"
  }

  attributes = {
    deregistration_delay.timeout_seconds   = 60
    stickiness.enabled                     = true
    stickiness.type                        = "lb_cookie"
    stickiness.lb_cookie.duration_seconds  = 86400
  }
}
```

Note: target_type can be instance, ip or lambda. ALB supports all three.

9. Register Targets (Attach EC2 Instances to TG)

```hcl
resource "aws_lb_target_group_attachment" "app_tg_att" {
  count            = 2
  target_group_arn = aws_lb_target_group.app_tg.arn
  target_id        = element(["i-0abc1234", "i-0def5678"], count.index)
  port             = 80
}
```

10. Attach Target Group to ALB Listener

```hcl
resource "aws_lb_listener_rule" "app_rule" {
 listener_arn  = aws_lb_listener.http_listener.arn
 priority      = 100

 action {
   type             = "forward"
   target_group_arn = aws_lb_target_group.app_tg.arn
 }

 condition {
   path_pattern {
     values = ["/api/*"]
   }
 }
}
```

Tip: Create multiple target groups for different applications or paths.

11. Best Practices

✓ Use appropriate target type (instance/ip/lambda).
✓ Configure proper health checks.
✓ Set deregistration delay to avoid connection drops.
✓ Use sticky sessions only when needed.
✓ Monitor with CloudWatch (UnHealthyHostCount, HealthyHostCount).
✓ Use multiple AZs for high availability.

Remember: Target Group is the heart of Load Balancer. If TG is unhealthy, app will be down.

12. Useful AWS CLI Commands

```
# Create target group
aws elbv2 create-target-group --name app-tg --protocol HTTP --port 80 \
  --vpc-id vpc-12345678 --target-type instance

# Describe target groups
aws elbv2 describe-target-groups

# Register targets
aws elbv2 register-targets --target-group-arn arn:aws:elasticloadbalancing:... \
  --targets Id=i-0abc1234,Port=80 Id=i-0def5678,Port=80
```

Target Group = Smart Routing + Health Checks + High Availability ☑

Engineer_cloudops

Module 13 : AWS Resources – Auto Scaling Group (ASG)

Engineer_cloudops

Auto Scaling Group (ASG) automatically adjusts the number of EC2 instances based on demand to maintain application availability and optimize cost.

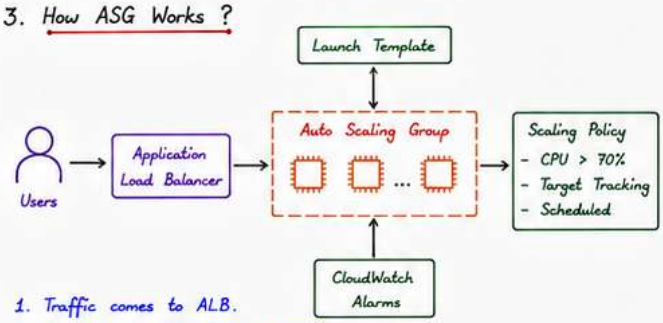
1. What is ASG ?

- ASG automatically adjusts EC2 instances in a group.
- Ensures high availability and fault tolerance.
- Works with Load Balancer.
- Supports scaling policies and health checks.

2. Key Concepts

Launch Template	:	Defines instance configuration.
Auto Scaling Group	:	Group of EC2 instances.
Min Size	:	Minimum number of instances.
Max Size	:	Maximum number of instances.
Desired Capacity	:	Number of instances to start with.
Scaling Policy	:	Rules to scale in or out.
Cooldown Period	:	Wait time after a scaling activity.
Health Check	:	Replaces unhealthy instances.

3. How ASG Works ?



1. Traffic comes to ALB.
2. ALB forwards to ASG instances.
3. ASG launches or terminates instances based on policies.
4. CloudWatch alarms trigger scaling actions.

4. Scaling Policies

Policy Type	Description
Target Tracking	Maintain a metric at target value (e.g. CPU 70%)
Step Scaling	Scale based on CloudWatch alarm thresholds
Simple Scaling	Scale in/out by a fixed number
Scheduled Scaling	Scale at specific date/time
Predictive Scaling	Use ML forecast to scale in advance

5. Health Check Types

Health Check Type	Description
EC2 Status Check	EC2 instance status (system status check)
ELB Health Check	Uses Load Balancer health check
Both	Instance must pass both checks

Best Practices

- ✓ Use Launch Template for flexibility.
- ✓ Use Target Tracking for smoother scaling.
- ✓ Configure proper cooldown period.
- ✓ Use multiple AZs for high availability.
- ✓ Enable health checks and notifications.

6. ASG Lifecycle

Lifecycle Hook	Description
Pending:Wait	Add custom actions before instance enters service
Terminating:Wait	Add custom actions before instance terminates

7. Example Use Cases

- ✓ Web applications with variable traffic.
- ✓ Background processing jobs.
- ✓ Microservices with unpredictable load.
- ✓ Cost optimization during non-peak hours.

8. Terraform Example – Auto Scaling Group

```
# Launch Template
resource "aws_launch_template" "app_lt" {
  name_prefix     = "app-lt-"
  image_id        = "ami-0abcdef1234567890"
  instance_type   = "t3.micro"

  vpc_security_group_ids = ["sg-0123456789abcdef0"]

  user_data = base64encode(<<-EOF
    #!/bin/bash
    yum install -y httpd
    systemctl enable httpd
    systemctl start httpd
    EOF
  )
}
# Auto Scaling Group
resource "aws_autoscaling_group" "app_asg" {
  name                = "app-asg"
  desired_capacity    = 2
  min_size            = 1
  max_size            = 5
  vpc_zone_identifier = ["subnet-aaa111", "subnet-bbb222"]
  health_check_type   = "ELB"
  health_check_grace_period = 300
  target_group_arns   = [aws_lb_target_group.app_tg.arn]
  launch_template {
    id      = aws_launch_template.app_lt.id
    version = "$Latest"
  }
  tag {
    key                 = "Name"
    value               = "app-instance"
    propagate_at_launch = true
  }
}
```

Note: Make sure the subnets are in different AZs.

9. Attach ASG to Target Group

```
resource "aws_lb_target_group" "app_tg" {
  name     = "app-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = "vpc-12345678"

  health_check {
    path                = "/health"
    interval            = 30
    timeout             = 5
    healthy_threshold   = 2
    unhealthy_threshold = 2
  }
}
```

Tip: ASG + ALB + Target Group is the most common and powerful combo in AWS.

10. Useful AWS CLI Commands

```
# Create Auto Scaling Group
aws autoscaling create-auto-scaling-group --auto-scaling-group-name app-asg \
 --launch-template LaunchTemplateId=lt-123456,Version='$Latest' \
 --min-size 1 --max-size 5 --desired-capacity 2 \
 --vpc-zone-identifier "subnet-aaa111,subnet-bbb222" \
 --target-group-arns arn:aws:elasticloadbalancing:region:acct:targetgroup/app-tg/xxxxxx
# Describe Auto Scaling Groups
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names app-asg
# Update ASG
aws autoscaling update-auto-scaling-group --auto-scaling-group-name app-asg \
 --min-size 2 --max-size 6 --desired-capacity 3
```

Auto Scaling = High Availability + Cost Optimization + Better Performance ☑

Engineer_cloudops

Module 13 : AWS Resources – Launch Template

Engineer_cloudops

Launch Template is a blueprint that contains the configuration information required to launch EC2 instances.
It is used by Auto Scaling Groups, EC2 Fleet, and Spot Fleet.

1. What is Launch Template ?

- It defines the instance configuration (AMI, Instance type, Key pair, Security groups, Storage, User data, etc).
- It is versioned. Every change creates a new version.
- It helps to maintain consistency and reuse.
- Used by ASG, EC2 Fleet, and Spot Fleet.

2. Key Concepts

Launch Template	:	Blueprint for launching EC2 instances.
Template ID	:	Unique ID of the launch template.
Version	:	Each update creates a new version.
Default Version	:	Version used by default if not specified.
Latest Version	:	Most recent version.

3. Components of Launch Template

Component	Description
AMI ID	Amazon Machine Image to use.
Instance Type	Type of EC2 instance.
Key Pair	Key pair for SSH access.
Security Groups	Security groups to associate.
User Data	Startup scripts (cloud-init, shell script).
Block Device Mapping	Root and additional EBS volumes.
IAM Instance Profile	IAM role for the instance.
Network Settings	Subnet, Assign Public IP, etc (optional).

4. How Launch Template Works ?



1. You create a Launch Template with instance configuration.
2. ASG or Fleet uses the template to launch instances.
3. When you update the template, a new version is created.
4. ASG can use the latest or specific version.

5. Launch Template vs Launch Configuration (Old)

	Launch Template	Launch Configuration
Versioning	Yes	No
Create Update	New version	New resource
Flexible	More options	Limited options
Recommended	Yes (Modern)	No (Legacy)

Best Practices

- ✓ Use Launch Templates with ASG for version control.
- ✓ Always test new template versions before setting default.
- ✓ Use User Data for automation.
- ✓ Use IAM Instance Profile instead of access keys.

6. Example Use Cases

- ✓ Auto Scaling Group for web applications.
- ✓ EC2 Fleet for large scale deployments.
- ✓ Spot Fleet for cost optimization workloads.

7. Terraform Example – Launch Template

```hcl
# Create Launch Template
resource "aws_launch_template" "app_lt" {
  name_prefix       = "app-lt-"
  image_id          = "ami-0abcde1234567890f"
  instance_type     = "t3.micro"
  key_name          = "my-keypair"
  vpc_security_group_ids    = ["sg-0123456789abcdef0"]

  user_data    = base64encode(<<-EOF
  #!/bin/bash
  yum update -y
  systemctl enable httpd
  systemctl start httpd
  echo "Hello from Launch Template" > /var/www/html/index.html
  EOF
)

block_device_mappings {
  device_name     = "/dev/xvda"
  ebs {
    volume_size       = 20
    volume_type       = "gp3"
    delete_on_termination  = true
  }
}

iam_instance_profile {
  name        = "ec2-instance-profile"
}

tag_specifications {
  resource_type  = "instance"
  tags
    Name          = "app-instance"
    Environment   = "dev"
  }
}
}
```

Note:
When you change any setting above, Terraform creates a new version of the Launch Template.

8. Using Launch Template in Auto Scaling Group

```hcl
resource "aws_autoscaling_group"  "app_asg" {
  name                 = "app-asg"
  desired_capacity     = 2
  max_size             = 5
  min_size             = 2
  vpc_zone_identifier  = ["subnet-aaa111", "subnet-bbb222"]

  launch_template {
    id          = aws_launch_template.app_lt.id
    version     = "$Latest"     # Or use a specific version number
  }

  target_group_arns      = [aws_lb_target_group.app_tg.arn]
  health_check_type          = "ELB"
  health_check_grace_period  = 300
}
```

9. Useful AWS CLI Commands

```
# Create Launch Template
aws ec2 create-launch-template --launch-template-name app-lt --version-description v1 \
  --launch-template-data file://launch-template.json

# Describe Launch Templates
aws ec2 describe-launch-templates  --launch-template-names  app-lt

# Create new version
aws ec2 create-launch-template-version --launch-template-name app-lt --source-version 1

# Delete Launch Template
aws ec2 delete-launch-template  --launch-template-name  app-lt
```

Launch Template = Consistency + Versioning + Automation ☑

Engineer_cloudops

Module 13 : AWS Resources – CloudWatch

Engineer_cloudops

Amazon CloudWatch is a monitoring and observability service for AWS resources and applications. It collects metrics, logs, and events, and helps you set alarms and take automated actions.

1. What is CloudWatch ?

- Monitor AWS resources, applications & services in real-time.
- Collects Metrics, Logs, and Events.
- Create Alarms and trigger actions.
- Helps in troubleshooting and performance tuning.

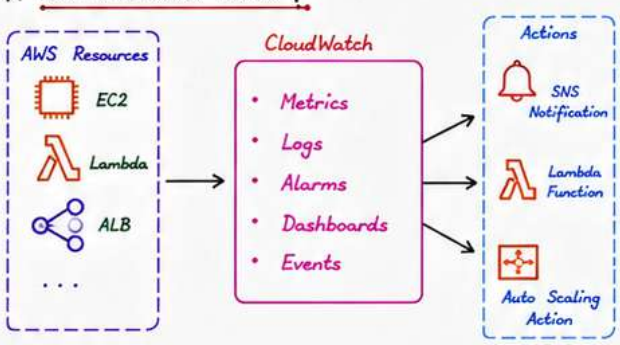
2. Key Concepts

Metrics	:	Numerical data points (CPUUtilization, NetworkIn, etc.)
Logs	:	Log data from EC2, Lambda, ALB, VPC Flow Logs, etc.
Alarms	:	Monitor metrics and perform actions when threshold is breached.
Dashboards	:	Visualize metrics in a single view.
Events	:	System events from AWS resources.
Namespaces	:	Logical container for metrics (AWS/EC2, AWS/Lambda, etc.)

3. CloudWatch Components

Component	Description
Metrics	Stores time-series data for AWS resources.
Logs	Centralized log storage and analysis.
Alarms	Set thresholds and get notified or trigger actions.
Dashboards	Custom dashboards to visualize metrics.
Events	Real-time events from AWS services.
Insights	Analyze logs using CloudWatch Logs Insights.

4. How CloudWatch Works ?



5. Common CloudWatch Metrics (EC2)

Metric Name	Description
CPUUtilization	CPU utilization percentage
NetworkIn	Bytes received
NetworkOut	Bytes sent
StatusCheckFailed	Instance status check failed
DiskReadOps	Number of disk read operations
DiskWriteOps	Number of disk write operations

Best Practices

- ✓ Use meaningful alarm thresholds.
- ✓ Enable detailed monitoring for critical resources.
- ✓ Use CloudWatch Logs retention to control cost.
- ✓ Create dashboards for quick visibility.
- ✓ Use SNS for instant notifications.

6. CloudWatch Alarm Example

Alarm Name	Metric	Condition	Action
HighCPUAlarm	CPUUtilization	> 80% for 5 min	SNS Notification

7. Terraform Example – CloudWatch (Complete)

```
# 1. Create CloudWatch Log Group
resource "aws_cloudwatch_log_group" "app_log_group" {
  name              = "/aws/ec2/my-app"
  retention_in_days = 14
  tags  {
    Environment     = "dev"
    Project         = "cloudwatch-demo"
  }
}

# 2. Create CloudWatch Alarm for EC2 CPU Utilization
resource "aws_cloudwatch_metric_alarm" "cpu_alarm" {
  alarm_name                = "HighCPUAlarm"
  comparison_operator       = "GreaterThanThreshold"
  evaluation_periods        = 2
  metric_name               = "CPUUtilization"
  namespace                 = "AWS/EC2"
  period                    = 300        # 5 minutes
  statistic                 = "Average"
  threshold                 = 80
  alarm_description         = "This metric monitors EC2 CPU utilization"
  dimensions = {
    InstanceId    = "i-1234567890abcdef0"   # Replace with your EC2 ID
  }
  alarm_actions  = [aws_sns_topic.cpu_sns.arn]
  ok_actions     = [aws_sns_topic.cpu_sns.arn]
}

# 3. Create SNS Topic for Alarm Notification
resource "aws_sns_topic" "cpu_sns" {
  name       = "cpu-alarm-topic"
}

# 4. Subscribe Email to SNS Topic
resource "aws_sns_topic_subscription" "email_sub" {
  topic_arn    = aws_sns_topic.cpu_sns.arn
  protocol     = "email"
  endpoint     = "your-email@example.com"
}

# 5. (Optional) CloudWatch Dashboard
resource "aws_cloudwatch_dashboard" "app_dashboard" {
  dashboard_name   = "App-Dashboard"
  dashboard_body   = jsonencode({
    widgets = [
      {
        type     = "metric"
        x        = 0
        y        = 0
        width    = 12
        height   = 6
        properties = {
          metrics = [
            ["AWS/EC2", "CPUUtilization", "InstanceId", "i-1234567890abcdef0"]
          ]
          period   = 300
          stat     = "Average"
          region   = "us-east-1"
          title    = "EC2 CPU Utilization"
        }
      }
    ]
  })
}
```

Note:

- Replace EC2 Instance ID with your actual instance.
- Replace email with your email address.

8. Useful AWS CLI Commands

```
# List Metrics
aws cloudwatch list-metrics --namespace AWS/EC2

# Describe Alarms
aws cloudwatch describe-alarms --alarm-names HighCPUAlarm

# Create Alarm (CLI Example)
aws cloudwatch put-metric-alarm --alarm-name HighCPUAlarm \
  --alarm-description "High CPU" --metric-name CPUUtilization \
  --namespace AWS/EC2 --statistic Average --period 300 \
  --threshold 80 --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2 --alarm-actions arn:aws:sns:us-east-1:123456789012:cpu-alarm-topic
```

CloudWatch = Visibility + Alerts + Automation ☑

Engineer_cloudops

Module 13 : AWS Resources - Lambda

Engineer_cloudops

AWS Lambda is a serverless compute service that lets you run code without provisioning or managing servers. You pay only for the time your code runs.

1. What is Lambda ?

- Serverless compute service.
- Runs code in response to events.
- Supports many runtimes (Python, Node.js, Java, Go, .NET, etc).
- Automatically scales and highly available.
- Pay-per-request + duration (ms).

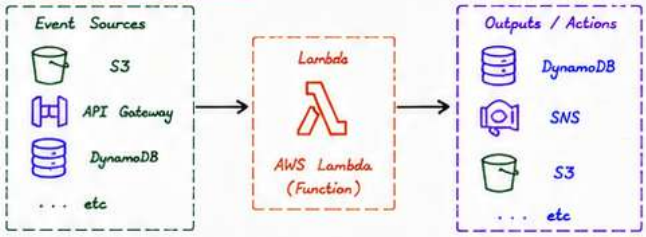
2. Key Concepts

Function	:	The Lambda code.
Handler	:	The entry point (e.g, index.handler).
Runtime	:	The language your code is written in.
Role	:	IAM role that Lambda assumes.
Trigger	:	The event source (S3, API Gateway, SNS, etc).
Memory & Timeout	:	Controls performance and max execution time.
Environment Variables	:	Key-value configuration for your code.

3. Common Use Cases

- ✓ Image resize on S3 upload.
- ✓ Process data from DynamoDB streams.
- ✓ Send emails on user sign-up.
- ✓ Scheduled tasks (cron jobs).
- ✓ API backend with API Gateway.

4. How Lambda Works ?



5. Lambda Components

Component	Description
Function	Your code packaged as a ZIP file or container image.
Trigger / Event Source	Service that invokes the function.
Execution Role	IAM role with permissions for AWS resources.
Memory / Timeout	Allocate memory (128MB - 10GB) and max runtime (1 - 15 min).
Environment Variables	Config values available to your code.
Layers (Optional)	Share code and dependencies across functions.

Best Practices

- ✓ Keep functions small and focused.
- ✓ Use meaningful names and tags.
- ✓ Set correct timeout and memory.
- ✓ Use environment variables, not hardcoded values.
- ✓ Enable X-Ray for tracing.
- ✓ Monitor with CloudWatch Logs & Alarms.

8. Useful AWS CLI Commands

```
# List Lambda functions
aws lambda list-functions

# Get function configuration
aws lambda get-function --function-name example-lambda-fn

# Invoke function
aws lambda invoke --function-name example-lambda-fn output.json

# View logs in CloudWatch
aws logs tail /aws/lambda/example-lambda-fn --follow
```

6. Lambda Example Code (Python)

```python
import json
import boto3

def lambda_handler(event, context):
    print("Event:", json.dumps(event))
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }
```

7. Terraform Example - Lambda Function with S3 Trigger

```
# 1. IAM Role for Lambda
resource "aws_iam_role" "lambda_role" {
  name =      "lambda-execution-role"

  assume_role_policy = jsonencode({
     Version          = "2012-10-17"
     Statement        = [{
        Action          =    = "sts:AssumeRole"
        Effect          =    = "Allow"
        Principal       =    { Service = "lambda.amazonaws.com" }
     }]
  })
}

# 2. Attach AWS managed policy for basic execution
resource "aws_iam_role_policy_attachment" "lambda_basic" {
  role       = aws_iam_role.lambda_role.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
}

# 3. Lambda Function
resource "aws_lambda_function" "example" {
  function_name    = "example-lambda-fn"
  role             = aws_iam_role.lambda_role.arn
  handler          = "index.lambda_handler"
  runtime          = "python3.11"

  filename         = "lambda_function.zip"
  source_code_hash = filebase64sha256("lambda_function.zip")

  timeout      = 30
  memory_size  = 256

  environment {
     variables = {
        ENV         = "dev"
     }
  }

  tags = {
     Project       = "CloudOps"
     ManagedBy     = "Terraform"
  }
}

# 4. S3 Bucket (for trigger)
resource "aws_s3_bucket" "lambda_bucket" {
  bucket     = "my-lambda-trigger-bucket-12345"
}

# 5. Permission for S3 to invoke Lambda
resource "aws_lambda_permission" "allow_s3" {
  statement_id     = "AllowExecutionFromS3"
  action           = "lambda:InvokeFunction"
  function_name    = aws_lambda_function.example.function_name
  principal        = "s3.amazonaws.com"
  source_arn       = aws_s3_bucket.lambda_bucket.arn
}

# 6. S3 Bucket Notification to trigger Lambda
resource "aws_s3_bucket_notification" "s3_notify" {
  bucket   = aws_s3_bucket.lambda_bucket.id

  lambda_function {
    lambda_function_arn   = aws_lambda_function.example.arn
    events                = ["s3:ObjectCreated:*"]
    filter_prefix         = "uploads/"
  }
}
```

Note: Upload any file to S3 bucket prefix "uploads/" -> Lambda will be triggered.

Lambda = Serverless + Scalable + Cost Effective ☑

Engineer_cloudops

Module 13 : AWS Resources – RDS (Relational Database Service)

Engineer_cloudops

Amazon RDS is a managed relational database service that makes it easy to set up, operate, and scale a relational database in the cloud.

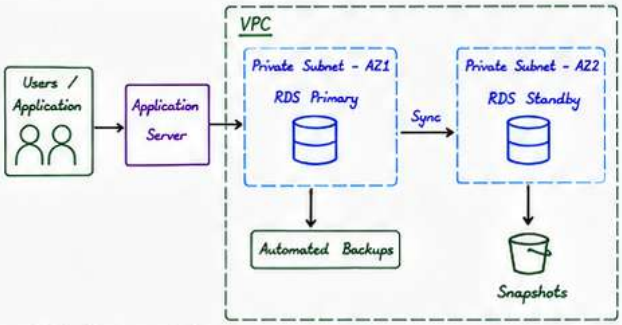
1. What is RDS ?

- Fully managed database service by AWS.
- Supports MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Aurora MySQL, Aurora PostgreSQL.
- Handles backups, patching, monitoring, failover, and scaling.
- You just focus on your data and applications.

2. Key Concepts

DB Instance	:	The actual database (e.g., MySQL instance).
Engine	:	Type of DB engine (mysql, postgres, etc).
Instance Class	:	Defines CPU, Memory, Network capacity.
Storage	:	Allocated storage (gp3/io1/io2).
Multi-AZ	:	Standby in another AZ for high availability.
Backup	:	Automated backups + snapshots.
Security Group	:	Controls access to the DB.
Subnet Group	:	Subnets where RDS can be placed.
Parameter Group	:	DB engine specific settings.

3. RDS Architecture (High Level)



4. RDS Storage Options

Storage Type	Use Case
gp3 (General Purpose SSD)	Most workloads (balanced price & performance)
io1 /io2 (Provisioned IOPS)	High IOPS critical workloads
Standard	Legacy workloads (not recommended now)

5. RDS Features

- ✓ Automated Backups
- ✓ Point-in-time Recovery
- ✓ Multi-AZ Deployment
- ✓ Read Replicas
- ✓ Performance Insights
- ✓ Encryption (at rest & in transit)
- ✓ Automatic Minor Version Upgrade
- ✓ Monitoring with CloudWatch
- ✓ Parameter Groups
- ✓ Snapshot & Restore
- ✓ IAM Database Authentication (MySQL/PostgreSQL)

Best Practices

- ✓ Use Multi-AZ for production databases.
- ✓ Enable automated backups & test restore.
- ✓ Use least privilege security groups.
- ✓ Store credentials in Secrets Manager.
- ✓ Monitor CPU, Connections, Disk using CloudWatch.
- ✓ Use Read Replicas for read-heavy workloads.

6. Terraform Example - RDS MySQL Instance

```hcl
# 1. DB Subnet Group
resource "aws_db_subnet_group" "rds_subnet_group" {
  name       = "rds-subnet-group"
  subnet_ids = [
    "subnet-0a1b2c3d4e5f6a7b1",
    "subnet-0a1b2c3d4e5f6a7b2"
  ]
  description = "Subnet group for RDS"
}

# 2. DB Parameter Group
resource "aws_db_parameter_group" "rds_pg" {
  name        = "mysql-8-parameter-group"
  family      = "mysql8.0"
  description = "Custom parameter group for MySQL 8"

  parameter {
    name  = "max_connections"
    value = "500"
  }
}

# 3. RDS Instance
resource "aws_db_instance" "mysql" {
  identifier             = "myapp-mysql-db"
  engine                 = "mysql"
  engine_version         = "8.0.33"
  instance_class         = "db.t3.medium"
  allocated_storage      = 20
  storage_type           = "gp3"
  username               = "admin"
  password               = "StrongPass@123"
                           Manager in real
  db_name                = "myappdb"
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  db_subnet_group_name   = aws_db_subnet_group.rds_subnet_group.name
  parameter_group_name   = aws_db_parameter_group.rds_pg.name
  backup_retention_period = 7
  backup_window          = "02:00-03:00"
  maintenance_window     = "sun:03:00-sun:04:00"
  multi_az               = true
  publicly_accessible    = false
  storage_encrypted      = true
  deletion_protection    = true
  skip_final_snapshot    = false
  final_snapshot_identifier  = "myapp-mysql-final-snapshot"
  tags = {
    Name        = "myapp-mysql-db"
    Environment = "dev"
    ManagedBy   = "terraform"
  }
}
# 4. Security Group for RDS
resource "aws_security_group" "rds_sg" {
  name        = "rds-sg"
  description = "Allow MySQL access from app server"
  vpc_id      = "vpc-0123456789abcdef0"
  ingress {
     from_port     = 3306
     to_port       = 3306
     protocol      = "tcp"
     security_groups = [aws_security_group.app_sg.id]  # Allow only from app SG
  }
  egress {
     from_port     = 0
     to_port       = 0
     protocol      = "-1"
     cidr_blocks   = ["0.0.0.0/0"]
  }
}
```

Note:

- Use Secrets Manager for storing DB password in real environments.
- Change instance class & storage as per workload.

7. Useful AWS CLI Commands

```
# List all RDS instances
aws rds describe-db-instances --region ap-south-1
# Create manual snapshot
aws rds create-db-snapshot --db-instance-identifier myapp-mysql-db \
  --db-snapshot-identifier myapp-mysql-snap-01
# Restore from snapshot
aws rds restore-db-instance-from-db-snapshot --db-instance-identifier myapp-mysql-db-restore \
  --db-snapshot-identifier myapp-mysql-snap-01
# Delete snapshot
aws rds delete-db-snapshot --db-snapshot-identifier myapp-mysql-snap-01
```

RDS = Managed + Available + Secure + Scalable ☑

Engineer_cloudops

Module 13 : AWS Resources – DynamoDB

Engineer_cloudops

Amazon DynamoDB is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability.

1. What is DynamoDB ?

- Fully managed NoSQL (Key-Value & Document) database.
- Single-digit millisecond latency at any scale.
- Seamlessly scales up and down.
- Built-in security, backup, and multi-region replication.

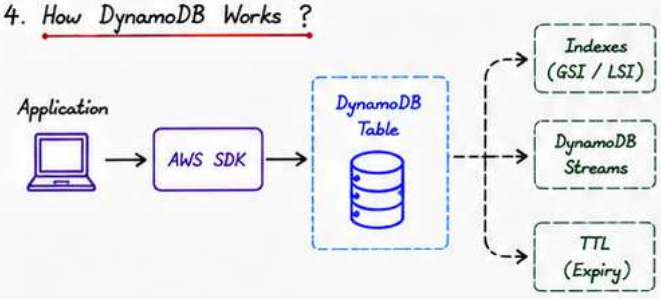
2. Key Concepts

Table	:	Container for your data.
Partition Key	:	Primary key (hash key) – must be unique.
Sort Key	:	Optional key (range key) – for sorting & queries.
Item	:	A row/record in a table.
Attribute	:	A field in an item.
Throughput	:	Read/Write capacity (RCU/WCU).
Index	:	Secondary Indexes for alternate queries.
TTL	:	Automatically delete expired items.
DAX	:	In-memory cache for microsecond performance.

3. DynamoDB Table Structure

Component	Description
Table Name	Unique name for the table.
Primary Key	Partition Key (and optional Sort Key).
Capacity Mode	Provisioned (RCU/WCU) or On-Demand.
Indexes	GSIs/LSIs for alternate access patterns.
TTL	Auto delete items after expiry time.
Streams	Capture changes to table items.

4. How DynamoDB Works ?



5. Use Cases

- ✓ User sessions, Shopping carts
- ✓ IoT device data
- ✓ Real-time analytics
- ✓ Leaderboards, Counters
- ✓ Config & feature flags

Best Practices

- ✓ Choose right partition key.
- ✓ Use On-Demand for unpredictable workloads.
- ✓ Enable TTL to reduce storage cost.
- ✓ Use GSIs wisely.
- ✓ Monitor with CloudWatch metrics.
- ✓ Enable Point-in-Time Recovery.

6. Terraform Example – DynamoDB Table

```hcl
# 1. DynamoDB Table
resource "aws_dynamodb_table" "user_sessions" {
  name           = "user-sessions"
  billing_mode   = "PAY_PER_REQUEST"    # On-Demand
  hash_key       = "userId"
  range_key      = "sessionId"

  attribute {
    name = "userId"
    type = "S"
  }

  attribute {
    name = "sessionId"
    type = "S"
  }

  ttl {
    attribute_name = "ttl"
    enabled        = true
  }

  point_in_time_recovery {
    enabled = true
  }

  tags = {
    Name        = "user-sessions-table"
    Environment = "dev"
    ManagedBy   = "terraform"
  }
}

# 2. Global Secondary Index (GSI)
resource "aws_dynamodb_table" "orders" {
  name         = "orders"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "orderId"
  attribute {
    name = "orderId"
    type = "S"
  }
  attribute {
    name = "customerId"
    type = "S"
  }
  global_secondary_index {
    name            = "gsi_customer"
    hash_key        = "customerId"
    projection_type = "ALL"       # or KEYS_ONLY / INCLUDE
  }
  tags = {
    Name        = "orders-table"
    Environment = "dev"
  }
}
```

Note:

- Change billing_mode to "PROVISIONED" and set read/write capacity if needed.
- Attribute "ttl" should be epoch time (seconds).

7. Useful AWS CLI Commands

```bash
# Create table
aws dynamodb create-table \
  --table-name user-sessions \
  --attribute-definitions AttributeName=userId,AttributeType=S \
                          AttributeName=sessionId,AttributeType=S \
  --key-schema AttributeName=userId,KeyType=HASH \
               AttributeName=sessionId,KeyType=RANGE \
  --billing-mode PAY_PER_REQUEST

# Describe table
aws dynamodb describe-table --table-name user-sessions
# Scan table
aws dynamodb scan --table-name user-sessions
# Query example
aws dynamodb query --table-name user-sessions \
   --key-condition-expression "userId = :u" \
   --expression-attribute-values '{":u":{"S":"user-123"}}'
# Delete table
aws dynamodb delete-table --table-name user-sessions
```

DynamoDB = Fast + Serverless + Highly Scalable ☑

Engineer_cloudops

Module 13 : AWS Resources – EKS (Elastic Kubernetes Service)

Engineer_cloudops

Amazon EKS is a fully managed Kubernetes service that makes it easy to run Kubernetes on AWS without installing or operating the control plane.

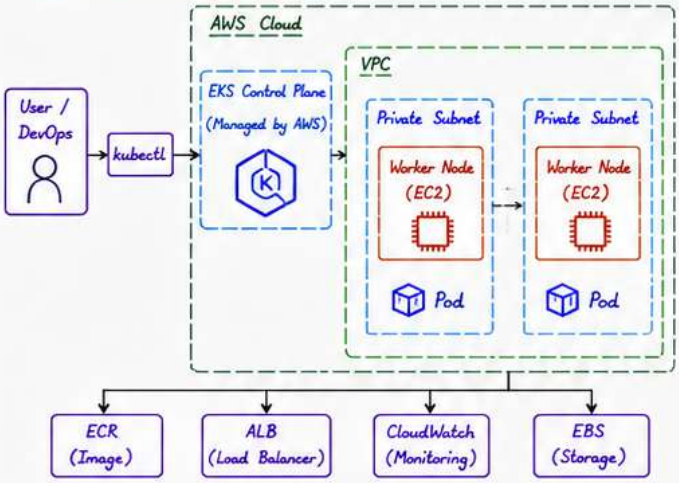
1. What is EKS ?

- Managed Kubernetes service by AWS.
- You manage worker nodes (EC2) or use Fargate.
- Highly available, secure & scalable.
- Integrates with IAM, VPC, Security Groups, CloudWatch, EBS, ECR, ALB, etc.

2. Key Concepts

Cluster	:	The EKS control plane.
Node Group	:	EC2 instances (worker nodes) that run your pods.
Fargate	:	Serverless compute for running pods.
VPC & Subnets	:	Networking for the cluster and nodes.
IAM Role	:	Permissions for EKS control plane & nodes.
Security Group	:	Control traffic to the cluster & nodes.
Add-ons	:	CoreDNS, kube-proxy, VPC CNI, etc.
kubectl	:	CLI tool to interact with the cluster.

3. EKS Architecture (High Level)



4. EKS Components

Component	Description
Control Plane	AWS manages the control plane (API server, etcd, scheduler).
Worker Node	EC2 instances where your containers (pods) run.
Fargate	Serverless compute for pods (no node management).
IAM Roles	Permissions for control plane & worker nodes.
Add-ons	Essential components like CoreDNS, VPC CNI, kube-proxy.
Networking	VPC, Subnets, Security Groups for cluster communication.

5. Best Practices

- ✓ Use private subnets for worker nodes.
- ✓ Enable IAM Roles for Service Accounts (IRSA).
- ✓ Keep Kubernetes & add-ons updated.
- ✓ Use Auto Scaling for worker nodes.
- ✓ Enable Control Plane logging to CloudWatch.
- ✓ Use least privilege IAM policies.

6. Terraform Example – EKS Cluster with Node Group

```hcl
# 1. EKS Cluster Role
resource "aws_iam_role" "eks_cluster_role" {
  name        = "eks-cluster-role"
  assume_role_policy = jsonencode({
    Version   = "2012-10-17",
    Statement = [{
      Action    = "sts:AssumeRole",
      Effect    = "Allow",
      Principal = { Service = "eks.amazonaws.com" }
    }]
  })
}

# Attach required policy
resource "aws_iam_role_policy_attachment" "eks_cluster_policy" {
  role       = aws_iam_role.eks_cluster_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}

# 2. EKS Cluster
resource "aws_eks_cluster" "eks" {
  name     = "demo-eks-cluster"
  role_arn = aws_iam_role.eks_cluster_role.arn
  version  = "1.29"

  vpc_config {
    subnet_ids              = var.private_subnet_ids
    endpoint_private_access = true
    endpoint_public_access  = true
  }
  depends_on = [aws_iam_role_policy_attachment.eks_cluster_policy]
}

# 3. Node Group Role
resource "aws_iam_role" "eks_node_role" {
  name        = "eks-node-role"
  assume_role_policy = jsonencode({
    Version   = "2012-10-17",
    Statement = [{
      Action    = "sts:AssumeRole",
      Effect    = "Allow",
      Principal = { Service = "ec2.amazonaws.com" }
    }]
  })
}
# Attach policies for worker nodes
resource "aws_iam_role_policy_attachment" "eks_node_policy_1" {
  role       = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
}
resource "aws_iam_role_policy_attachment" "eks_node_policy_2" {
  role       = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}
resource "aws_iam_role_policy_attachment" "eks_node_policy_3" {
  role       = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
}

# 4. EKS Node Group
resource "aws_eks_node_group" "node_group" {
  cluster_name    = aws_eks_cluster.eks.name
  node_group_name = "demo-node-group"
  node_role_arn   = aws_iam_role.eks_node_role.arn
  subnet_ids      = var.private_subnet_ids
  instance_types  = ["t3.medium", "t3.small"]
  scaling_config {
    desired_size = 2
    min_size     = 1
    max_size     = 4
  }
  depends_on = [aws_iam_role_policy_attachment.eks_node_policy_1,
                aws_iam_role_policy_attachment.eks_node_policy_2,
                aws_iam_role_policy_attachment.eks_node_policy_3]
}
```

7. Useful AWS CLI Commands

```bash
# List EKS Clusters
aws eks list-clusters
# Describe Cluster
aws eks describe-cluster --name demo-eks-cluster
# Update kubeconfig
aws eks update-kubeconfig --name demo-eks-cluster --region ap-south-1
# List Node Groups
aws eks list-nodegroups --cluster-name demo-eks-cluster
# Describe Node Group
aws eks describe-nodegroup --cluster-name demo-eks-cluster --nodegroup-name demo-node-group
```

EKS = Managed + Secure + Scalable + Highly Available ☑

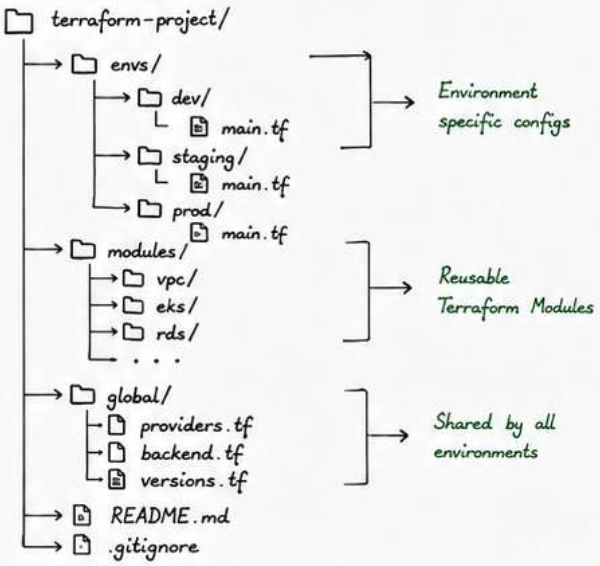
Engineer_cloudops

Module 14: Best Practices

Engineer_cloudops

Good practices in Terraform save time, reduce errors and make infra easy to maintain & scale.

1. FOLDER STRUCTURE



Why this structure?

- ✓ Separation of environments (dev/staging/prod).
- ✓ Reusable modules → DRY principle.
- ✓ Common configs (providers, backend) in one place.
- ✓ Easy to manage & collaborato in teams.

2. NAMING CONVENTION

Resource Type	Naming Convention (Example)
S3 Bucket	-<project>-<env>-<purpose>-s3 myapp-dev-logs-s3
DynamoDB Table	-<project>-<env>-<purpose>--ddb myapp-prod-cart-ddb
EC2 Instance	-<project>-<env>-<role>--ec2 myapp-stg-web-ec2
RDS Instance	-<project>-<env>-<db>--rds myapp-prod-app-rds
EKS Cluster	-<project>-<env>--eks myapp-prod-eks
VPC	-<project>-<env>-<a2>--subnet myapp-dev-vpc
Subnets	-<myapp-dev-a-subnet

Tips:

- Use lowercase letters, numbers & hyphens.
- Keep it short but meaningful.
- Always include environment (dev/stg/prod).
- Be consistent across all resources.

3. VARIABLE FILES

- Use variables for all dynamic values.
- Keep common variables in variables.tf.
- Use separate .tfvars files for each environment.
- Never hardcode values.

Example Structure

```
variables.tf        # variable definitions
dev.tfvars          # values for dev
staging.tfvars      # values for staging
prod.tfvars         # values for prod
terraform.tfvars    # (optional) common values
```

Example 1: variables.tf

```
variable "aws_region" {
  description = "AWS region"
  type        = string
}

variable "environment" {
  description = "Environment name"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.medium"
}
```

No hardcoding
Use variables
Default where possible

Example 2: dev.tfvars

```
aws_region    = "ap-south-1"
environment   = "dev"
instance_type = "t3.small"
```

Example 3: Using in main.tf

```
provider "aws" {
  region = var.aws_region
}

module "vpc" {
  source      = "../../modules/vpc"
  environment = var.environment
}

resource "aws_instance" "app" {
  ami           = "ami-0abcd1234abcd1234"
  instance_type = var.instance_type
  tags = {

    Name = "${var.environment}-app-ec2"
  }
}
```

All values from variables

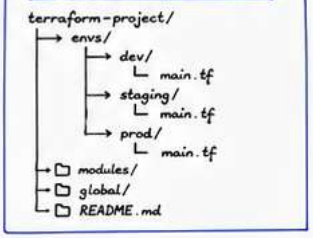
4. OTHER BEST PRACTICES

- ✓ Use Remote Backend (S3 + DynamoDB) for state locking.
- ✓ Enable Versioning on S3 bucket.
- ✓ Use Workspaces for multiple environments (if needed).
- ✓ Follow least privilage IAM policies.
- ✓ Use terraform fmt, validate, plan before apply.
- ✓ Add meaningful descriptions & tags to all resources.
- ✓ Keep modules small, focused & reusable.

5. EXAMPLE: ENVIRONMENT USAGE

1. Initialize backend (one time)
   `terraform init`
2. Select workspace (optional)
   `terraform workspace select dev`
3. Plan using tfvars
   `terraform plan -var-file="dev.tfvars"`
4. Apply
   `terraform apply -var-file="dev.tfvars"`

Example tree again (short)



Follow these practices → Clean Code → Easy Maintenance → Happy DevOps ☑

Engineer_cloudops

Module 14: Best Practices - Secrets Management

Engineer_cloudops

Secrets Management is the practice of securely storing, rotating and controlling access to sensitive information like passwords, API keys, DB creds, etc.

1. WHY SECRETS MANAGEMENT ?

- Don't hardcode secrets in code or config.
- Centralized, encrypted and access controlled.
- Automatic rotation reduces risk.
- Audit trail and integration with AWS services.

2. AWS SECRETS MANAGER - KEY FEATURES

- Store secrets (up to 64KB).
- Automatic rotation with Lambda.
- Fine-grained IAM access control.
- Seamless integration with AWS services.
- Secrets are encrypted using KMS.

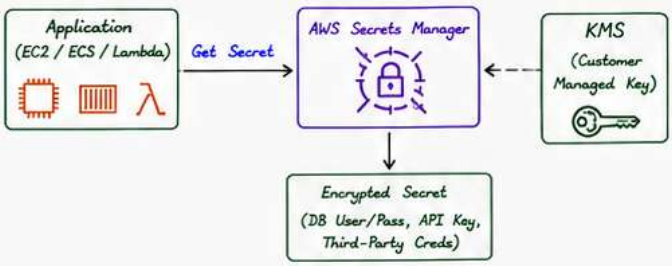
3. BEST PRACTICES

- ✓ Never commit secrets to Git.
- ✓ Use IAM roles - NOT access keys, when possible.
- ✓ Enable automatic rotation for supported services.
- ✓ Use resource-based policy for cross-account access (if required).
- ✓ Use KMS CMK for encryption (customer managed key).
- ✓ Enable CloudTrail & CloudWatch monitoring.
- ✓ Use least privilege policies.

Remember:

Secrets Manager > SSM Parameter Store (SecureString) when you need rotation, auditing & fine-grained access.

4. ARCHITECTURE OVERVIEW



5. SECRETS MANAGER USE CASES

- ✓ RDS database credentials
- ✓ Third party API keys
- ✓ Application secrets (JWT, OAuth, etc.)
- ✓ SSH private keys
- ✓ Any sensitive configuration

Quick Tips

- ✓ Rotate secrets regularly.
- ✓ Don't use root user credentials.
- ✓ Monitor access using CloudTrail.
- ✓ Use IAM roles for applications.
- ✓ Tag your secrets for ownership & environment.

6. TERRAFORM EXAMPLE - SECRETS MANAGER

```hcl
# 1. KMS Key for Secrets Manager (Optional but Recommended)
resource "aws_kms_key" "secrets_kms" {
    description             = "KMS key for Secrets Manager"
    enable_key_rotation     = true
}

resource "aws_kms_alias" "secrets_kms_alias" {
    name            = "alias/secrets-manager-key"
    target_key_id   = aws_kms_key.secrets_kms.key_id
}

# 2. Secrets Manager Secret
resource "aws_secretsmanager_secret" "db_secret" {
    name          = "prod/rds/db-credentials"
    description   = "RDS database credentials for production"
    kms_key_id    = aws_kms_key.secrets_kms.key_id

    tags = {
        Environment = "prod"
        ManagedBy   = "terraform"
    }
}

# 3. Secret Version (Initial Secret Value)
resource "aws_secretsmanager_secret_version" "db_secret_version" {
    secret_id       = aws_secretsmanager_secret.db_secret.id
    secret_string   = jsonencode({
        username    = "dbadmin"
        password    = "Str0ngP@ssw0rd!"
        host        = "mydb.cluster-xyz.ap-south-1.rds.amazonaws.com"
        port        = "5432"
        dbname      = "myapp"
    })
}

# 4. Resource Policy (Allow specific IAM role to access secret)
resource "aws_secretsmanager_secret_policy" "db_secret_policy" {
    secret_arn = aws_secretsmanager_secret.db_secret.arn
    policy = jsonencode({
        "Version": "2012-10-17",
        "Statement": [{
            "Sid"": "AllowAppRole",
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::123456789012:role/my-application-role"
            },
            "Action": [ "secretsmanager:GetSecretValue" ],
            "Resource": "${aws_secretsmanager_secret.db_secret.arn}"
        }]
    })
}
```

7. CONSUMING THE SECRET (EXAMPLE - EC2 IAM ROLE)

- Attach IAM policy to EC2 role:

```
{
    "Version": "2012-10-17",
    "Statement": [{
        "Effect": "Allow",
        "Action": [ "secretsmanager:GetSecretValue" ],
        "Resource": "arn:aws:secretsmanager:ap-south-1:123456789012:secret:
                    prod/rds/db-credentials-*"
    }]
}
```

- In EC2 (using AWS CLI):

```
aws secretsmanager get-secret-value \
   --secret-id prod/rds/db-credentials \
   --region ap-south-1 --query SecretString --output text | jq
```

- In Application (SDK Example - Python boto3):

```
import boto3, json
client = boto3.client('secretsmanager', region_name='ap-south-1')
response = client.get_secret_value(SecretId='prod/rds/db-credentials' )
secret = json.loads(response['SecretString'])
print(secret['username'], secret['password'])
```

Secure your secrets, Secure your infra ! ☑

Engineer_cloudops

Module 14: Best Practices – Remote State Security, Module Design, Version Pinning

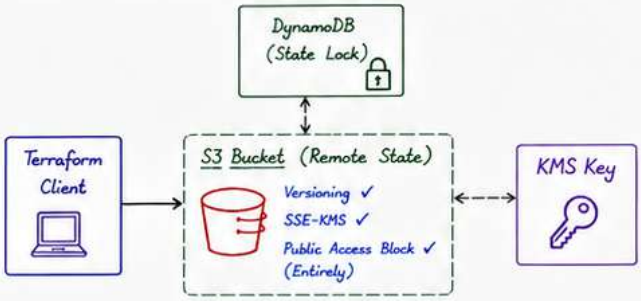
Engineer_cloudops

1. REMOTE STATE SECURITY

Terraform state file contains all the infrastructure information. If someone gets access to state file, they can see & even modify your infra.

Best Practices

- ✓ Store state in S3 with encryption (SSE-KMS).
- ✓ Enable versioning on S3 bucket.
- ✓ Block public access to the bucket.
- ✓ Use DynamoDB for state locking.
- ✓ Use least privilege IAM role.
- ✓ Enable MFA delete (optional).
- ✓ Enable CloudTrail & S3 access logs.



Example: backend.tf

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "envs/dev/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-lock-table"
    encrypt        = true
    kms_key_id     = "arn:aws:kms:ap-south-1:123456789012:key/abcd-1234"
  }
}
```

Example: S3 Bucket (HCL)

```
resource "aws_s3_bucket" "tf_state" {
  bucket = "my-terraform-state-bucket"
}

resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.tf_state.id
  versioning_configuration {
    status = "Enabled"
  }
}
resource "aws_s3_bucket_server_side_encryption_configuration" "enc" {
  bucket = aws_s3_bucket.tf_state.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id= "arn:aws:kms:ap-south-1:123456789012:key/abcd-1234"
  }
}
```

Note:

- Never commit terraform.tfstate to Git.
- If state is compromised, attacker can modify/destroy resources.
- Enable access logs to track who is accessing the state.

2. MODULE DESIGN

Good module design = Reusability + Readability + Maintainability
Keep modules small, focused & configurable.

Best Practices

- ✓ One module = One responsibility.
- ✓ Use meaningful input variables.
- ✓ Use outputs for required values only.
- ✓ Use locals for repeated values.
- ✓ Provide sensible defaults.
- ✓ Document inputs, outputs & usage.
- ✓ Keep modules DRY and environment agnostic.

Example Module Structure

```
modules/
└── vpc/
    ├── main.tf
    ├── variables.tf
    ├── outputs.tf
    ├── locals.tf
    ├── versions.tf
    └── README.md
└── examples/
    └── simple/
        ├── main.tf
        └── terraform.tfvars
```

Example: modules/vpc/variables.tf

```
variable "cidr_block" {
    description  = "VPC CIDR Block"
    type         = string
}

variable "azs" {
    description  = "List of AZs"
    type         = list(string)
}

variable "enable_nat" {
    description  = "Enable NAT Gateway"
    type         = bool
    default      = true
}
```

Example: modules/vpc/main.tf (Simplified)

```
resource "aws_vpc" "this" {
   cidr_block             = var.cidr_block
   enable_dns_hostnames   = true
   tags = {
      Name  = "${var.name}-vpc"
}
# ... subnets,  igw, route tables etc.
```

Example: modules/vpc/outputs.tf

```
output "vpc_id"   {
  value  = aws_vpc.this.id
}
output "public_subnet_ids" {
  value  = aws_subnet.public[*].id
}
```

3. VERSION PINNING

Pin provider & module versions to avoid unexpected changes and ensure consistent deployments.

Best Practices

- ✓ Always pin provider versions.
- ✓ Pin module versions (use specific version, not "latest").
- ✓ Update versions after testing in lower environments.

Provider Version Pinning (versions.tf)

```
terraform {
  required_version   = ">= 1.3.0, < 2.0.0"
  required_providers {
    aws = {
      source   = "hashicorp/aws"
      version  = "~> 5.48" # Locks to 5.x, latest 5.x
  }
}
```

Module Version Pinning (root main.tf)

```
module "vpc" {
  source   =  "git::https://github.com/org/terraform-aws-vpc.git"
  version  =  "1.2.3"    # specific version

  cidr_block = "10.0.0.0/16"
  azs        = ["ap-south-1a", "ap-south-1b"]
  name       = "myapp"
}
```

Note: Use "~>" operator for patch/minor updates (recommended).

- Avoid using "latest".
- Always read CHANGELOG before upgrading.

Secure State → Reusable Modules → Stable Deployments

Engineer_cloudops

Module 14 : Best Practices – Git Integration, CI/CD Integration, Terraform Documentation

Engineer_cloudops

1. GIT INTEGRATION

Store all terraform code in Git repository for version control, collaboration and safe rollbacks.

Best Practices:

- ✓ Use a dedicated repo for Infrastructure as Code.
- ✓ Follow a clean branch strategy (main / dev / feature/*).
- ✓ Keep state file out of Git (use remote backend).
- ✓ Use .gitignore to exclude .terraform/ and *.tfstate* files.
- ✓ Review changes through Pull Requests.
- ✓ Tag releases for production deployments.

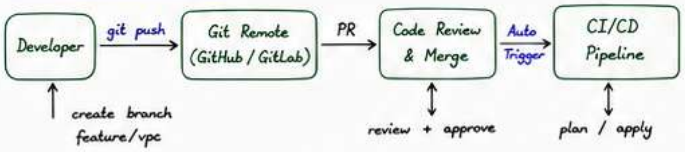
Example .gitignore

```
.terraform/
*.tfstate
*.tfstate.*
crash.log
override.tf
override.tf.json
.terraform.lock.hcl
```

Example Repo Structure

```
terraform-infra/
├── envs/
│   ├── dev/
│   │   └── main.tf
│   ├── staging/
│   │   └── main.tf
│   └── prod/
│       └── main.tf
├── modules/
│   ├── vpc/
│   └── rds/
├── README.md
└── .gitignore
```

Example: Git Workflow



2. CI/CD INTEGRATION

Automate terraform workflow (init, plan, apply) using CI/CD pipeline for consistency and speed.

Best Practices:

- ✓ Run terraform fmt, validate and plan on every PR.
- ✓ Require manual approval before apply in prod.
- ✓ Use separate workspaces or folders per environment.
- ✓ Store backend config securely using variables / secrets.
- ✓ Notify team on success/failure.

Example: GitHub Actions Workflow (.github/workflows/terraform.yml)

```yaml
name: Terraform CI/CD

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  TF_VERSION: "1.5.7"
  AWS_REGION: "ap-south-1"

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${{ env.TF_VERSION }}

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Terraform Init
        run: terraform init -backend-config="key=envs/prod/terraform.tfstate"

      - name: Terraform Validate
        run: terraform validate -no-color

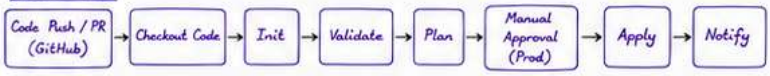
      - name: Terraform Plan
        run: terraform plan -var-file="envs/prod/terraform.tfvars" -out=tfplan

      - name: Terraform Apply (on main only)
        if: github.ref == 'refs/heads/main'
        run: terraform apply -auto-approve tfplan
```

Note:

- Plan runs on all PRs.
- Apply runs only on merged code (main).

Pipeline Flow



3. TERRAFORM DOCUMENTATION

Good documentation makes code easy to understand, maintain and handover.

Best Practices:

- ✓ Use clear README.md with purpose, setup and usage.
- ✓ Document input variables, outputs and resources.
- ✓ Add comments in complex logic.
- ✓ Use examples for running terraform.
- ✓ Keep architecture diagrams if needed.

Example: README.md (snippet)

```
# VPC Module
This module creates a VPC with public and private subnets.

## Usage
```hcl
module "vpc" {
  source = "../../modules/vpc"
  cidr_block = "10.0.0.0/16"
  azs = ["ap-south-1a", "ap-south-1b"]
  enable_nat_gateway = true
  tags = { Environment = "prod" }
}
```

Inputs
Name	Description	Type	Default	Required
cidr_block	VPC CIDR	string	-	yes
azs	List of AZs	list(string)	-	yes
enable_nat_gateway	Enable NAT GW	bool	true	no
tags	Additional tags	map(string)	{}	no

Outputs
Name	Description
vpc_id	ID of the VPC
public_subnets	List of public subnet IDs
private_subnets	List of private subnet IDs

Example
terraform init
terraform plan -var-file="example.tfvars"
terraform apply -var-file="example.tfvars"
```
```

Example: Variables & Outputs Docs

variables.tf (with description)

```hcl
variable "cidr_block" {
  description = "CIDR block for VPC"
  type        = string
}

variable "azs" {
  description = "List of Availability Zones"
  type        = list(string)
}

variable "enable_nat_gateway" {
  description = "Enable NAT Gateway"
  type        = bool
  default     = true
}
```

outputs.tf (with description)

```hcl
output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.this.id
}

output "public_subnets" {
  description = "Public subnet IDs"
  value       = aws_subnet.public[*].id
}
```

Tip:

- Keep README updated with each change.
- Use diagrams (draw.io, mermaid) for architecture clarity.
- Good docs = easy onboarding for anyone 🚀

Extra Tips (All 3 Topics)

- ✓ Protect main branch (require PR & reviews).
- ✓ Use meaningful commit messages.
- ✓ Run pre-commit hooks (fmt, validate).
- ✓ Use terraform workspace or separate folders per environment.
- ✓ Keep modules reusable and minimal.
- ✓ Automate notifications (Slack / Email).
- ✓ Always test in lower environments first.

Recommended Tools

- GitHub / GitLab / Bitbucket
- GitHub Actions / GitLab CI / Jenkins
- Terraform Registry / Private Module Registry
- Markdown (README) + Diagrams
- Slack / Email notifications

Code in Git, Automate in CI/CD, Document everything – That's how Infrastructure stays Secure, Consistent & Scalable !

Engineer_cloudops
```

Module 15 : Real-Time Scenarios

Engineer_cloudops

MULTI-ENVIRONMENT DEPLOYMENT

Goal : Use the same Terraform codebase to deploy in Dev, Staging & Prod with isolation, safety and easy management.

1. How we structure it

```
terraform-root/
├── modules/
│   ├── vpc/
│   ├── eks/
│   ├── rds/
│   └── ...
├── envs/
│   ├── dev/
│   ├── staging/
│   └── prod/
└── shared/
    ├── backend.tf
    └── providers.tf
```

Why this ?
- Environment specific configs are in envs/ folder.
- Modules are reusable across environments.

2. Backend (S3 + DynamoDB for Locking)

```
# backend.tf (in root or env level)
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "global/s3/backend.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "terraform-lock-table"
    encrypt        = true
  }
}
```

One state file per environment. Keeps state isolated.

3. Environment Directory Structure (Example)

envs/dev/	envs/staging/	envs/prod/
backend.tf	backend.tf	backend.tf
providers.tf	providers.tf	providers.tf
terraform.tfvars	terraform.tfvars	terraform.tfvars
main.tf	main.tf	main.tf
outputs.tf	outputs.tf	outputs.tf

We run terraform from inside each env folder.

4. providers.tf (Same in all envs)

```
terraform {
  required_version = ">= 1.3.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
provider "aws" {
  region = var.aws_region
}
```

Region and other values come from terraform.tfvars per environment.

Summary
- Keep code DRY using modules.
- Environment specific values in .tfvars.
- State isolation is the key to safe deployments.

5. Example : main.tf (Calling Modules)

```
module "vpc" {
  source = "../../modules/vpc"

  cidr_block = var.vpc_cidr
  azs        = var.azs
  env_name   = var.env_name
}

module "eks" {
  source       = "../../modules/eks"
  cluster_name = "${var.env_name}-eks"
  subnet_ids   = module.vpc.private_subnet_ids
  env_name     = var.env_name
}

module "rds" {
  source         = "../../modules/rds"
  identifier     = "${var.env_name}-db"
  db_subnet_group_name = module.vpc.db_subnet_group_name
  env_name       = var.env_name
}
```

Same modules used for all environments.

6. terraform.tfvars (Environment Specific Values)

```
# envs/dev/terraform.tfvars
env_name   = "dev"
aws_region = "ap-south-1"
vpc_cidr   = "10.0.0.0/16"
azs        = ["ap-south-1a", "ap-south-1b"]
tags       = { Environment = "dev" }
```

```
# envs/staging/terraform.tfvars
env_name   = "staging"
aws_region = "ap-south-1"
vpc_cidr   = "10.1.0.0/16"
azs        = ["ap-south-1a", "ap-south-1b"]
tags       = { Environment = "staging" }
```

```
# envs/prod/terraform.tfvars
env_name   = "prod"
aws_region = "ap-south-1"
vpc_cidr   = "10.2.0.0/16"
azs        = ["ap-south-1a", "ap-south-1b", "ap-south-1c"]
tags       = { Environment = "prod" }
```

Only values change, code stays same!

7. How to Deploy

```
cd envs/dev
terraform init
terraform plan -out=dev.tfplan
terraform apply dev.tfplan

cd ../staging
terraform init
terraform plan -out=staging.tfplan
terraform apply staging.tfplan

cd ../prod
terraform init
terraform plan -out=prod.tfplan
terraform apply prod.tfplan
```

Separate state file per env.

Safe, isolated & easy Rollback!

8. Best Practices

- ✓ Separate State file per environment.
- ✓ Use Workspaces (optional) OR separate folders (recommended).
- ✓ Restrict production changes with approvals.
- ✓ Use variable validation for safety.
- ✓ Tag all resources with Environment.

Pro Tip
Use CI/CD (GitHub Actions / Jenkins) to automate plan & apply per environment with manual approval for production.

Same Code + Different Configs = Scalable & Maintainable Infrastructure

Engineer_cloudops

Module 15: Real-Time Scenarios

1. Reusable VPC Module | 2. Highly Available Web Application

Engineer_cloudops

1. Reusable VPC Module

We create a generic VPC module which can be used in Dev, Staging, Prod with different inputs.

Folder Structure

```
modules/
└── vpc/
    ├── main.tf
    ├── variables.tf
    ├── outputs.tf
    └── README.md
```

Features

- ✓ Multi AZ
- ✓ Public & Private Subnets
- ✓ NAT Gateway
- ✓ Route Tables
- ✓ Reusable & Clean

Input Variables (variables.tf)

```
variable "vpc_cidr" {
  type = string
}

variable "public_subnets" {
  type = list(string)
}

variable "private_subnets" {
  type = list(string)
}

variable "azs" {
  type = list(string)
}

variable "name" {
  type = string
}
```

Main (main.tf) - Simplified

```
resource "aws_vpc" "this" {                                  # VPC
  cidr_block = var.vpc_cidr
  tags       = { Name = "${var.name}-vpc" }
}

resource "aws_internet_gateway" "igw" {                      # IGW
  vpc_id = aws_vpc.this.id
  tags   = { Name = "${var.name}-igw" }
}

resource "aws_subnet" "public" {                             # Public Subnets
  count                   = length(var.public_subnets)
  vpc_id                  = aws_vpc.this.id
  cidr_block              = var.public_subnets[count.index]
  availability_zone       = var.azs[count.index]
  map_public_ip_on_launch = true
  tags = {
    Name = "${var.name}-public-${count.index+1}"
  }
}

resource "aws_subnet" "private" {                            # Private Subnets
  count             = length(var.private_subnets)
  vpc_id            = aws_vpc.this.id
  cidr_block        = var.private_subnets[count.index]
  availability_zone = var.azs[count.index]
  tags = {
    Name = "${var.name}-private-${count.index+1}"
  }
}

# NAT Gateway (one per AZ)
# Route Tables, Associations ... (omitted for brevity)
```

Outputs (outputs.tf)

```
output "vpc_id" {
  value = aws_vpc.this.id }

output "public_subnet_ids" {
  value = aws_subnet.public[*].id }

output "private_subnet_ids" {
  value = aws_subnet.private[*].id }
```

How to Use

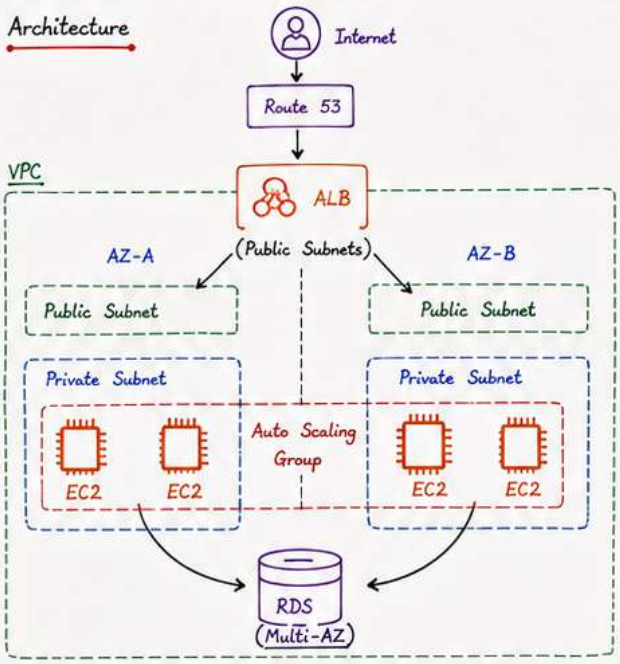
```
module "vpc" {
source     = "../modules/vpc"
vpc_cidr   = "10.0.0.0/16"
public_subnets  = ["10.0.1.0/24", "10.0.2.0/24"]
private_subnets = ["10.0.11.0/24", "10.0.12.0/24"]
azs        = ["ap-south-1a", "ap-south-1b"]
name       = "dev"
}
```

Note:
This module is environment agnostic. Just pass different cidr, subnets, azs & name.

2. Highly Available Web Application

Goal: Deploy a highly available web application using ALB, Auto Scaling, Multi AZ, RDS (Multi-AZ).

Architecture



All resources spread across 2 AZs for High Availability.

Terraform Implementation (Simplified)

1. Use our VPC Module

```
module "vpc" { source = "../modules/vpc" ... }
```

2. Create Security Groups

```
resource "aws_security_group" "alb_sg" { ... }
resource "aws_security_group" "web_sg" { ... }
```

3. Launch Application (EC2 + ASG)

```
resource "aws_launch_template" "web" { ... }
resource "aws_autoscaling_group" "web_asg" { ... }
```

4. Application Load Balancer

```
resource "aws_lb" "web_alb" { ... }
resource "aws_lb_target_group" "web_tg" { ... }
resource "aws_lb_listener" "http" { ... }
```

5. RDS Multi-AZ

```
resource "aws_db_subnet_group" "db" { ... }
resource "aws_db_instance" "appdb" {
  multi_az = true
}
```

Key Points

- ✓ ALB in Public Subnets
- ✓ EC2 in Private Subnets
- ✓ ASG across multiple AZs
- ✓ RDS Multi-AZ for HA
- ✓ Security Groups control traffic

Benefits

- ✓ No Single Point of Failure
- ✓ Auto Healing with ASG
- ✓ Scalable
- ✓ Secure
- ✓ Production Ready

Build Once → Reuse Everywhere → Scale Without Worries

Engineer_cloudops

Module 15: Real-Time Scenarios

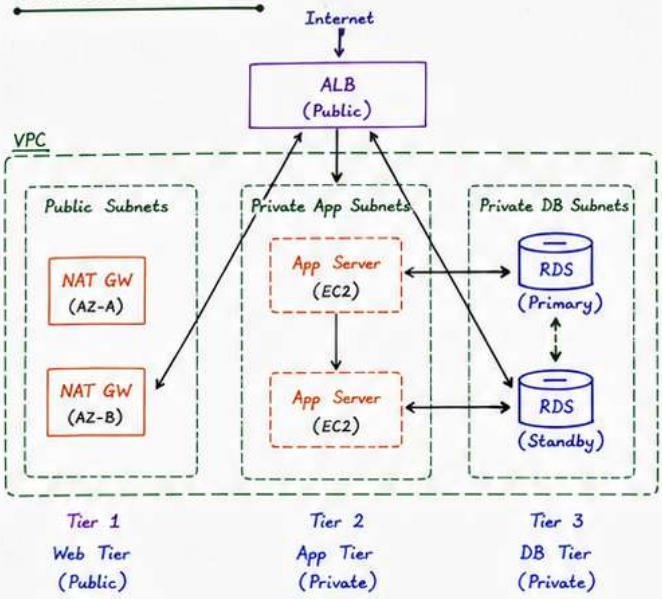
3. Three-Tier Architecture with Terraform

Engineer_cloudops

Goal :

Deploy a Three-Tier Web Application using Terraform.
Presentation (Web), Application (App), Database (DB).

Architecture Overview



Folder Structure

```
three-tier/
├── modules/
│   ├── vpc/
│   ├── app/
│   └── db/
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── terraform.tfvars
│   │   └── backend.tf
│   └── prod/
│       ├── main.tf
│       ├── terraform.tfvars
│       └── backend.tf
└── README.md
```

Tech Choices

- ✓ VPC with 2 AZs
- ✓ Public Subnets for ALB
- ✓ Private Subnets for App & DB
- ✓ NAT Gateway for private internet access
- ✓ ALB → App (EC2 ASG)
- ✓ RDS Multi-AZ for DB

Note :

- Keep DB in private subnets only.
- Use Security Groups to control traffic.
- Enable RDS Multi-AZ for High Availability.
- Store state in S3 + DynamoDB.

Environment Example (environments/dev/main.tf)

```
module "vpc" { source = ".../modules/vpc" ... }
module "sg"  { source = "../../modules/sg"  vpc_id
module "alb" { source = "../../modules/alb" ... }
module "app" { source = "../../modules/app" ... }
module "db"  { source = "../../modules/db"  ... }
```

Pass outputs from one module to another.
Example: module.vpc.public_subnet_ids

Terraform Implementation (Simplified)

1. VPC Module (modules/vpc/main.tf)

```
module "vpc" {
  source          = "../../modules/vpc"
  cidr_block      = var.vpc_cidr
  azs             = var.azs
  public_subnets  = var.public_subnets
  private_app_subnets = var.private_app_subnets
  private_db_subnets  = var.private_db_subnets
  enable_nat_gateway = true
}
```

Creates VPC, Subnets, IGW, NAT GW, Route Tables

2. Security Groups (modules/sg/main.tf)

```
resource "aws_security_group" "alb_sg" {
  name   = "alb-sg"
  vpc_id = var.vpc_id
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

- ALB SG → allow HTTP from Internet
- App SG → allow from ALB
- DB SG → allow from App SG

3. ALB (modules/alb/main.tf)

```
resource "aws_lb" "app_alb" {
  name     ="app-alb"
  internal = false
  load_balancer_type = "application"
  subnets   = var.public_subnet_ids
  security_groups = [var.alb_sg_id]
}
```

4. Application Tier (modules/app/main.tf) - Auto Scaling Group

```
resource "aws_launch_template" "app_lt" { ... }
resource "aws_autoscaling_group" "app_asg" {
  desired_capacity    = 2
  max_size            = 4
  min_size            = 2
  vpc_zone_identifier = var.app_subnet_ids
  target_group_arns   = [aws_lb_target_group.app_tg.arn]
  launch_template {
    id      = aws_launch_template.app_lt.id
    version = "$Latest"
  }
}
```

5. Database Tier (modules/db/main.tf)

```
resource "aws_db_subnet_group" "db_subnet" { ... }
resource "aws_db_instance" "app_db" {
  identifier          = "app-db"
  engine              = "mysql"
  instance_class      = "db.t3.micro"
  allocated_storage   = 20
  db_subnet_group_name = aws_db_subnet_group.db_subnet.name
  vpc_security_group_ids = [var.db_sg_id]
  multi_az            = true
  skip_final_snapshot = true
}
```

Three-Tier with Terraform → Secure, Scalable & Highly Available ☑

Engineer_cloudops

Module 15: Real-Time Scenarios

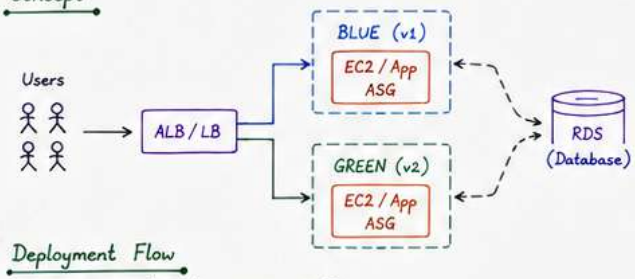
4. BLUE-GREEN DEPLOYMENT with Terraform

Engineer_cloudops

GOAL:

Deploy new version of application (Green) alongside current version (Blue). Test Green. Switch traffic to Green. Keep Blue as rollback. This ensures zero-downtime.

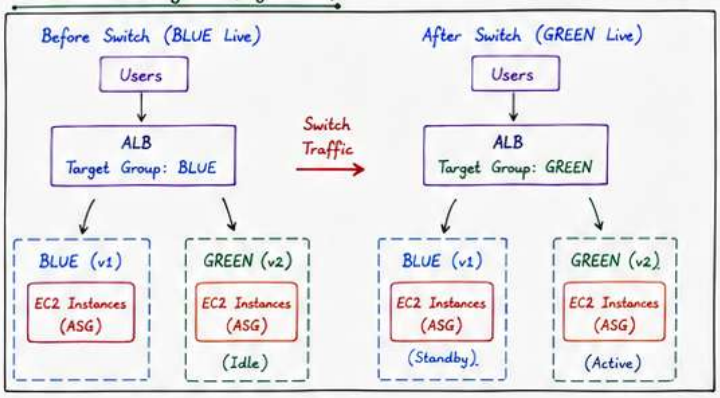
Concept



Deployment Flow

1. Current Prod (BLUE) is serving traffic.
2. Deploy new version in GREEN environment.
3. Run health checks / tests on GREEN.
4. Switch traffic from BLUE to GREEN (update target group).
5. Keep BLUE for quick rollback.
6. After stable, destroy BLUE resources.

Architecture Diagram (High Level)



Terraform Implementation (Simplified)

1. Variables (variables.tf)

```
variable "app_name" { default = "myapp" }
variable "env" { description = "blue or green" }
variable "instance_type" { default = "t3.micro" }
variable "desired_capacity" { default = 2 }
variable "image_id" {}
variable "target_group_arn" {}
```

2. Providers (providers.tf)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

3. Launch Template (lt.tf)

```
resource "aws_launch_template" "app" {
  name_prefix   = "${var.app_name}-${var.env}-"
  image_id      = var.image_id
  instance_type = var.instance_type
  user_data     = filebase64("user_data.sh")
  tag_specifications {
    resource_type = "instance"
    tags = {
      Name        = "${var.app_name}-${var.env}"
      Environment = var.env
    }
  }
}
```

4. Auto Scaling Group (asg.tf)

```
resource "aws_autoscaling_group" "app" {
  name                = "${var.app_name}-${var.env}-asg"
  desired_capacity    = var.desired_capacity
  max_size            = var.desired_capacity * 2
  min_size            = var.desired_capacity
  vpc_zone_identifier = var.private_subnet_ids
  target_group_arns   = [var.target_group_arn]
  health_check_type   = "ELB"
  launch_template {
    id      = aws_launch_template.app.id
    version = "$Latest"
  }
  tag {
    key                 = "Environment"
    value               = var.env
    propagate_at_launch = true
  }
}
```

5. Target Groups (tg.tf)

```
resource "aws_lb_target_group" "app" {
  name     = "${var.app_name}-${var.env}-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = var.vpc_id
  health_check {
    path                = "/health"
    matcher             = "200"
    interval            = 30
    timeout             = 5
    healthy_threshold   = 3
    unhealthy_threshold = 2
  }
}
```

6. Application Load Balancer (alb.tf)

```
resource "aws_lb" "app" {
  name               = "${var.app_name}-alb"
  load_balancer_type = "application"
  subnets            = var.public_subnet_ids
  security_groups    = [var.alb_sg_id]
}
```

7. Listener (listener.tf)

```
resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type             = "forward"
    target_group_arn = var.target_group_arn  # BLUE or GREEN TG
  }
}
```

To switch traffic, just change target_group_arn from BLUE TG to GREEN TG and apply!

8. Outputs (outputs.tf)

```
output "alb_dns" { value = aws_lb.app.dns_name }
output "blue_tg_arn" { value = aws_lb_target_group.blue.arn }
output "green_tg_arn" { value = aws_lb_target_group.green.arn }
```

How to Deploy (Step by Step)

1. Deploy BLUE environment:
   `terraform apply -var "env=blue" -var "target_group_arn=<blue_tg_arn>"`
2. Test application on BLUE.
3. Deploy GREEN environment:
   `terraform apply -var "env=green" -var "target_group_arn=<green_tg_arn>"`
4. Validate GREEN (health checks, smoke tests).
5. Switch traffic to GREEN by updating listener to GREEN TG and apply.
6. Monitor for some time.
7. Keep BLUE as rollback.
8. Destroy BLUE when confident:
   `terraform destroy -var "env=blue"`

Benefits

- ✓ Zero downtime deployments
- ✓ Easy rollback (switch back to BLUE)
- ✓ Safe testing of new version
- ✓ No impact to users during deployment

Notes / Best Practices

- Keep environments isolated (separate ASG, TG, etc).
- Use meaningful tags (Name, Environment).
- Run automated tests on GREEN before switch.
- Use CloudWatch Alarms for both environments.
- After switch, monitor before decommissioning BLUE.

Blue Today, Green Tomorrow – Zero Downtime Always! ☑

Engineer_cloudops

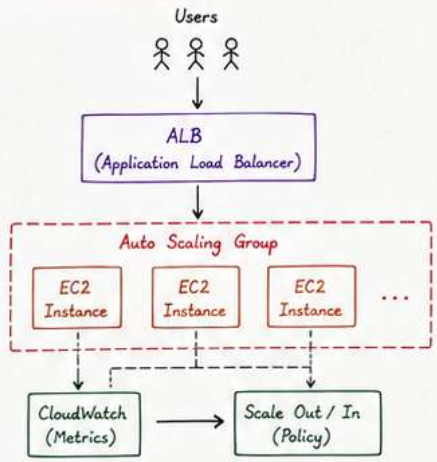
Module 15: Real-Time Scenarios

5. AUTO SCALING INFRASTRUCTURE with Terraform

Engineer_cloudops

Goal: Build an Auto Scaling Infrastructure using Terraform with ALB + ASG + Launch Template + Target Tracking Policy

Architecture Overview



How it works:

1. Traffic comes to ALB.
2. ALB forwards to ASG instances.
3. CloudWatch monitors metrics (CPU, Request Count etc).
4. If metrics cross threshold → Scale Out
5. If metrics go down → Scale In

Terraform Implementation (Simplified)

1. Launch Template (launch_template.tf)

```
resource "aws_launch_template" "app_lt" {
name_prefix     = "app-lt-"
image_id        = var.ami_id
instance_type   = var.instance_type
key_name        = var.key_name

user_data = base64encode(<<-EOF
    #!/bin/bash
    yum install -y httpd
    systemctl start httpd
    systemctl enable httpd
    echo "Hello from $(hostname)" > /var/www/html/index.html
  EOF
)
tag_specifications {
  resource_type = "instance"
  tags = {
     Name     = "app-instance"
  }
}
```

2. Target Group (target_group.tf)

```
resource "aws_lb_target_group" "app_tg" {
name        = "app-tg"
port        = 80
protocol    = "HTTP"
vpc_id      = var.vpc_id
health_check {
  path                = "/"
  matcher             = "200"
  interval            = 30
  timeout             = 5
  healthy_threshold   = 2
  unhealthy_threshold = 2
 }
}
```

3. Auto Scaling Group (asg.tf)

```
resource "aws_autoscaling_group" "app_asg" {
name                = "app-asg"
desired_capacity    = 2
min_size            = 2
max_size            = 6
vpc_zone_identifier = var.private_subnet_ids
target_group_arns   = [aws_lb_target_group.app_tg.arn]
health_check_type   = "ELB"
health_check_grace_period = 300

launch_template {
    id      = aws_launch_template.app_lt.id
    version = "$Latest"
}

tag {
    key                 = "Name"
    value               = "app-instance"
    propagate_at_launch = true
 }
}
```

4. Application Load Balancer (alb.tf)

```
resource "aws_lb" "app_alb" {
name               = "app-alb"
internal           = false
load_balancer_type = "application"
security_groups    = [var.alb_sg_id]
subnets            = var.public_subnet_ids
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app_alb.arn
  port              = "80"
  protocol          = "HTTP"
  default_action {
     type             = "forward"
     target_group_arn = aws_lb_target_group.app_tg.arn
  }
}
```

5. Scaling Policy (scale_policy.tf)

```
resource "aws_autoscaling_policy" "cpu_target_tracking" {
name                   = "cpu-target-tracking"
autoscaling_group_name = aws_autoscaling_group.app_asg.name
policy_type            = "TargetTrackingScaling"
estimated_instance_warmup = 120

target_tracking_configuration {
   predefined_metric_specification {
      predefined_metric_type = "ASGAverageCPUUtilization"
   }
   target_value = 60.0     # keep avg CPU around 60%
 }
}
```

6. CloudWatch Alarm (Optional - for Scale In Protection)

```
resource "aws_cloudwatch_metric_alarm" "high_cpu_alarm" {
alarm_name          = "high-cpu-alarm"
comparison_operator = "GreaterThanOrEqualToThreshold"
evaluation_periods  = 2
metric_name         = "CPUUtilization"
namespace           = "AWS/EC2"
period              = 120
statistic           = "Average"
threshold           = 70
alarm_description   = "Scale out when CPU > 70%"
dimensions = {
   AutoScalingGroupName = aws_autoscaling_group.app_asg.name
 }
}
```

7. Security Group (sg.tf)

```
resource "aws_security_group" "alb_sg" {
name        = "alb-sg"
description = "Allow HTTP"
vpc_id      = var.vpc_id

ingress {
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
```

8. Variables (variables.tf)

```
variable "vpc_id" {}
variable "public_subnet_ids" {
    type = list(string)
}

variable "private_subnet_ids" {
    type = list(string)
}

variable "ami_id" {}
variable "instance_type" {
    default = "t3.micro"
}

variable "key_name" {}
variable "alb_sg_id" {}
```

9. Outputs (outputs.tf)

```
output "alb_dns_name" {
    value = aws_lb.app_alb.dns_name
}

output "asg_name" {
  value = aws_autoscaling_group.app_asg.name
}
```

Important Notes

- ✓ Keep desired_capacity = Min size for cost optimization.
- ✓ Use Target Tracking Policy (recommended).
- ✓ Set proper Health Checks to avoid bad instances.
- ✓ Use separate SG for ALB and Instances.
- ✓ Monitor in CloudWatch: CPU, RequestCount, 5XX errors.

Auto Scaling gives Elasticity + High Availability + Cost Optimization ☑

Engineer_cloudops

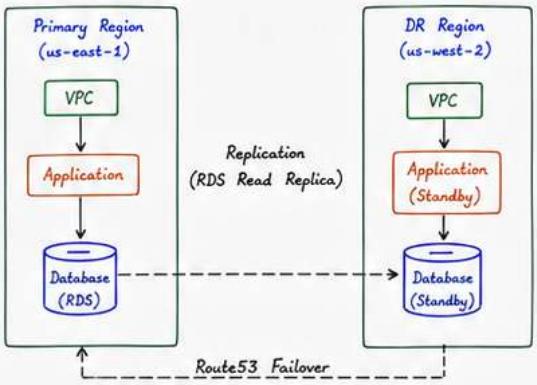
Module 15: Real-Time Scenarios

6. DISASTER RECOVERY SETUP with Terraform

Engineer_cloudops

Goal: Design a Multi-Region Disaster Recovery Setup using Terraform.
Primary Region (Active) → Secondary Region (Standby)
In case of disaster, failover to DR Region.

Architecture Overview



Disaster Recovery Flow

1. Primary (us-east-1) is active.
2. Data is replicated to DR (us-west-2) using RDS Read Replica.
3. Route53 Health Check monitors primary.
4. If primary fails → Route53 switches traffic to DR.
5. DR becomes active.

Terraform Implementation (Overview)

1. VPC in both regions
2. RDS Primary in Primary Region
3. RDS Read Replica in DR Region
4. Route53 Failover Policy
5. Health Check for Primary

1. Provider Configuration (providers.tf)

```
provider "aws" {
  alias  = "primary"
  region = "us-east-1"
}

provider "aws" {
  alias  = "dr"
  region = "us-west-2"
}
```

2. VPC (vpc.tf)

```
module "vpc_primary" {
  source    = "terraform-aws-modules/vpc/aws"
  version   = "~> 5.0"
  providers = { aws = aws.primary }

  name            = "app-vpc-primary"
  cidr            = "10.0.0.0/16"
  azs             = ["us-east-1a", "us-east-1b"]
  public_subnets  = ["10.0.1.0/24", "10.0.2.0/24"]
  private_subnets = ["10.0.3.0/24", "10.0.4.0/24"]
}

module "vpc_dr" {
  source    = "terraform-aws-modules/vpc/aws"
  version   = "~> 5.0"
  providers = { aws = aws.dr }

  name            = "app-vpc-dr"
  cidr            = "10.1.0.0/16"
  azs             = ["us-west-2a", "us-west-2b"]
  public_subnets  = ["10.1.1.0/24", "10.1.2.0/24"]
  private_subnets = ["10.1.3.0/24", "10.1.4.0/24"]
}
```

3. RDS Primary & Read Replica (rds.tf)

```
# Primary RDS (us-east-1)
resource "aws_db_instance" "primary" {
  provider            = aws.primary
  identifier          = "app-db-primary"
  engine              = "mysql"
  instance_class      = "db.t3.micro"
  allocated_storage   = 20
  db_name             = "appdb"
  username            = "admin"
  password            = var.db_password
  skip_final_snapshot = true
  multi_az            = true
  backup_retention_period      = 7
}

# Read Replica (us-west-2)
resource "aws_db_instance" "read_replica" {
  provider            = aws.dr
  identifier          = "app-db-dr"
  replicate_source_db = aws_db_instance.primary.arn
  instance_class      = "db.t3.micro"
  skip_final_snapshot = true
  multi_az            = true
}
```

4. Route53 Failover (route53.tf)

```
resource "aws_route53_zone" "main" {
  name = "example.com"
}

# Health Check for Primary
resource "aws_route53_health_check" "primary" {
  fqdn              = "app.example.com"
  type              = "HTTP"
  port              = 80
  resource_path     = "/health"
  failure_threshold = 3
  request_interval  = 30
}

# Failover Record
resource "aws_route53_record" "app" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "app.example.com"
  type    = "A"

  set_identifier  = "primary"
  failover        = "PRIMARY"
  health_check_id = aws_route53_health_check.primary.id
  records         = ["<ALB-DNS-PRIMARY>"]
  ttl             = 60
}

resource "aws_route53_record" "app_dr" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "app.example.com"
  type    = "A"

  set_identifier  = "dr"
  failover        = "SECONDARY"
  records         = ["<ALB-DNS-DR>"]
  ttl             = 60
}
```

5. Variables (variables.tf)

```
variable "db_password" {
  type      = string
  sensitive = true
}

variable "domain_name" {
  type    = string
  default = "example.com"
}
```

Failover Process

1. Primary goes down (Health Check fails).
2. Route53 detects failure.
3. Traffic automatically routed to DR (us-west-2).
4. DR becomes Active.
5. Fix Primary and Re-sync if needed.

Benefits

- ✓ High Availability
- ✓ Automatic Failover
- ✓ Minimal Downtime
- ✓ Data Protection
- ✓ Business Continuity

Important Notes

- ✓ Keep Secondary (DR) in Standby (No active traffic).
- ✓ Use RDS Read Replica for DB replication.
- ✓ Regularly test Failover to DR.
- ✓ Monitor with CloudWatch Alarms.

Plan for Failure, Build for Recovery, Stay Resilient ☑

Engineer_cloudops

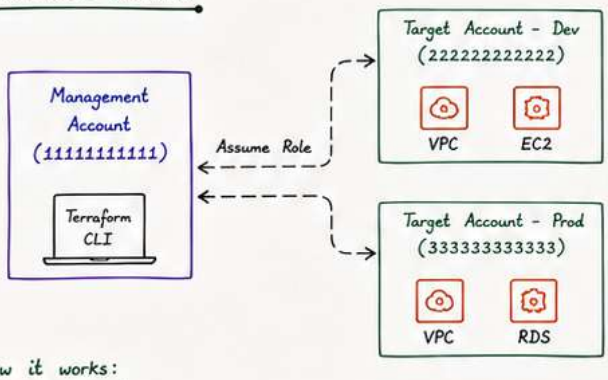
Module 15: Real-Time Scenarios

7. CROSS-ACCOUNT DEPLOYMENT with Terraform

Engineer_cloudops

GOAL: Deploy infrastructure in multiple AWS accounts using Terraform. Central account will manage (Provision) resources in Target accounts.

Architecture Overview



How it works:

1. Terraform in Management Account uses IAM Role (in target account) via AssumeRole.
2. Terraform gets temporary credentials.
3. Resources are created in Target Account.
4. Repeat for other accounts / environments.

Prerequisites:

- ✓ IAM Role in each target account with required permissions.
- ✓ Trust relationship to allow Management account to assume role.
- ✓ Proper IAM policies attached to the role.

Terraform Implementation

1. Provider Configuration (providers.tf)

```
# Provider for Management Account (default)
provider "aws" {
  region  = "us-east-1"
}

# Provider for Dev Account
provider "aws" {
  alias   = "dev"
  region  = "us-east-1"
  assume_role {
    role_arn    = "arn:aws:iam::222222222222:role/TerraformExecutionRole"
    session_name  = "terraform-dev-session"
  }
}
# Provider for Prod Account
provider "aws" {
  alias   = "prod"
  region  = "us-east-1"
  assume_role {
    role_arn    = "arn:aws:iam::333333333333:role/TerraformExecutionRole"
    session_name  = "terraform-prod-session"
  }
}
```

2. Variables (variables.tf)

```
variable "aws_region" {
  default     = "us-east-1"
}

variable "dev_account_id" {
  default   = "222222222222"
}

variable "prod_account_id" {
  default   = "333333333333"
}
```

3. Resources in Dev Account (dev.tf)

```
resource "aws_vpc" "dev_vpc" {
  provider      = aws.dev
  cidr_block    = "10.0.0.0/16"
  tags  = {
    Name        = "dev-vpc"
    Environment = "Dev"
  }
}

resource "aws_instance" "dev_ec2" {
  provider      = aws.dev
  ami           = "ami-0abcdef1234567890"
  instance_type = "t3.micro"
  subnet_id     = "subnet-12345678"
  tags = {
    Name        = "dev-ec2"
  }
}
```

↔ Resources will be created in Dev Account (2222...)

4. Resources in Prod Account (prod.tf)

```
resource "aws_db_instance" "prod_db" {
  provider      =  aws.prod
  identifier        = "prod-db"
  engine            = "mysql"
  instance_class    = "db.t3.micro"
  allocated_storage =  20
  username          = "admin"
  password          = "StrongPass@123"
  skip_final_snapshot =  true
  tags  = {
    Name            = "prod-db"
    Environment     = "Prod"
  }
}
```

↔ Resources will be created in Prod Account (3333...)

5. Outputs (outputs.tf)

```
output "aws_vpc_id" {
  value     = aws_vpc.dev_vpc.id
}

output "prod_db_endpoint" {
  value     = aws_db_instance.prod_db.endpoint
}
```

IAM Role Setup in Target Account (Important)

a) Trust Policy (in Target Account)

```
{
    "Version" : "2012-10-17",
    "Statement" : [
      {
        "Effect" : "Allow",
        "Principal" : {
          "AWS" : "arn:aws:iam::111111111111:root"
        },
        "Action" : "sts:AssumeRole"
      }
    ]
}
```

b) Permission Policy (Attach to Role)

```
{
    "Version" : "2012-10-17",
    "Statement" : [
      {
          "Effect" : "Allow",
          "Action" : [
              "ec2:*",
              "rds:*",
              "vpc:*",
              "iam:PassRole"
          ],
          "Resource" : "*"
      }
    ]
}
```

Usage Steps

1. Create IAM Role in each target account.
2. Update role_arn in providers.tf
3. Run: terraform init
4. Run: terraform plan
5. Run: terraform apply

Best Practices

- ✓ Use least privilege IAM policies.
- ✓ Use different roles for dev/prod (if needed).
- ✓ Store role ARNs in variables or tfvars.
- ✓ Enable MFA delete for state backend.
- ✓ Use Workspaces for multiple environments.

One Terraform Codebase, Multiple Accounts, Full Automation! ☑

Engineer_cloudops

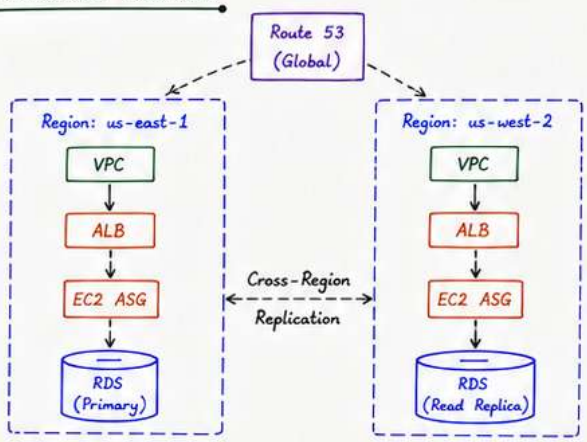
Module 15: Real-Time Scenarios

8. MULTI-REGION DEPLOYMENT with Terraform

Engineer_cloudops

GOAL: Deploy same application in multiple AWS regions for High Availability, Low Latency & Disaster Recovery using Terraform.

Architecture Overview



How it works:

1. Route53 routes users to closest region (Latency Based Routing).
2. Application & DB are deployed in multiple regions.
3. RDS Read Replica keeps data in sync.
4. If one region fails, traffic automatically goes to healthy region.

Terraform Implementation (Overview)

1. Use Provider Aliases for multiple regions.
2. Create VPC, Subnets, ALB, ASG in each region.
3. Use RDS Multi-AZ in each region with Cross-Region Replica.
4. Use Route53 for traffic routing.

1. Providers (providers.tf)

```
provider "aws" {
  alias  = "use1"
  region = "us-east-1"
}

provider "aws" {
  alias  = "usw2"
  region = "us-west-2"
}
```

2. VPC in us-east-1 (vpc-use1.tf)

```
resource "aws_vpc" "use1_vpc" {
  provider   = aws.use1
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "use1-vpc"
  }
}
```

3. VPC in us-west-2 (vpc-usw2.tf)

```
resource "aws_vpc" "usw2_vpc" {
  provider   = aws.usw2
  cidr_block = "10.1.0.0/16"

  tags = {
    Name = "usw2-vpc"
  }
}
```

4. ALB in us-east-1 (alb-use1.tf)

```
resource "aws_lb" "app_alb_use1" {
  provider           = aws.use1
  name               = "app-alb-use1"
  internal           = false
  load_balancer_type = "application"
  subnets            = var.use1_public_subnets
  security_groups    = [aws_security_group.alb_use1.id]
}
```

5. ALB in us-west-2 (alb-usw2.tf)

```
resource "aws_lb" "app_alb_usw2" {
  provider           = aws.usw2
  name               = "app-alb-usw2"
  internal           = false
  load_balancer_type = "application"
  subnets            = var.usw2_public_subnets
  security_groups    = [aws_security_group.alb_usw2.id]
}
```

6. Auto Scaling Group in us-east-1 (asg-use1.tf)

```
resource "aws_autoscaling_group" "app_asg_use1" {
  provider            = aws.use1
  name                = "app-asg-use1"
  desired_capacity    = 2
  max_size            = 4
  min_size            = 2
  vpc_zone_identifier = var.use1_private_subnets
  target_group_arns   = [aws_lb_target_group.app_tg_use1.arn]
  launch_template {
    id      = aws_launch_template.app_lt_use1.id
    version = "$Latest"
  }
}
```

7. Auto Scaling Group in us-west-2 (asg-usw2.tf)

```
resource "aws_autoscaling_group" "app_asg_usw2" {
  provider            = aws.usw2
  name                = "app-asg-usw2"
  desired_capacity    = 2
  max_size            = 4
  min_size            = 2
  vpc_zone_identifier = var.usw2_private_subnets
  target_group_arns   = [aws_lb_target_group.app_tg_usw2.arn]
  launch_template {
    id      = aws_launch_template.app_lt_usw2.id
    version = "$Latest"
  }
}
```

8. RDS in Primary Region (use1) (rds-use1.tf)

```
resource "aws_db_instance" "app_db_primary" {
  provider            = aws.use1
  identifier          = "app-db-primary"
  engine              = "mysql"
  instance_class      = "db.t3.micro"
  allocated_storage   = 20
  username            = "admin"
  password            = "StrongPass@123"
  multi_az            = true
  backup_retention_period = 7
  skip_final_snapshot = true
}
```

9. RDS Read Replica in us-west-2 (rds-usw2.tf)

```
resource "aws_db_instance" "app_db_replica" {
  provider            = aws.usw2
  identifier          = "app-db-replica"
  replicate_source_db = aws_db_instance.app_db_primary.arn
  instance_class      = "db.t3.micro"
  skip_final_snapshot = true
  publicly_accessible = false
}
```

10. Route53 (route53.tf)

```
resource "aws_route53_record" "app_dns" {
  zone_id = "Z1234567890ABCDEFG"
  name    = "app.example.com"
  type    = "A"
  set_identifier = "app-latency"
  latency_routing_policy {
    region = "us-east-1"
  }
  alias {
    name                   = aws_lb.app_alb_use1.dns_name
    zone_id                = aws_lb.app_alb_use1.zone_id
    evaluate_target_health = true
  }
}
```

(Create another record for us-west-2 with its ALB DNS)

Benefits

- ✓ High Availability
- ✓ Low Latency
- ✓ Disaster Recovery
- ✓ Scalable Global App

Important Notes

- Use Provider Aliases
- Keep CIDR blocks non-overlapping
- Use Cross-Region Replication for DB
- Test failover by disabling a region

Multiple Regions, One Goal → High Availability & Happy Users! ☑

Engineer_cloudops

Module 15: Real-Time Scenarios

9. PRODUCTION ROLLBACK STRATEGY

Engineer_cloudops

GOAL: Quickly and safely rollback a failed deployment with minimal downtime and data loss.
Be ready before the incident, not during it.

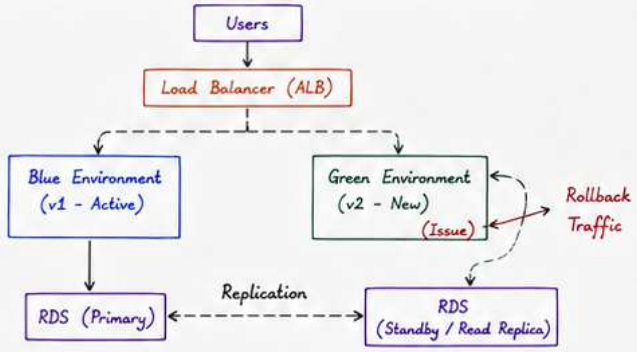
Rollback Overview

1. Detect Issue → 2. Validate the Issue → 3. Decide Rollback → 4. Execute Rollback → 5. Verify & Monitor

Rollback Approaches (Choose based on architecture)

1. **Application Level Rollback**
   Revert to previous version (via deployment, AMI, container image, etc.)
2. **Infrastructure Rollback**
   Revert infra changes using Terraform state or previous plan
3. **Database Rollback**
   Use automated backups, snapshots, PITR
4. **Traffic Rollback**
   Switch traffic back (Blue-Green / Canary / DNS / ALB)
5. **Configuration Rollback**
   Revert config changes (Feature flags, parameter store, etc.)

Rollback Flow (Example using Blue-Green Deployment)



Pre-Rollback Checklist (Before you rollback)

- ✓ Know last successful version (tag / image / AMI)
- ✓ Ensure DB backups are available
- ✓ Have rollback scripts / automation ready
- ✓ Communicate with team & stakeholders
- ✓ Monitor system health & business metrics

Rollback Commands / Actions (Examples)

1. Kubernetes - Rollback to previous deployment

```
kubectl rollout undo deployment/app-deploy -n prod
```

2. AWS - Rollback to previous AMI in Auto Scaling Group

```
aws autoscaling set-instance-protection
  --instance-ids <id> --no-protected-from-scale-in
```

3. Terraform - Revert Infrastructure

```
terraform apply -var-file=rollback.tfvars
```

4. RDS - Restore from Snapshot

```
aws rds restore-db-instance-from-db-snapshot
  --db-snapshot-identifier  <snapshot-id>
  --db-instance-identifier <new-db>
```

Terraform Rollback Strategy

1. Keep remote state in S3 + DynamoDB (Locking)
2. Every change via Pull Request & Code Review
3. If new change fails:
   - Revert the code to previous commit
   - Run terraform plan
   - Run terraform apply
4. Or use terraform state rm / import if required
5. Always backup state before major changes

Example: Rollback Infrastructure Change

Problem: A new security group blocked traffic.

Steps:

① Identify the change
② Revert code to previous version (Git)
③ Run terraform plan (verify)
④ Run terraform apply
⑤ Verify resources are back to previous state

Database Rollback Strategy

1. Take automated snapshots before release
2. If issue occurs:
   - Restore from snapshot
   - Or use PITR (Point-in-time recovery)
3. Validate data & application connection

Best Practices

- ✓ Automate Rollback (Scripts / Pipelines)
- ✓ Use Blue-Green or Canary for zero/low downtime
- ✓ Monitor metrics & logs after deployment
- ✓ Keep backups & snapshots tested
- ✓ Document rollback steps & keep updated

CI/CD Rollback (Jenkins Example)

```
stage('Rollback') {
  when { expression { params.ROLLBACK == 'true' } }
  steps {
    echo "Starting Rollback..."
    sh 'kubectl rollout undo deployment/app-deploy -n prod"
    sh "terraform apply -var-file=rollback.tfvars -auto-approve'
  }
}
```

Incident Response Quick Notes

- → Stay calm. Identify the impact.
- → Roll back fast, then fix.
- → Communicate clearly.
- → Do Post-Incident Review (PIR).
- → Automate what can be automated.

Deploy with Confidence, Rollback with Speed! ☑

EC Engineer_cloudops

Module 16: Interview & Troubleshooting

TOP 100 TERRAFORM INTERVIEW QUESTIONS

Engineer_cloudops

A. Basic (1-20)

1. Q: What is Terraform?
   A: IaC tool by HashiCorp to provision infra.
2. Q: Terraform vs CloudFormation?
   A: Terraform is multi-cloud, uses HCL.
3. Q: What is HCL?
   A: HashiCorp Configuration Language.
4. Q: What is terraform init?
   A: Initializes working directory.
5. Q: What is terraform plan?
   A: Shows execution plan.
6. Q: What is terraform apply?
   A: Applies the changes.
7. Q: What is terraform destroy?
   A: Destroys the resources.
8. Q: What is state file?
   A: Tracks real infra vs config.
9. Q: Where is state stored by default?
   A: Locally in terraform.tfstate.
10. Q: What is a provider?
    A: Plugin to interact with APIs.
11. Q: What is a resource?
    A: A managed infrastructure object.
12. Q: What is a data source?
    A: Read-only resource from provider.
13. Q: What is a module?
    A: Reusable collection of resources.
14. Q: What is terraform.lock.hcl?
    A: Locks provider versions.
15. Q: What is input variable?
    A: User input via variables.
16. Q: What is output value?
    A: Exposes values after apply.
17. Q: What is terraform fmt?
    A: Formats HCL code.
18. Q: What is terraform validate?
    A: Validates config syntax.
19. Q: What is terraform version?
    A: Shows Terraform version.
20. Q: What is terraform providers?
    A: Lists required providers.

B. Intermediate (21-40)

21. Q: How to pass variables?
    A: Using -var or tfvars file.
22. Q: What is count?
    A: Creates multiple instances.
23. Q: What is for_each?
    A: Creates resources from map/set.
24. Q: Difference between count & for_each?
    A: count uses index, for_each uses keys.
25. Q: What is a local value?
    A: Reuse expressions within config.
26. Q: How to use data source?
    A: data "aws_ami" "example" { ... }
27. Q: What is depends_on?
    A: Explicit dependency between resources.
28. Q: How to handle sensitive data?
    A: Use sensitive = true.
29. Q: What is a null_resource?
    A: Executes provisioners.
30. Q: What are provisioners?
    A: Run scripts (local/remote).
31. Q: How to output sensitive values?
    A: output with sensitive = true.
32. Q: How to manage multiple providers?
    A: Use alias providers.
33. Q: What is workspace?
    A: Multiple state files in one config.
34. Q: How to create a module?
    A: Put resources in a folder.
35. Q: How to call a module?
    A: module "name" { source = "..." }
36. Q: What is terraform graph?
    A: Generates dependency graph.
37. Q: How to import existing resource?
    A: terraform import <addr> <id>
38. Q: What is taint?
    A: Marks resource to recreate.
39. Q: How to remove taint?
    A: terraform untaint <addr>
40. Q: What is replacement?
    A: Resource destroyed and recreated.

C. Advanced (41-70)

41. Q: How to work with multiple envs?
    A: Workspaces / separate state files.
42. Q: What is remote state?
    A: Store state in S3, Azure, GCS, etc.
43. Q: How to configure S3 backend?
    A: backend "s3" { ... }
44. Q: What is state locking?
    A: Prevent concurrent state changes.
45. Q: How to enable locking in S3?
    A: Use DynamoDB table.
46. Q: What is tfvars file?
    A: Auto-loaded variable file.
47. Q: How to override variables?
    A: -var-file, -var, env vars.
48. Q: What is dynamic block?
    A: Generate nested blocks dynamically.
49. Q: What is lifecycle block?
    A: Control resource behavior.
50. Q: What is create_before_destroy?
    A: Creates new before destroying old.
51. Q: What is ignore_changes?
    A: Ignore specific attribute changes.
52. Q: How to prevent resource destroy?
    A: lifecycle { prevent_destroy = true }
53. Q: What is time_sleep resource?
    A: Wait for given time.
54. Q: How to handle dependencies?
    A: Implicit + depends_on.
55. Q: What is external data source?
    A: Call external program.
56. Q: How to use templatefile()?
    A: Render templates.
57. Q: What is jsonencode()?
    A: Convert object to JSON.
58. Q: How to split configuration?
    A: Modules + env specific tfvars.
59. Q: How to validate variables?
    A: validation block in variable.
60. Q: What is a provider alias?
    A: Use multiple configs of provider.
61. Q: How to pass providers to module?
    A: providers = { aws = aws.usw2 }
62. Q: What is precondition/postcondition?
    A: Validate conditions in lifecycle.
63. Q: How to use for expressions?
    A: [for x in list : x.name]
64. Q: What is setproduct()?
    A: Cartesian product of sets.
65. Q: What is try() function?
    A: Handle errors in expressions.
66. Q: What is can() function?
    A: Returns true if expression valid.
67. Q: How to manage secrets?
    A: Use AWS Secrets Manager / SSM.
68. Q: How to version modules?
    A: Use Git tags / branches.
69. Q: What is terraform cloud?
    A: Remote infra management.
70. Q: Difference between plan & apply?
    A: Plan shows changes, apply executes.

D. Troubleshooting (71-90)

71. Q: State file out of sync?
    A: Refresh or import resource.
72. Q: Error: Provider configuration not present?
    A: Add correct provider block.
73. Q: Error: Resource already exists?
    A: Import or remove manually.
74. Q: Error: Unsupported argument?
    A: Check provider docs & version.
75. Q: Error: Invalid index?
    A: Check count/for_each usage.
76. Q: Error: Cycle detected?
    A: Remove circular dependency.
77. Q: How to debug plan?
    A: terraform plan -debug
78. Q: How to see more logs?
    A: TF_LOG=DEBUG terraform plan
79. Q: State corrupted?
    A: Restore from backup.
80. Q: Lock file exists?
    A: Remove lock manually.

80. Q: Lock ga fail for corrup?
81. Q: Remove lock manually
88. Q: Chec ian. ırle ıösan?
82. Q: APrate cemit exceeded?
    A: Increase limits or retry.
83. Q: Resource stuck in destroying?
    A: Remove from state and retry.
84. Q: Remote state not found?
    A: Check bucket/key & permissions.
85. Q: Different results on apply?
    A: Check time-based functions.
86. Q: Plan shows no changes, but
    A: A...$medx external. ünges.
87. Q: Module not found error?
    A: Check source path/version.
88. Q: Wat is -replace flag?
    A: Keeqe replace a resource.
89. Q: How to refresh state?
    A: terraform refresh
90. Q: How to clean age jo...dis?
    A: Run terrafmrn ieuuen aprage.
    A: Check ıocat sate, plan first, apply.

E. Best Practices (91-100)

91. Q: How to structure Terraform code?
    Root modul + env folders.
92. Q: Where to store state file?
    A: Remote (backend)
93. Q: How to secure state file?
    A: Enrypt: backend + restrict access.
94. Q: How to version control Terraform?
    Store in Git with branching.
95. Q: How to review changes resiage?
    Faste cinsuan...
95. Q: How to manage Sariablas?
    + retonje a access.
98. Q: How to tag a manage from?
    Sotrert in variabl." }
99. Q: What is a provisuce
    Use default tags / locals.
100. Q: Gallen u ofor providers
    A: Keep state safe, plan first, then apply.

Quick Tips

- Always run terraform plan before apply
- Store state remotely with locking
- Use modules for reusability
- Keep backups of state file
- Automate with CI/CD

Practice is the key!
Build, Break, Fix, Repeat. ☑



Engineer_cloudops

Module 16: Interview & Troubleshooting

COMMON TERRAFORM ERRORS

Engineer_cloudops

#	ERROR / MESSAGE	CAUSE	SOLUTION / FIX	EXAMPLE / NOTES
1.	Error: No valid credential sources found	AWS credentials not configured or expired.	• Configure AWS credentials using aws configure • Set AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY • Or use IAM role.	aws configure terraform init terraform plan
2.	Error: Backend initialization required, please run "terraform init"	Working with backend but init not run.	Run terraform init to initialize the backend.	terraform init terraform plan

3.	Error: Provider configuration not present	Provider block missing or not correctly defined.	Add required provider block with version and region.	provider "aws" {
				region = "us-east-1"
				}

4.	Error: Reference to undeclared resource	Resource/variable/module name is wrong.	Check spelling and make sure resource exists in config.	aws_instance.web (not aws_instance.web1)
5.	Error: Invalid index	Using wrong index in count or for_each.	Verify index/key exists in the list/map.	resource[0] not resource[5]
6.	Error: Cycle: ...	Cyclic dependency between resources.	Remove circular dependency. Use depends_on only when required.	A → B → C → A (cycle) ✕
7.	Error: Unsupported argument	Argument not supported in this resource/provider.	Check provider docs for valid arguments.	e.g. specifying "subnet_id" in aws_s3_bucket (invalid)
8.	Error: Duplicate resource configuration	Two resources with same name in same module.	Rename one of the resources or use separate modules.	resource "aws_s3_bucket" "b1" ... resource "aws_s3_bucket" "b1" ... (duplicate)
9.	Error: Error locking state	State file locked by another process/user.	Wait and retry or manually unlock the state.	terraform force-unlock <LOCK_ID> (Use with caution)
10.	Error: State snapshot was created by a newer version of Terraform	Terraform version mismatch (state is newer).	Upgrade your Terraform to same or higher version.	Check version with: terraform version
11.	Error: Missing required argument	Required argument for resource is missing.	Add the missing argument as per documentation.	e.g. ami in aws_instance is required.
12.	Error: Forbidden	IAM permissions not sufficient.	Attach proper IAM policy or permissions.	Check CloudTrail / IAM Access Analyzer.
13.	Error: 409 Conflict	Resource already exists with same name.	Import existing resource or change the name.	terraform import <resource> <resource_id>
14.	Error: Context deadline exceeded	Timeout while creating resource.	Increase timeout or check network / service limits.	Use timeouts {} block in resource if needed.
15.	Error: Provider produced inconsistent result	Provider bug or API change.	Retry the apply. If persists, upgrade provider or report the issue.	terraform init -upgrade terraform apply

Debugging Tips

- Run terraform plan -out=plan.out
- Use terraform show plan.out
- Enable logs: TF_LOG=DEBUG terraform apply
- Validate configuration: terraform validate
- Check docs: registry.terraform.io

Golden Rule

Read the error carefully,
Understand the cause,
Apply the right fix,
And Re-run with confidence! ✓

My Checklist

- ☑ terraform init
- ☑ terraform validate
- ☑ terraform plan
- ☑ terraform apply
- ☑ Verify in cloud console

Understand Errors → Fix Fast → Deploy Confidently! ☑



Engineer_cloudops

Module 16: Interview & Troubleshooting

STATE LOCK ISSUES

Engineer_cloudops

What is State Lock?

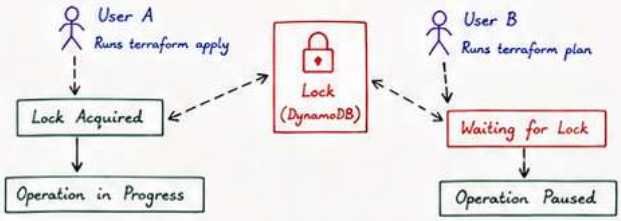
Terraform uses state locking to prevent multiple users/processes from changing the state file at the same time. It ensures state consistency and avoids corruption.

Why State Lock is Important?

- ✓ Prevents concurrent operations.
- ✓ Avoids state file corruption.
- ✓ Ensures one operation at a time.
- ✓ Protects infrastructure from inconsistent changes.

How State Lock Works?

1. When you run `terraform plan/apply/destroy`, Terraform tries to acquire a lock on the state.
2. Lock info is stored in the backend (S3 + DynamoDB, etc).
3. Other operations wait until the lock is released.
4. Lock is released automatically when operation completes.



Common State Lock Issues

1. Previous operation was interrupted (CTRL+C, system crash).
2. Network failure during apply/plan.
3. User forgot to release lock.
4. Stale lock left in DynamoDB.
5. Manual changes in backend.

Identify Lock

1. Check if lock exists in DynamoDB

```
aws dynamodb scan \
  --table-name <your-lock-table> \
  --query "Items[].LockID" --output table
```

2. Or use Terraform command

```
terraform force-unlock -force <LOCK_ID>
  (after verifying the lock)
```

3. Lock Information Example (DynamoDB Item)

```
{
 "LockID": "abc123-xyz",
 "Info": "{"ID":"aaa","Operation":"OperationTypeApply",
      "Who":"user@company.com","Version":"1.6.3","Created":"2024-05-20T10:15:00Z"}",
 "LeaseDuration": 300,
 "AcquireTime": "2024-05-20T10:15:00Z"
}
```

How to Fix State Lock Issues?

1. Verify no running Terraform process.
   - Check CI/CD, other users, or local terminal.
2. Check the lock info in backend (DynamoDB, etc).
3. If it's a stale lock → remove it.
4. Run force-unlock if needed.

Important:

- Never force-unlock if another user is still running Terraform.
- It can cause state corruption.
- Always confirm before removing lock.

Force Unlock

Command:

```
terraform force-unlock <LOCK_ID>
```

Example:

```
terraform force-unlock abc123-xyz
```

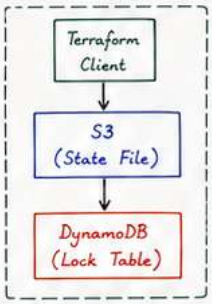
Manual Unlock in DynamoDB (If Needed)

1. Go to DynamoDB Console.
2. Open your Terraform lock table.
3. Find the lock item (check Info -> Who, Operation, Created).
4. Delete the item (if it's stale and no operation in progress).

```
Table Name Example: terraform-locks
Partition Key: LockID (String)
```

Prevent State Lock Issues (Best Practices)

- ✓ Use remote backend with locking (S3 + DynamoDB).
- ✓ Set proper IAM permissions.
- ✓ Avoid running multiple operations simultaneously.
- ✓ Use CI/CD queue or approval process.
- ✓ Monitor running jobs.
- ✓ Don't interrupt operations.
- ✓ Enable logging and alerts.



Backend Example with Locking (S3 + DynamoDB)

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "env/prod/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

← State file in S3

← Lock info in DynamoDB

Quick Checklist When You Face Lock Issue

- ✓ Check if any Terraform process is running.
- ✓ Verify lock in DynamoDB.
- ✓ If stale → remove or force-unlock.
- ✓ Run terraform plan to confirm it's working.

Lock Smartly, Deploy Safely! ☑

Engineer_cloudops

Module 16: Interview & Troubleshooting

DEPENDENCY PROBLEMS (Terraform)

Engineer_cloudops

What is Dependency Problem?

When Terraform tries to create/update/destroy resources in wrong order because dependencies are not clear, it leads to errors.

Why Dependencies are Important?

- ✓ Resources rely on other resources.
- ✓ Wrong order = Failures.
- ✓ Explicit dependencies avoid race conditions.
- ✓ Ensures stable and predictable deployments.

Common Dependency Problems

1. Resource not found (depends on not-yet-created resource)
2. Circular dependency
3. Implicit dependency not recognized
4. Module dependency issues
5. Data source depends on resource not ready

1. Resource not found

Error Example:

```
Error: Error creating S3 Bucket Policy:
  NoSuchBucket: The specified bucket does not exist
```

Cause:

Bucket policy tried to attach before the S3 bucket was created.

Fix:

Use implicit dependency (reference) or depends_on.

```
resource "aws_s3_bucket" "app" {
  bucket = "my-app-bucket"
}
resource "aws_s3_bucket_policy" "app_policy" {
  bucket = aws_s3_bucket.app.id
  policy = data.aws_iam_policy_document.policy.json
  depends_on = [aws_s3_bucket.app]  # explicit (if needed)
}
```

2. Circular Dependency

Error Example:

```
Error: Cycle: aws_security_group.sg,
       aws_instance.web
```

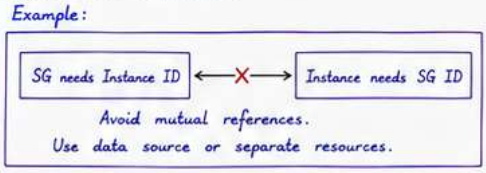
Cause:

Resource A depends on B and B depends on A.

Fix:

Break the cycle. Avoid using each other's attributes. Refactor the configuration.

Example:



Avoid mutual references.
Use data source or separate resources.

3. Implicit dependency not recognized

Error Example:

```
Error: InvalidParameter: Subnet not found
```

Cause:

Terraform didn't understand that the subnet depends on the VPC (no reference used).

Fix:

Reference the resource so Terraform can infer dependency.

Example:

```
resource "aws_vpc" "vpc" {
  cidr_block = "10.0.0.0/16"
}
resource "aws_subnet" "subnet" {
  vpc_id     = aws_vpc.vpc.id  # implicit dependency
  cidr_block = "10.0.1.0/24"
}
```

4. Module Dependency Issues

Problem:

Module B uses output from Module A, but A is not applied or referenced.

Fix:

Use module outputs and depends_on if required.

Example:

```
module "vpc" {
  source = "./modules/vpc"
}
module "ec2" {
  source     = "./modules/ec2"
  subnet_id  = module.vpc.public_subnet_id
  depends_on = [module.vpc] # if needed
}
```

5. Data source depends on resource

Error Example:

```
Error: InvalidID.NotFound: The resource
  does not exist
```

Cause:

Data source tries to read a resource that will be created in the same run.

Fix:

Use depends_on or avoid using data source before resource creation.

Example:

```
data "aws_ami" "app" {
  most_recent = true
  owners      = ["self"]
  depends_on  = [aws_ami.my_custom] # when needed
}
```

How Terraform Handles Dependencies?

- → Automatically builds a dependency graph.
- → Uses references (implicit) to determine order.
- → Use depends_on for explicit control.
- → Creates → Updates → Destroys in correct order.

Best Practices

- ✓ Always use references instead of hardcoding IDs.
- ✓ Use depends_on only when necessary.
- ✓ Keep modules loosely coupled.
- ✓ Avoid circular dependencies.
- ✓ Run terraform plan and review the graph.

Check Dependency Graph

1. terraform graph
2. terraform graph | dot -Tpng > graph.png
3. Review using tools like Graphviz

Real World Example

Scenario:

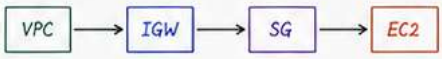
EC2 instance needs Security Group,
SG needs VPC, VPC needs Internet Gateway.

Wrong Order:

EC2 → SG → VPC → IGW (Fail)

Correct Order:

VPC → IGW → SG → EC2 (Success)



Troubleshooting Checklist

- Check the error message carefully.
- Identify which resource is missing.
- Check references and variable values.
- Look for circular dependencies.
- Use terraform graph to visualize.
- Add depends_on if required.
- Re-run terraform plan.

Quick Commands

- terraform graph
- terraform plan -out=plan.out
- terraform apply plan.out
- terraform state list
- terraform state show <resource>
- terraform state rm <resource> (use carefully)

Right Order, Happy Terraform! ☑

Engineer_cloudops

Module 16: Interview & Troubleshooting

RESOURCE DRIFT (Terraform)

Engineer_cloudops

What is Resource Drift?

Resource Drift happens when the actual infrastructure in the cloud is different from what is defined in your Terraform configuration or stored in the state file. This usually occurs when someone makes changes outside of Terraform.

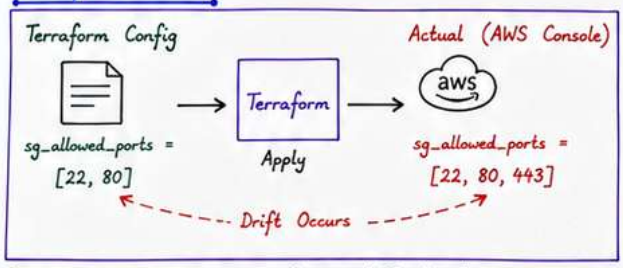
Why is it a Problem?

- ✓ Terraform plan will show unexpected changes.
- ✓ Can lead to failed applies or accidental overwrites.
- ✓ Infrastructure becomes unpredictable.
- ✓ Compliance and security risks.

Common Causes of Resource Drift

1. Manual changes in AWS Console / CLI.
2. Changes by other teams or tools (CloudFormation, etc).
3. Default values changed by cloud provider.
4. Out-of-band updates (backup scripts, automation).
5. State file not updated or locked properly.

Example Scenario



Port 443 was added manually in AWS Console.
Terraform is not aware of this change.

How to Detect Resource Drift?

1. Run: terraform plan
   → Shows differences between config, state and real infra.
2. Use: terraform refresh (deprecated in v0.15+, use plan)
3. Check in Cloud Console / CLI manually.
4. Use Drift detection tools (e.g., Steampipe, TF Drift, CloudCustodian)

How to Fix Resource Drift?

1. Review the changes shown in terraform plan.
2. If changes are valid, update the Terraform config.
3. If changes are not valid, revert manually or with Terraform.
4. Run terraform apply to bring infra back in desired state.
5. Always communicate with team before making manual changes.

Quick Checklist

- ✓ Run terraform plan regularly.
- ✓ Avoid manual changes in cloud console.
- ✓ Use remote state with locking.
- ✓ Document allowed exceptions.
- ✓ Automate as much as possible.

Example: Drift Detection Output

```
$ terraform plan
...
~ resource "aws_security_group" "web_sg" {
  id = "sg-0abc123"
  ingress = [
    { from_port = 22, to_port = 22, protocol = "tcp" }
    { from_port = 80, to_port = 80, protocol = "tcp" }
  - { from_port = 443, to_port = 443, protocol = "tcp" } # exists in real
  + { from_port = 443, to_port = 443, protocol = "tcp" } # will be removed
  ]
...
Plan: 0 to add, 1 to change, 0 to destroy.
```

Best Practices to Prevent Drift

1. Use Infrastructure as Code for everything.
2. Restrict access to cloud console (least privilege).
3. Use IAM policies / SCPs to prevent unauthorized changes.
4. Enable CloudTrail and Config for monitoring.
5. Use Terraform Cloud/Remote backend with state locking.
6. Run plan in CI/CD pipelines and alert on drift.

Detect Drift in CI/CD (Example)

```
# GitHub Actions Example
- name: Terraform Plan
    run: terraform plan -detailed-exitcode -out=plan.out
- name: Check Drift
  run: |
       terraform show -no-color plan.out
       # exit code 2 means drift detected
       if [ $? -eq 2 ]; then
         echo "Drift Detected! Please review."
         exit 1
       fi
```

Tools to Detect & Manage Drift

- Steampipe (steampipe query aws_config)
- AWS Config + CloudTrail
- Terraform Cloud (Drift detection)
- TFDrift / Driftctl
- Spacelift / Atlantis

Key Takeaway

Drift is not just a technical issue, it's a process issue.
People + Process + Tools = Drift Free Infrastructure.

No Drift, No Surprises! Keep Infra in Sync ☑

EC Engineer_cloudops

Module 16: Interview & Troubleshooting

PROVIDER VERSION CONFLICTS (Terraform)

Engineer_cloudops

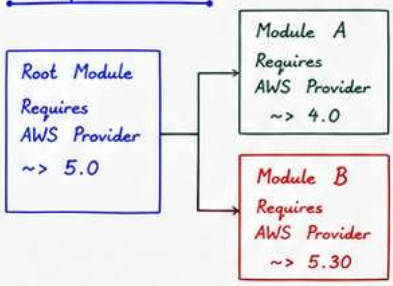
What is Provider Version Conflict?

Provider version conflict happens when different modules or configurations require different versions of the same provider. Terraform can't satisfy both, leading to errors during init or plan.

Why it Happens?

- ✓ Root module and child modules specify different versions.
- ✓ Multiple modules with incompatible version constraints.
- ✓ Using outdated provider in one module.
- ✓ Directly specifying providers in modules (not recommended).
- ✓ Not using required_providers + version constraints properly.

Example Scenario



Conflict:
Module A wants 4.x but Root wants 5.x
Terraform can't install both.

Common Error Messages

- Error: Failed to query available provider packages
- Error: Incompatible provider version
- Error: Could not satisfy version constraints
- Error: Locked provider does not match configured version

How to Fix Provider Version Conflicts?

1. **Define version in root module only**
   Let root module control provider versions.
   Remove provider blocks from child modules.
2. **Use required_providers with version constraints**
   Use compatible version constraints (>=, ~>)
3. **Update modules to use newer provider**
   Upgrade old modules to support the same version.
4. **Run terraform init -upgrade**
   Forces Terraform to re-download the correct version.
5. **Use provider lock file (.terraform.lock.hcl)**
   Commit the lock file to ensure same provider for all.
6. **Check module registry or source for compatibility**
   Some modules may not support latest provider.

Best Practices

- ✓ Define provider versions only in root module.
- ✓ Use ~> (pessimistic constraint) for stability.
- ✓ Regularly update providers.
- ✓ Test modules with latest provider versions.
- ✓ Commit .terraform.lock.hcl to version control.

Example: Provider Conflict

Root Module (main.tf)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"  # Root wants 5.x
    }
  }
}
```

Child Module A (modules/vpc/versions.tf)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 4.0"  # Module wants 4.x (conflict)
    }
  }
}
```

Child Module B (modules/iam/versions.tf)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.30"  # Compatible with root
    }
  }
}
```

Solution: Define Provider in Root Only

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.30"  # Use a version that satisfies all
    }
  }
}
```

In Child Modules: Remove provider block completely.
Child modules will inherit provider from root.

Check Installed Providers

```
$ terraform providers
$ terraform providers | grep aws
```

← Shows which versions are being used.

Steps to Resolve (Quick Checklist)

- ✓ Run: terraform init -upgrade
- ✓ Check provider constraints in all modules.
- ✓ Remove provider blocks from child modules.
- ✓ Align version constraints.
- ✓ Run: terraform plan

One Provider, One Version, One Truth! ☑

Engineer_cloudops

Module 16: Interview & Troubleshooting

MODULE DEBUGGING (Terraform)

Engineer_cloudops

Why Debugging Modules?

Modules help us reuse code, but when something fails inside a module, it's not always obvious where the issue is.

Common Issues:

- ✓ Wrong variable passed
- ✓ Missing output
- ✓ Conditional logic not working
- ✓ for_each / count issues
- ✓ Provider / permission errors

1. Understand the Error

Read the error carefully. Terraform usually tells the resource + line number.

Example Error:

```
Error: Invalid value for variable
on modules/vpc/main.tf line 12,
in variable "cidr":
The given value is not a valid CIDR block.
```

2. Add Debug Outputs

Add temporary outputs inside the module to see what values are being passed.

```
# modules/vpc/outputs.tf
output "debug_cidr" {
  value = var.vpc_cidr
}
```

Then run:

```
terraform plan -out=tfplan
terraform show tfplan | grep debug_cidr
```

3. Use terraform console

Best tool to test expressions and variables.

```
$ terraform console
> var.vpc_cidr
  "10.0.0.0/16"
> local_name
  "my-vpc"
> can(regex("^10", var.vpc_cidr))
  true
```

Quick Checklist

- ✓ Read the error fully
- ✓ Check variable values
- ✓ Use outputs / console
- ✓ Validate expressions
- ✓ Use plan, graph, logs
- ✓ Fix and re-run plan

4. Validate Variables

Add validations in variables.tf to catch issues early.

```
variable "cidr" {
  type = string
  validation {
    condition     = can(cidrnetmask(var.cidr))
    error_message = "CIDR must be valid."
  }
}
```

5. Check terraform plan

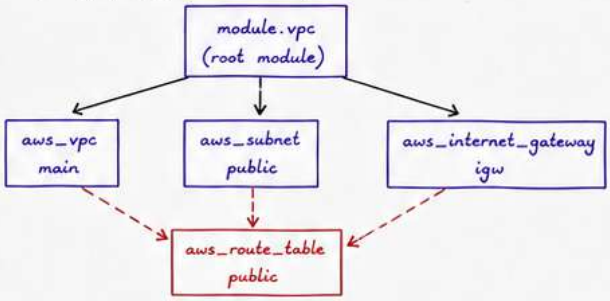
- Look for resources that are planned to be created / changed.
- If something looks unexpected, check the input values.
- Use -out and show to inspect the plan in detail.

6. Visualize Dependency

Use graph to understand dependency and flow.

```
terraform graph | dot -Tpng > graph.png
```

Open graph.png to see the flow.



7. Enable Detailed Logs

Enable logs to get more insight.

```
TF_LOG=DEBUG terraform plan 2>&1 | tee tfdebug.log
```

Then search in tfdebug.log for more info.

8. Isolate the Module

Create a separate test config just for that module. This helps to reproduce and debug easily.

```
module "vpc" {
  source   = "../modules/vpc"
  vpc_cidr = "10.0.0.0/16"
  env      = "dev"
}
```

← Minimal config to test the module in isolation.

Pro Tips

- ✓ Use meaningful variable names and descriptions.
- ✓ Keep modules small and focused.
- ✓ Add validations and default values.
- ✓ Use pre-commit (tflint, terraform fmt, checkov).

Debug Smart, Deploy Confidently! ☑

EC Engineer_cloudops

Module 16: Interview & Troubleshooting

REAL-TIME PRODUCTION SCENARIOS (Terraform)

Engineer_cloudops

Real incidents we handle in Production

These are real-world scenarios we face while managing infrastructure with Terraform. How we troubleshoot and fix them is more important than just knowing the theory.

1. Accidentally deleted production resource

Problem: Someone ran "terraform destroy" or removed a resource from config and applied.

Impact: Resource is deleted (e.g., S3 bucket, RDS, EC2)

Fix:

- ✓ Check if resource can be restored from backup.
- ✓ If not, re-create using Terraform.
- ✓ If state is gone - import if resource still exists.

```
terraform import aws_s3_bucket.mybucket my-bucket-name
```

Prevention:

- ✓ Use separate workspaces for prod.
- ✓ Enable MFA Delete, deletion protection.
- ✓ Use "-target" carefully and with approval.

2. Terraform apply fails in production

Problem: Plan looks good, but apply fails in the middle.

Common Reasons:

- > IAM permission denied
- > Quota limits
- > Resource already exists / conflicts
- > Network / API timeouts

Fix:

- ✓ Read the error fully.
- ✓ Fix the root cause (permissions, limits, config).
- ✓ Re-run "terraform apply".
- ✓ If partial changes happened - run plan again.

```
terraform plan -out=tfplan
terraform apply tfplan
```

Prevention:

- ✓ Run plan and review carefully.
- ✓ Use CI/CD with policy checks.
- ✓ Monitor quotas and limits.

3. State file is corrupted or lost

Problem: State file is corrupted, manual changes or deleted.

Impact: Terraform loses track of real infra.

Fix:

- ✓ Restore from latest backup (S3 versioning).
- ✓ If no backup - recreate state manually using import.
- ✓ Verify all resources with "terraform plan".

Prevention:

- ✓ Enable S3 versioning + DynamoDB locking.
- ✓ Never edit state file manually.
- ✓ Take regular state backups.

4. Resource Drift Detected in Production

Problem: Someone changed a resource manually in console.

Example: Changed EC2 instance type from t3.medium to t3.large.

Fix:

- ✓ Run "terraform plan" to see the drift.
- ✓ Decide: keep manual change or revert it.
  - → To keep: Import or update the config to match.
  - → To revert: Apply the original config.

```
terraform plan
terraform apply
```

Prevention:

- ✓ Restrict console access.
- ✓ Enable CloudTrail & Config rules.
- ✓ Use Terraform for all changes.

5. Module / Provider Version Conflict

Problem: Different modules or providers require different versions.

Error Example:

```
Error: Incompatible provider version
Module A requires aws "~> 4.0
Module B requires aws "~> 5.0
```

Fix:

- ✓ Check each module's required_providers block.
- ✓ Use compatible version that satisfies all.
- ✓ Update modules if required.

```
terraform init -upgrade
```

Prevention:

- ✓ Lock provider versions in root module.
- ✓ Test module compatibility in lower env.

6. Lock Error: Error acquiring the state lock

Problem: Another Terraform process is running / crashed.

Error:

```
Error acquiring the state lock
Lock Info:
  ID:        1a2b3c4d-5678-90ab-cdef-1234567890ab
  Operation: OperationTypeApply
  Who:       user@company.com
  Version:   1.6.2
  Created:   2024-05-20 10:30:00
```

Fix:

- ✓ Check if any Terraform process is running.
- ✓ If not, remove lock from DynamoDB.
- ✓ Or use "terraform force-unlock <LOCK_ID>"

Golden Rules (Always Follow)

- ✓ Always run plan before apply
- ✓ Never run on production directly
- ✓ Protect your state file
- ✓ Use workspaces
- ✓ Take backups
- ✓ Automate checks in CI/CD
- ✓ Document your infra
- ✓ Monitor & Alert
- ✓ Think before you apply

Real experience + Good process = Production confidence ☑

Engineer_cloudops